

Shaoying Liu

Agile-SOFL

Agile Formal Engineering Method

Agile-SOFL

Shaoying Liu

Agile-SOFL

Agile Formal Engineering Method

Shaoying Liu
Graduate School of Advanced Science
and Engineering
Hiroshima University
Higashihiroshima, Hiroshima, Japan

ISBN 978-981-97-2284-6 ISBN 978-981-97-2285-3 (eBook)
<https://doi.org/10.1007/978-981-97-2285-3>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2024

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

Paper in this product is recyclable.

To my family with great gratitude, in particular to my wife Atsuko. It would be difficult for me to complete this book without her heartfelt support all the time.

Preface

Software productivity and reliability are two critical properties for software development projects, but how to guarantee both of them to a high standard has been and is still a significant challenge. Productivity is usually emphasized by companies since it is concerned with the cost of software development, while reliability is mostly the interest of customers because they are likely to suffer from low reliability of software systems in operation.

Several approaches to developing software systems have been well studied by the research community and applied by industry, but most of them can hardly provide assurance for both productivity and reliability in practice. The key problem is that none of them can efficiently deal with the evolution of software during both development and maintenance processes, especially for large-scale systems. Software evolution is an inherent nature of software projects due to the fact that almost all development activities have to be done by humans. Humans need communications and their abilities of managing communications and evolutions vary in practice, which makes their activities difficult to control and predict.

Model-Driven Development (MDD) is a well-researched method that emphasizes the importance of building an abstract model of the potential system before coding. While the model may allow the developer to have a preliminary understanding of the system, building it with a high quality usually requires considerable efforts and takes time. Even suppose the model is well built, it has to be evolved (changed) from time to time during the development process for many reasons (e.g., the customer's requirements changes, the developer's inaccurate understanding, missing information). Unfortunately, the evolution of models, especially for those represented graphically like the Unified Modeling Language (UML), is usually time-consuming because the developer will have to manage the graphical representation within a limited space. Furthermore, it is rare in practice to see no changes of the model during coding, but to manage the changes to keep consistency with the model will take time and needs efforts that almost every developer would like to avoid if possible.

Formal methods are intended to even go further, taking advantage of formal proof for a rigorous verification of program correctness or other properties. To fulfill this goal, accurate formal specifications or properties are required to form in advance.

Since both writing formal specifications and carrying out formal verifications have to be done by humans, frequent evolutions of the specification and the implemented code as well as the process of verification must be unavoidable. This would make the formal methods too difficult and expensive to use in practice, especially for large-scale and complex systems. Of course, the limitations of the capability of formal methods in managing modern software systems would make the situation even worse.

Component-based software development paradigm presents another hope to address the challenge in assuring software productivity and reliability since well-developed software components can be utilized to efficiently construct new systems. However, this approach is limited in the sense that it can only work well when software components possess a simple functionality and be small in scale. The reason is that when components are large, the developer would face a challenge to accurately understand their functionality and to make a correct judgement on whether they well fit for the situation of the new system under construction. Ensuring the reliability of large-scale components is also a challenge, almost the same as for developing general application software systems. Just in case an unreliable component is deployed in the new system without being noticed, the reliability of the entire system will be affected. In contrast, small components are relatively easier to develop and their reliability is easier to ensure. That is why more and more software packages with essential software components for specific programming languages have been developed and used in practice.

Agile software development approach gives a full consideration of the evolutionary nature of software development and values four principles in the Agile manifesto: individuals and interactions, working software, customer collaboration, and responding to change. Using the Agile approach, a software prototype is quickly built as a running model based on a preliminary understanding of the customer's requirements to facilitate the communication with the customer in order for the developer to gain a comprehensive and more accurate understanding of the requirements than before. The prototype is then evolved by modifying the existing functionality and/or by adding more functions to cope with the new progresses gained through the communications. Repeating this process will eventually make the software system mature enough for the delivery to the customer. While this approach may be appropriate for realistic software development, it faces several challenges. One challenge comes from the risk of having to carry out big changes in the system architecture, data structures, or algorithms due to the lack of sufficient analysis and consideration before the beginning of coding. Another challenge is the risk of chaotic changes of the code because of the lack of a well-recorded traceability to show clearly the responsibility of each developer involved and how the final code is constructed. Further, when the current code is evolved, it must first be read and understood by the developer. While this may be relatively easy for a small-scale code, it would be considerably difficult when the code is large and complex.

Having read the above, the reader must be wondering whether there is any better solution for the problem than the existing development approaches. This book aims to provide the reader with a satisfactory solution by systematically introducing a

more effective Agile development method, known as Agile-SOFL. SOFL stands for Structured Object-oriented Formal Language. It is designed by the author of this book for the writing of comprehensible formal specifications for software systems in the late 1990s. Agile-SOFL is intended to offer a highly practical and systematic method for Agile development using SOFL, striking a good balance of efforts for enhancing both software productivity and reliability.

Specifically, Agile-SOFL is characterized by five features: (1) systematic approach to constructing hybrid specifications for requirements-related faults prevention, (2) specification-based incremental programming for quality implementation, (3) specification-based inspection and testing for system validation, (4) automatic testing-based formal verification for the correctness of code, and (5) effective project management for high effectiveness and efficiency in applying Agile-SOFL. The details of each feature and its related theory and technologies can be found in the relevant chapters of this book.

After introducing the fundamental concepts and the general idea of the Agile formal engineering method Agile-SOFL, many of which differ from what we understand in existing development paradigms, this book provides a detailed, systematic description and discussion of all the techniques involved. The techniques result from the intensive research of my group in the field over the last fifteen years and are properly organized to support the unique principle of Agile-SOFL. Some of them are already tried by the industry while some others present new progresses for future software engineering. To help readers study Agile-SOFL easily, I have tried to make the descriptions as precise and comprehensible as possible. I have also tried to avoid unnecessary formal descriptions to help students and practitioners easily learn Agile-SOFL. Numerous examples are given throughout the book to help the explanation of the techniques involved and readers improve their understanding of the material they have studied.

The objective of this book is to help readers learn the latest effective Agile development concepts and techniques and to take them to the point where they can easily apply Agile-SOFL to develop their own software systems with a high productivity and reliability. The new concepts include *the three-step hybrid specification approach*, *the role of writing the specification*, *the relationship between the specification and the implementation*, *the specification-based incremental programming*, *the specification-based validation and verification*, and *the total-football development management*. Although these concepts may share the same English words used in some of existing development paradigms, their connotation differs considerably in a way that they can significantly facilitate developers to enhance both software productivity and reliability.

Audience

This book is written for students who wish to update their knowledge on the state-of-the-art development technologies, researchers who wish to find new topics to work on in software engineering, and practitioners who want to learn new knowledge and skills for efficiently developing highly reliable systems. The readers who are

interested in finding methods for quickly constructing high quality software but feel frustrated or disappointed at the results so far, will greatly benefit from this book. Experience with any programming languages and basic knowledge on the first-order predicate logic would help readers appreciate and digest the contents of this book.

Using This Book

This book can be used as a textbook for teaching or training in three ways. Firstly, Chaps. 1 to 4 can be used to teach a course on *Practical Software Specification and Programming*. Any level of the undergraduate students can learn this course as long as they have some experience of programming and knowledge on the first-order predicate logic. Secondly, Chaps. 5 to 7 provide sufficient materials for a course on *Specification-Based Software Validation and Verification* for graduate students. Finally, The whole chapters in the book can be used for a course on *Agile Formal Engineering Method* for both undergraduate and graduate students.

The book can also be used as a reference book to support the study of other related courses or individual study of the Agile formal engineering method Agile-SOFL for software development. To make the book easy to use, I have organized the materials into ten chapters:

Introduction. Chapter 1 explains the general principle of Agile-SOFL and the basic concepts involved, including specification, hybrid specification, programming, software evolution, and specification-based validation and verification.

Introduction to SOFL. Chapter 2 gives a brief introduction to the SOFL specification language based on which Agile-SOFL is established. Since most of the elements in SOFL are inherited by Agile-SOFL, obtaining basic knowledge about SOFL is necessary and helpful for studying the contents of this book.

Informal Specification. Chapter 3 describes the notion of informal specification by explaining its purpose and structure as well as basic techniques for writing good quality informal specifications.

Hybrid Specification. Chapter 4 elaborates on the important concept of hybrid specification adopted in Agile-SOFL. A hybrid specification properly integrates various level specifications, including semi-formal specification, formal specification, and human-machine interface specification.

The Three-Step Hybrid Specification Approach. Chapter 5 discusses how a hybrid specification can be constructed by taking a natural evolutionary approach. A comprehensible example is provided to facilitate the reader to understand the steps and the principle of the approach.

Incremental Programming from Specification. Chapter 6 discusses how an incremental approach to programming from a hybrid specification can be taken to efficiently construct a program that implements the specification.

Specification-Based Inspection. Chapter 7 discusses a theory and the related techniques for inspection of the code against the specification it implements.

Specification-Based Testing. Chapter 8 presents how the specification can be used effectively to facilitate the validation of the code implemented based on the specification. In the context of Agile-SOFL, specification-based testing is mainly used for validation of both specification and code.

Testing-Based Formal Verification. Chapter 9 describes a rigorous technique for formally verifying the correctness of code with respect to its specification. This technique utilizes testing to find out the target for formal verification and can be fully automated with a tool support.

Software Project Management. Chapter 10 discusses managerial issues in relation to applying Agile-SOFL in various practical situations.

Higashihiroshima, Japan

Shaoying Liu

Acknowledgements

The development of Agile-SOFL has benefited from numerous discussions with my students and industrial practitioners during the period of 2008 to 2023. In the descriptions throughout the book, the SOFL specification language is often used. I take the style of introducing the relevant parts of the SOFL language whenever they are necessary for the reader to understand the issues discussed. A detailed introduction to the SOFL language can be found in my previous book [19]. Some parts of this book are more comprehensible presentations of the techniques reported in our recent publications in journals or conferences. Finally, my thanks go to all the colleagues and students who have read the draft of this book for their constructive comments and suggestions, and the editor Mio Sugino for her encouragement and suggestions that helped me write this book.

Contents

1	Introduction	1
1.1	Model-Driven Development	2
1.2	Formal Methods	3
1.3	Agile Development	4
1.4	Agile Formal Engineering Method	5
	References	8
2	Introduction to SOFL	9
2.1	General Structure of SOFL Specification	9
2.1.1	Module and CDFD	9
2.1.2	Hierarchical CDFDs and Modules	13
2.2	SOFL Logic	16
2.2.1	Basic Predicate Expressions	17
2.2.2	Compound Predicate Expressions	18
2.3	Data Types in SOFL	19
2.3.1	Basic Types	19
2.3.2	Set Types	22
2.3.3	Sequence and String Types	24
2.3.4	Composite Types	26
2.3.5	Product Types	27
2.3.6	Map Types	28
2.3.7	Union Types	30
2.4	Other Components in SOFL	31
	Reference	31
3	Informal Specification	33
3.1	The Structure	33
3.2	Example	34
3.3	Completeness	35

4	Hybrid Specification	37
4.1	Semi-formal Specification	37
4.1.1	Structure and Characteristics	38
4.1.2	Process Specification	39
4.1.3	Condition Data Flow Diagram (CDFD)	40
4.1.4	Hierarchy of Modules	40
4.1.5	Example of Semi-formal Specification	41
4.2	User Interface Specification	44
4.2.1	Focus of GUI Specification	45
4.2.2	GUI Elements	45
4.2.3	Structure of GUI Specification	46
4.2.4	Example of GUI Specification	46
4.2.5	Formal Specification	49
5	Three-Step Hybrid Specification Approach	51
5.1	Informal Specification	52
5.1.1	Describing Functions	53
5.1.2	Describing Data	54
5.1.3	Describing Constraints	54
5.1.4	Building Connections	54
5.1.5	Example	55
5.2	Semi-formal Specification + GUI Specification	57
5.2.1	Techniques for Semi-formal Specification	57
5.2.2	Example of Semi-formal Specification	59
5.2.3	Techniques for GUI Specification	68
5.3	Formal Specification	69
5.3.1	Preparing Skills for Formalization	70
5.3.2	Handling Quantified Predicate Expressions	70
5.3.3	Example of Formal Specification	71
6	Specification-Based Implementation	73
6.1	Module-Based Incremental Implementation	73
6.1.1	Strategy for Module Selection	74
6.1.2	Module Implementation	74
6.2	Iterative Coding	76
6.3	Building Trace Links	78
6.3.1	Trace Links Table	78
6.3.2	Comprehensibility of Trace Link Table	79
7	Specification-Based Inspection	81
7.1	Essential Ideas of FSBI	81
7.2	Inspection Process	84
7.3	An Example	87
7.3.1	Specification of PSS	87
7.3.2	Implementation of PSS	89
7.3.3	Linking Scenarios to Paths in PSS	91

7.3.4	Inspecting Paths	95
7.3.5	Creating Inspection Report	96
	References	96
8	Specification-Based Testing	99
8.1	Specification-Based Testing for Validation	100
8.2	Framework of FSBT	101
8.3	Test Data Generation	103
8.3.1	Test Data Generation Criteria	104
8.3.2	Example	105
8.3.3	Test Data Generation Algorithms	107
8.3.4	“Vibration” Testing Method—V-Method	116
8.4	Test Oracle Derivation	119
8.5	Practical Issues About FSBT	119
8.5.1	Specification Language	120
8.5.2	Feasibility of Automation	120
8.5.3	Usage of Test Data and Test Oracle	120
8.5.4	Software Tools for FSBT	121
8.5.5	Ultimate Goal of FSBT	123
	References	123
9	Testing-Based Formal Verification	125
9.1	The Principle of TBFV	126
9.1.1	Backward Symbolic Execution (BSE)	126
9.1.2	Forward Symbolic Execution (FSE)	127
9.1.3	Theorem Proving	128
9.2	Examples	129
9.2.1	Buy Railway Ticket	129
9.2.2	Search an Element	133
9.3	Practical Issues	137
	References	138
10	Software Project Management	139
10.1	“Total Football” Method	139
10.2	Novel View on the Role of Documents	140
10.3	Management for Outsourced Development	141
10.4	Flexibility in Using Techniques	142

Chapter 1

Introduction



Software development is an evolutionary process by nature since it must be developed by humans. Evolution is a natural process for humans to learn or create new things, such as writing a novel, painting a picture, and developing a software system. In the context of software development, evolution basically means changes with the attempt to improve the current understanding and expression of software artifacts that are documents produced during the development.

There are three necessary and important activities in software development: *understanding*, *creation*, and *confirmation*. The software developer must first understand the customer's requirements for the potential system. On the basis of this understanding, the developer then needs to create a system as the technical solution. Since the creation has to be done by humans, who can hardly escape from making mistakes, the created solution must be confirmed to ensure that it does satisfy the customer's requirements.

Understanding the customer's requirements is essential and critical for the success of the whole development project because a wrong interpretation of the requirements will not lead to a satisfactory solution. There are several ways to capture the requirements. One is to write a document to define the requirements. Such a document is called *specification*. In general, a specification focuses on what to be done under what constraints by the potential system, it usually does not deal with the issue of how requirements are realized by algorithms. Writing a specification involves evolution of the specification because it is expected to reflect the customer's perception of requirements that can hardly captured at one time.

Creation is involved in the design of software, including the architecture, data structures, and efficient algorithms. Software design is also an evolutionary process that keeps improving the current version of design.

Confirmation needs to be carried out to check whether the design and the implementation of the system contains any bugs (or defects). Bugs can be roughly classified into two categories: *requirement-related bugs* and *implementation-related bugs*. Requirement-related bugs refer to those that will cause the distortion of the required functionality. For instance, one-time withdrawal from an automated teller machine

(ATM) is required to be less than 200,000 Japanese yen (JPY), but it is implemented wrongly as less than 2,000,000 JPY in the program. Implemented-related bugs usually refer to those caused by inappropriate use of the programming language that could lead to failures of normal operations of the system. For example, most of the exceptions defined in the Exception class hierarchy in Java, such as `ArrayIndexOutOfBoundsException` and `ArithmeticException`, can be regarded as implementation-related bugs. Both categories of bugs are the threats to software reliability, and therefore they must be prevented and detected for the final software product.

However, fulfilling this goal is rather difficult with existing software development methods and technologies, as the discussions later in this chapter shows. This poses a great challenge to software engineering in both research and practice. Many software development paradigms and specific methods and technologies have been proposed and developed over the last six decades, but how they can be effectively and efficiently applied in practice is still a question for practitioners. Below I will discuss the three most popular development paradigms by explaining their motivation, merits, and demerits. The three paradigms are Model-Driven Development, Formal Methods, and Agile Software Development. As a matter of fact, there are many more specific development methods and technologies proposed in the literature, such as Domain-Driven Development and Incremental Development, but it seems that all of them can be classified into one of these three paradigms or their combinations. Therefore, as long as we understand the merits and demerits of these three paradigms, we will be able to understand the others.

1.1 Model-Driven Development

Model-Driven Development (MDD) is characterized by the fact that a model of the potential system is first constructed and then its implementation is carried out based on the model. A model of the system is intended to describe the system in an abstract manner. It is often the case that the model of a system is composed of a collection of representations, each describing a different aspect of the system. The model can be represented in graphical languages, natural languages, or their combination.

MDD can be used in both Structured Methods (SM) [1] and Object-Oriented Methods (OOM) [2]. In the SM, the emphasis is usually put on the design of operations and their connections, as well as necessary data items. The representation of a model can be data flow diagrams, entity-relation diagrams, or their combination like in the SSADM [3]; it can also be control diagrams, such as structure chart for describing the architecture of systems and flowchart for representing abstract algorithms [4]. These graphical representations are often used together with natural language descriptions of the data or functional details.

In contrast, the OOM usually puts the emphasis on the definition of objects and their associations. A model in the OOM usually needs to show the structure of classes, the association between classes, and the interaction of objects. The Unified Modeling

Language (UML) is a popular language for object-oriented design that was adopted as a standard by the Object Management Group (OMG) and has been managed by this organization ever since [5]. A UML model may be composed of different diagrams that can be classified into three categories: *structure diagrams*, *behavior diagrams*, and *interaction diagrams*.

Structure diagrams are used to describe the static aspects of a system. For example, the *component diagram* represents the connection of different components in the system, showing the dependencies among these components, and the *class diagram* defines the structure of classes and their associations. Behavior diagrams reflect the dynamic aspect of the system, including *activity diagrams* and *use case diagrams*. These diagrams are mainly used to illustrate the behavior of the system from different angles. For instance, the activity diagram describes the business and operational activities of the components step by step in the system, while the use case diagram explains how users interact with the system being modeled. Interaction diagrams are part of behavior diagrams, which are used to represent the flow of control and data among the objects in the system. For example, the *sequence diagram* shows how objects communicate with each other by means of sending and receiving messages in time order and the *communication diagram* focuses on the structure of the messages that pass between the objects in the interaction.

While the diagram-based MDD approach is believed by some people to provide a comprehensible way to develop software systems on the basis of the popular view “a picture is worth a thousand words”, it suffers from several difficulties in practice. Firstly, drawing diagrams usually takes more time and space than textual descriptions. Secondly, evolution of diagrams, which is usually necessary during a development project, tends to be more inconvenient and takes more time than that of textual descriptions. Thirdly, diagrams on a large scale tend to be difficult to manage and may not be comprehensible enough due to the density of drawing within a limited space. Finally, keeping consistency between the model and the program implemented based on the model would consume considerable amount of time and give rise to the high cost of development. My personal experience in developing a supporting tool for testing COBOL programs in the past suggests that using diagram-based MDD be almost impossible for a project within a limited schedule.

1.2 Formal Methods

Formal methods provide a mathematically-based verified design approach to software development [6–10]. To construct a system, a formal specification is first written in a formal notation to define what to be done by the potential program. Such a definition is usually given at an abstract level, focusing on the relation between the initial states and the final states of operations. The formal specification is of course expected to accurately reflect the user’s requirements, although the expectation may not be realized in practice. On the basis of the specification, the implementation of the system is carried out. During the process of implementation, formal verification

can be done to ensure that the expected properties defined in the specification are satisfied by the implemented program.

The biggest advantage of formal methods over informal approaches is that the correctness of the program can be proved based on logics, such as predicate logic and Hoare logic [11, 12]. In the early time when formal methods were proposed, many people believed that formal verification can replace testing in software quality assurance because the former can ensure the correctness of programs while the latter cannot. However, the truth of formal methods have become much clearer for us now [13, 14], after extensive research and various experiences of applying formal methods in practice.

Formal specification, as one of the important techniques in formal methods, may be helpful in clarifying the user's requirements and precisely defining the functionality of the system under construction, if appropriate approaches are taken. Unfortunately, few people pay attention to research on specific techniques for writing formal specifications so far, especially in the context of software engineering.

Formal verification, on the other hand, is found to be too challenging to be used in practice [15]. After a program is constructed, it is almost always the case that it contains many faults. To prove its correctness using formal verification techniques is therefore impossible in this situation. Compared with conventional software quality assurance techniques, such as review, inspection, and testing, formal verification does not seem to be cost-effective in detecting bugs due to two reasons. One is that formal verification requires high skills in using mathematics and patience for dealing with complicated, detailed steps in formal verification, but few practitioners, even experts, can meet this requirements, especially for large-scale systems in practice. Another reason is that formal verification is a static analysis technique; it does not run the program to demonstrate its behaviors, thus cannot provide opportunities for the developer and the user to evaluate the desired functional services and other qualities in relation to the services, such as usability and efficiency. In fact, even if a program is formally proved to be correct with respect to its specification, it does not necessarily ensure that the program will satisfy the user's requirements because the user's desired requirements may not be accurately and/or completely defined in the specification. Our experience suggests that this possibility is considerably high in practice. It is now quite clear to us that formal verification and testing are complementary rather than exclusive, but testing is still used as the main stream technique for software quality assurance.

1.3 Agile Development

Considering evolution of software as the intrinsic nature of software development projects, the Agile development approach, proposed by a group of people in 2001 [16], emphasizes the importance of communication and collaboration between the developer and the user (client) in evolving software artifacts into a final product. As indicated by the Agile manifesto [17], four specific principles, also characteris-

tics, of Agile development are proposed. These principles include (1) individuals and interactions over processes and tools, (2) working software over comprehensive documentation, (3) customer collaboration over contract negotiation, and (4) responding to change over following a plan. Simply speaking, Agile development emphasizes the importance of starting a software project by trying to first build working software (i.e., code) and then continues to evolve it until a satisfactory product is provided. Since the working software can run on computer, it will help individual interactions, customer collaboration, and the developer to understand what need to be changed in the current version of software.

While the Agile development approach may make good sense for software projects that aim to construct a small-scale software system by a small team, it will face many challenges in the development of large-scale software where the development team may include many people.

Firstly, without a planned process and the use of tools, the capability of individuals would be limited and individual interactions would be inefficient and error-prone. Secondly, without careful consideration through abstract level documentation, building working software may make wrong decisions and proceed in wrong directions. The potential consequence is the wasting of time and efforts. Thirdly, the customer collaboration is a sensible strategy because the software under construction must eventually meet the customer's requirements, but whether this is realistic in practice and how the collaboration can be done properly still remain an open issue. Finally, responding to change requires the understanding of the working software. This may be easy for small-scale software but would be much hard for large-scale systems. Especially, when changes occur frequently, which is very likely due to the lack of careful consideration before constructing the working software, the responding to the changes would introduce many new bugs and inconsistencies within the software system.

1.4 Agile Formal Engineering Method

Experience in research and application of the above different methods suggests that they are actually complementary in contributing to the enhancement of software productivity and quality. The important thing is to strike a good balance between the documentation and coding, between detecting bugs in code and verifying its correctness, and between manual operations and tool support. The greatest challenge is how such a balance can be made to facilitate software projects, especially for large-scale ones.

After our intensive research and testing applications over the last sixteen years since 2008, we have established several specific principles and techniques for fulfilling this goal by properly integrating the SOFL formal engineering method [18] with the Agile development approach. As a result of our efforts, we have established an *Agile Formal Engineering Method* (AFEM) called *Agile-SOFL*. Agile-SOFL is characterized by the following principles and techniques:

abstract and concise fashion. To help prevent unnecessary faults possibly caused by the developer misunderstanding the requirements, the requirements must be clarified in the Clarifying Requirements phase. This will result in a hybrid specification that rather precisely defines the requirements and reflects the dependency relationship between operations (called processes in Agile-SOFL). As we will discuss in detail in Chap. 4, a hybrid specification is an integration of user interface specification (UIS), semi-formal specification, and formal specification for critical processes. The UIS is intended to reflect the user's requirement for the way the user will use the system. The semi-formal specification is expected to define most of the processes in the system rather precisely. The formal specification is applied only to the critical or extremely important processes to define their functionality. Agile-SOFL provides a three-step approach to constructing a hybrid specification, as detailed in Chap. 5.

After gaining a good understanding through writing the hybrid specification, the developer can carry out an implementation of the whole system in an incremental manner. Due to the availability of the specification, such an implementation can be done systematically in a planned order. As discussed in Chap. 6, Agile-SOFL offers a strategy to perform the incremental implementation based on the specification, which will help the early release of the software product completed.

When implementing each operation, what strategy should we take for efficiency and quality? Considering the nature of coding as an evolutionary process, we provide an iterative coding approach in Agile-SOFL. The iterative approach is characterized by interweaving the activities of constructing code with active code review for detecting and preventing bugs as well as maintaining the required code style. Since review can be applied anytime and anywhere during the implementation, it will be extremely useful for code quality assurance. In Chap. 6, we will discuss this technique in detail.

While checking the code statically using code review during the coding phase is helpful, systematically inspecting the completed code against the specification will help significantly validate both the specification and the code. Further, analyzing the runtime behaviors of the system using testing is even more important for validation since software can provide the expected functionality and services only when it is executed. The question is how the inspection and testing can be done in a manner that will be able to help significantly save time but ensure the reliability. In Agile-SOFL, the techniques for specification-based inspection, "Vibration" testing, and testing-based formal verification are offered for this purpose. These techniques will be discussed in detail in Chaps. 7, 8, and 9, respectively.

To deal with evolutions in software development, building a good requirement traceability will enable the developer to track how each requirement defined in the specification is gradually clarified into code throughout the whole development process. In Chap. 6, we will discuss techniques for building trace links to ensure requirements traceability.

References

1. E. Yourdon. *Modern Structured Analysis*. Prentice Hall, 1989.
2. R. B. France, S. Ghosh, T. Dinh-Trong, and A. Solberg. Model-Driven Development Using UML 2.0: Promises and Pitfalls. *The Computerl*, 39(2):59–66, Feb. 2006.
3. D. Hargrave. *Ssadm 4+ for Rapid Systems Development (International Software Engineering)*. McGraw-Hill Book Co Ltd, 1995.
4. D. Budgen. *Software Design*. Addison-Wesley, 1994.
5. T.Dinh-Trong, A. Solberg, S. Ghosh, and R. B. France. Model-Driven Development Using UML 2.0: Promises and Pitfalls. *Computer*, 39(02):59–66, 2006.
6. C. B. Jones. *Systematic Software Development Using VDM*. 2nd edition, Prentice Hall, 1990.
7. J. Woodcock and J. Davies. *Using Z: Specification, Refinement, and Proof*. Prentice Hall, 1996.
8. C. A. R. Hoare and J. He. *Unifying Theories of Programming*. Prentice Hall Europe, 1998.
9. R.-J. Back and J. von Wright. *Refinement Calculus: A Systematic Introduction*. Springer-Verlag, 1998.
10. J. R. Abrial, M. Butler, S. Hallerstede, T. S. Hoang, F. Mehta, and L. Voisin. Rodin: An Open Toolset for Modelling and Reasoning in Event-B. *International Journal on Software Tools for Technology Transfer*, 12(6):447–466, 2010.
11. C. A. R. Hoare and N. Wirth. An Axiomatic Definition of the Programming Language PASCAL. *Acta Informatica*, 2(4):335–355, 1973.
12. C. Hoare and C. B. Jones. *Essays in Computer Science*. Prentice Hall International, 1989.
13. S. Liu and R. Adams. Limitations of Formal Methods and An Approach to Improvement. In *Proceedings of 1995 Asia-Pacific Software Engineering Conference (APSEC'95)*, pages 498–507, Brisbane, Australia, Dec 1995. IEEE Computer Society Press.
14. J. Woodcock, P. G. Larsen, J. Bicarregui, and J. Fitzgerald. Formal Methods: Practice and Experience. *ACM Computing Surveys*, 41(4):1–39, 2009.
15. D. L. Parnas and M. Lawford. The Role of Inspection in Software Quality Assurance. *IEEE Transactions on Software Engineering*, 29(8):674–676, August 2003.
16. P. Abrahamsson, O. Salo, J. Ronkainen, and J. Warsta. *Agile Software Development Methods: Review and Analysis*. Espoo 2002, VTT Publications 478, 2002.
17. Manifesto for agile software development. August 2001.
18. S. Liu. *Formal Engineering for Industrial Software Development Using the SOFL Method*. Springer-Verlag, ISBN 3-540-20602-7, 2004.

Chapter 2

Introduction to SOFL



SOFL stands for Structured Object-Oriented Formal Language [1]. It is a practical specification language that integrates data flow diagram (DFD) and formal operation specifications in VDM-SL. Given that Agile-SOFL employs an extended version of SOFL as its specification language, this chapter provides a concise introduction to the pertinent aspects of SOFL. This aims to aid readers in comprehending the contents of this book.

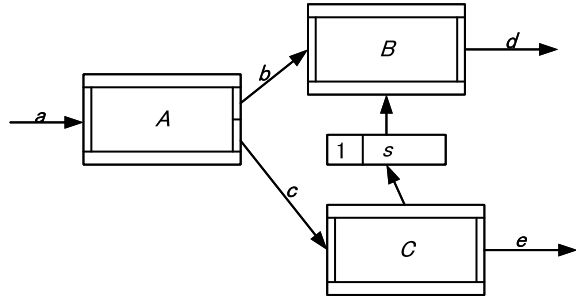
2.1 General Structure of SOFL Specification

A specification in SOFL is mainly a set of inter-related modules. The modules are organized in a hierarchical fashion that is formed through the decomposition of high level processes. In this section, we will explain all these concepts and structures in detail.

2.1.1 Module and CDFD

A module is a mechanism for defining a subsystem of the whole system under construction using a formalized DFD, called Condition Data Flow Diagram (CDFD), and for specifying processes occurring in the CDFD. The CDFD describes the architecture of the subsystem, showing how the processes are connected based on data flows and data stores. A process is an operational component that models a transformation from input to output. It can be implemented as a function in the programming language C or a method defined in a class in Java. The functionality of each process occurring in the CDFD is specified in pre- and post-conditions. Further, to help obtain the modularity of a process specification for curbing its complexity and enhancing its readability, mathematical functions can also be defined, either by an

Fig. 2.1 No. 1: The CDFD of the module *Example*



explicit specification or implicit specification. In other words, a module has two parts: a CDFD and a structure containing the specifications of the processes involved.

Figure 2.1 shows an example CDFD depicting how three processes *A*, *B*, and *C* are connected based on data flows *a*, *b*, *c*, *d*, and *e* as well as data store (or simply store) *s* to work together to fulfil the functionality of a subsystem. A data flow represents a data item moving from one process to another. A store denotes a data item or a collection of data items in a certain structure in rest. Compared with a data flow that is active, a store is passive, always waiting for some processes to read or write (or update). The CDFD is assigned number No. 1 in order for it to be referenced in the specification. The module associated with the CDFD is given as follows as an example:

```

module Example;
const
  ConstantIdentifier = constant;
...
type
  TypeIdentifier = TypeExpression;
...
var
  s: int;
inv
...
behav CDFD_No.1;
process Init()
pre true
post Q(s)
end_process;
process A(a: int) b: int | c: int
pre P1(a)
post Q1(a, b, c)
end_process;
process B(b: int) d: int
ext rd s: int

```



```

pre  $P2(b, s)$ 
post  $Q2(b, s, d, f(b, s))$ 
decom  $B\_Decom$ 
end_process;
process  $C(c: int) e: int$ 
ext wr  $s: int$ 
pre  $P3(c, s)$ 
post  $Q3(c, e, \sim s, s, g(c, s))$ 
end_process;
function  $f(b: int, s: int): int$ 
==  $E(b, s)$ 
end_function;
function  $g(c: int, s: int): int$ 
pre  $C1(c, s)$ 
post  $C2(g, c, s)$ 
end_function
end_module;

```

The major role of the module is to specify all the processes occurring in the corresponding CDFD. Each module has a unique process, named *Init*, for initializing the local stores occurring in the CDFD. The functionality of initialization of the stores is also specified in pre- and post-conditions, as indicated in the above module. However, if the module does not have any local store to be initialized, the process *Init* will not be provided. Moreover, if the specification of a process needs to use some named constants, such as *max_value* representing the constant 10000, and/or user-defined types to declare variables, the named constants must be declared in the section *const* and the user-defined types must be declared in the section *type*, as indicated in the above module. The declared constant identifier can be used in the process specifications and the declared type identifier can be used to declare variables in the module. The section *var* in the module is used to declare all the store variables. In CDFD 1, there is only one local store *s* and it is declared with the type *int* in section *var* of the module. If more than one store occurs in the CDFD, they must all be declared with appropriate types in the section *var*. Further, if some user-defined type needs a restriction on its values and the restriction must be sustained throughout the entire system, then such an restriction can be defined as a type invariant in the section *inv*. Similarly, if a store variable is required to satisfy a condition throughout the entire system, the condition can be defined as a state invariant also in the same section *inv*. Moreover, we need to clearly link the module to the associated CDFD by writing *CDFD_number* after the reserved word *bahav*, where the number refers to the number assigned to the CDFD. For instance, the CDFD in Fig. 2.1 is assigned the number No.1, therefore, the CDFD_No.1 is written as the link to show the association between the module and the CDFD. All these sections in a module are not explained with specific examples in the above module, but our another specification for a realistic application to be given later will show the details of these sections.

Generally, a process can have a set of input variables, a set of output variables, and a set of store variables it accesses or updates. For example, process A has one input variable a and two output variables b and c that must be produced exclusively as indicated by the vertical bar separating the declarations of b and c . Process A takes an input a satisfying the pre-condition and generates b or c , but not both, which satisfies the post-condition. There is no store variable involved in the specification of process A . In the case of process B , apart from input variable b and output variable d , there is also a store variable s declared. Since the value of a store variable accessed by a process exists before and after the execution of the process, we call a store variable an external variable as well. In the process specification, we put the reserved word `ext` and the way to use the store variable, which is either `rd` (read only) or `wr` (write), before the declaration of the relevant store variable, such as s .

The functionality of a process is formally specified in pre- and post-conditions. The pre-condition defines a constraint on the input variables and the initial values of the external variables. Such a constraint is expected to be satisfied before the execution of the process. The post-condition describes another constraint on the final values of the output variables. The final value of the external variable is the result of updating its initial value by the process. Let us take process B as an example. The specification tells that if the pre-condition $P2(b, s)$ is satisfied by input variable b and the external variable s before the execution of process B , then the post-condition $Q2(b, d, s, f(b, s))$ is required to be satisfied by input variable b , output variable d , external variable s , and the result of the function application $f(b, s)$. Since process B only reads from s , without updating it, the initial value and the final value of s are the same before and after the execution of the process. In this case, syntactically we use s to denote the value of s before and after the execution of the process.

Compared with the pre-condition, it is not difficult to see that the post-condition actually defines a relation between the initial state, including the values of input variables and the external variables, and the final state, including the values of the external variables and the output variables. Simply speaking, the post-condition of a process defines a relation between the input and the output of the process.

Note that in the pre- or post-condition of a process specification, a mathematical function (or simply function) may be applied, such as $f(b, s)$ in the post-condition $Q2(b, d, s, f(b, s))$ of process B . There are two ways to define a function: explicit specification and implicit specification. An explicit specification tells how the parameters of the function are used directly to calculate the result of the function while an implicit specification uses pre- and post-conditions to define the function. Since the pre- and post-conditions only focus on defining the relation between the parameters of the function and its result, the idea of how the parameters join the calculation of the result of the function may not be stated explicitly. For example, function f is defined using an explicit specification while function g is defined using an implicit specification. Note that in the case of implicit specification, the name of the function, for example g , is used to represent the result of the function, thus making the expression of the relation between the parameters and the result of the function possible. The definition of functions f and g given above shows the syntax of function definitions.

It is worth mentioning that the textual contents of a module, called the module specification, and the associated CDFD are complementary in defining the functionality of the related subsystem. The CDFD focuses on the architecture describing the composition of processes while the module specification concentrates on defining the precise functionality of each process. Only when both the CDFD and the module specification work together, the functionality of the related subsystem can be properly and clearly defined. Note that the CDFD and the module specification cannot replace with each other because the semantics of processes is not given in the CDFD while the information on how the processes are connected in terms of data flows and stores are not explicitly provided in the module specification. In fact, this is an essential idea in the design of the SOFL specification language since keeping both CDFD and module specification can facilitate the construction of the specification and guarantee a good readability of it. Since specifications must be written by humans to allow them to learn the user's requirements accurately, this characteristic of the SOFL language is an important element to make it practical for industrial applications.

2.1.2 Hierarchical CDFDs and Modules

To build a specification in SOFL for a large scale system, we need to address two problems. One is how to curb the complexity of one level CDFD and its module specification. The other is how to allow different developers to work on different parts of the same project in parallel to achieve high software productivity.

These two problems can be effectively handled by decomposing high level processes into low level CDFDs. In general, a high level process with a complex functionality would be difficult to specify using pre- and post-conditions. There are some reasons for this. One is that the high level process needs to be described using a sequential structure of processes. Another possible reason is that there is a lack of sufficient information that disallows the relation between the input and the output of the process to be expressed precisely in its post-condition. To specify the functionality of the high level process, we can decompose it into a low level CDFD that shows how low level processes can be used to fulfil the task of defining its functionality. In this case, the pre- and post-conditions of the process that is decomposed would not need to be given in detail.

Let us take process *B* in CDFD No.1 above as an example. Assuming that process *B* is rather complex and needs to be decomposed into a CDFD. Figure 2.2 shows the decomposed CDFD that describes how low level processes *B1*, *B2*, *B3*, and *B4* are connected based on data flows *b*, *b1*, *b2*, *b3*, *b4*, and *d* as well as store *s* to fulfil the task of the high level process *B* to transform its input *b* to output *d*. Since the decomposed CDFD is intended to spell out the details of how process *B* transforms its input to output, it is essential to ensure that all the input variable *b*, output variable *d*, and store variable *s* connected to process *B* are consistently used in its decomposition. This means that input variable *b* must be used as an input variable

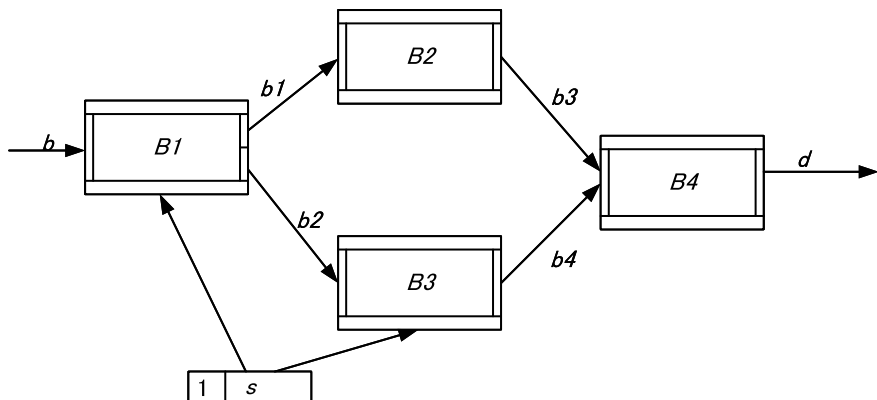


Fig. 2.2 No. 2: The decomposition of the high level process B

of some low level starting process like $B1$; output variable d must also be used as an output variable of some terminating process like $B4$; and store variable s must be accessed by some processes like $B1$ and $B3$ in the same way it is accessed by the high level process B . As indicated in Fig. 2.1, store s is read by process B , therefore it can also be read by low level processes like $B1$ and $B3$ in Fig. 2.2. There should be no process in this CDFD that writes to store s ; otherwise, this CDFD would implement some operations on store s that are not specified by the high level process B .

Again, to specify the low level processes in the decomposed CDFD in Fig. 2.2, we need to construct a module specification associated with it. As an example, the structure of the module specification is given as follows:

```

module B_Decom/Example;
const
...
type
...
var
ext s: int;
inv
...
behav CDFD_No.2;
process B1(b: int) b1: int | b2: int
ext rd s: int
pre P21(b, s)
post Q21(b, s, b1, b2)
end_process;
process B2(b1: int) b3: int
pre P22(b1)
post Q22(b1, b3)

```

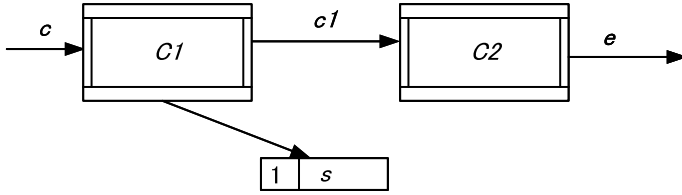


Fig. 2.3 No. 3: The decomposition of the high level process *C*

```

end_process;
process B3(b2: int) b4: int
ext rd s: int
pre P23(b2, s)
post Q23(b2, s, b4)
end_process;
process B4(b3, b4: int) d: int
pre P24(b3, b4)
post Q24(b3, b4, d)
end_process;
end_module;

```

To ensure the readability of the decomposed module, we name it as *B_Decom* according to the module naming convention of SOFL, indicating the fact that the module is a decomposition of high level process *B*. We also explicitly present the parent module Example in which high level process *B* is introduced (i.e., defined and used for the first time). Further, in the specification of the high level process *B*, a reserved word *Decom* is used to indicate into which module the process is decomposed. As we see from the specification of process *B* given before, *B_Decom* is written to indicate that process *B* is decomposed into the module called *B_Decom*. Such a link between the high level process and its decomposition will play a significant role to facilitate the developer in efficiently evolving the specification during its construction. Further, applying the same principle of constructing a module for a CDFD, all the four processes *B1*, *B2*, *B3*, and *B4* are specified formally and all the relevant declaration sections, such as *const*, *type*, *var*, and *inv* are properly supplied.

It is possible that more than one high level process can be decomposed. For example, the process *C* in the module Example can be decomposed into the CDFD in Fig. 2.3. The module specification associated with this CDFD is given below.

```

module C_Decom/Example;
const
...
type
...
var
ext s: int;

```

```

inv
...
behav CDFD_No.3;
process C1(c: int) c1: int
ext wr s: int
pre P31(c, s)
post Q31(c, ~s, s, c1)
end_process;
process C2(c1: int) e: int
pre P32(c1)
post Q32(c1, e)
end_process;
end_module;

```

In theory, the specification of the whole system in SOFL should be organized into a hierarchy of modules that is formed through the decomposition of high level processes, but in practice, it can be quite difficult to do so for the whole system. The reason is that the specification should focus on the definition of what to be done by the potential program system. Therefore, it should avoid paying great attention to the design of the architecture of the whole system. Another possible reason is that even a great effort is made to design the architecture of the system in CDFDs, it will likely be changed during the implementation. Therefore, convincing practitioners to spend time on the architecture design in the specification can be unrealistic in many cases.

A more flexible way to construct a specification in SOFL is to treat it as a set of modules. In these modules, some of them are inter-related in terms of process decomposition while others remain independent (i.e., the module is not a decomposition of any other process and also no process defined in the module has any decomposition). Apparently, such a specification may lack sufficient information on the system architecture, but designing the architecture can be realized during the implementation phase. The author's experience suggests that this should be more practical.

2.2 SOFL Logic

SOFL basically adopts predicate logic for the description of pre- and post-conditions of processes as well as invariants in a module specification. Predicate logic consists of a language for describing predicate expressions and a set of axioms and rules for logical inferences. A predicate expression is divided into two kinds: basic predicate expressions and compound predicate expressions. In this section, we introduce these two expressions, respectively.

Table 2.1 Logical operators and quantifiers in SOFL

Operators and quantifiers	Mathematical symbols	Meaning	Priority
<i>forall</i> [$x : X$] $P(x)$	$\forall_{x \in X} \cdot P(x)$	Universally quantified expression	
<i>exists</i> [$x : X$] $P(x)$	$\exists_{x \in X} \cdot P(x)$	Existentially quantified expression	
<i>not</i>	$\neg$	Not	Highest
<i>and</i>	$\wedge$	And	Next
<i>or</i>	$\vee$	Or	Next
$\Rightarrow$	$\Rightarrow$	Implies	Next
$\Leftrightarrow$	$\Leftrightarrow$	Is equivalent to	Lowest

2.2.1 Basic Predicate Expressions

A basic predicate expression can be an atomic predicate, for example, $x > y$, or a predicate expression formed by using logical operators and quantifiers. Table 2.1 shows all the logical operators and quantifiers used in the SOFL logic. The syntax adopted is intended to enhance the usability and readability of specifications since they are English-like and easy to type in from an ordinary keyboard. Too many mathematical symbols in specifications are likely to keep ordinary students in software engineering and practitioners in industry away from using formal specifications in their projects, especially in large-scale software development projects.

Below are some examples of logical expressions:

- (1) $x > y$ and $y > z \Rightarrow x > z$
- (2) $\text{not } a \Leftarrow b$ and $x > a + b$ or $x < a * b \Leftarrow w > a \Rightarrow \text{not } a > b$
- (3) *forall*[$i : \text{int}$]*exists*[$j : \text{int}$]| $j > 0$ and $j > i$

A predicate expression is evaluated from the left to the right and the priority given in Table 2.1 must be applied during the evaluation. The operator *not* has the highest priority. The priority will go down in turn for the other operators: *not*, *or*, $\Rightarrow$, and $\Leftarrow$. and $\Leftarrow$ has the lowest priority. In the case of predicate expression (1), the operator *and* is first applied and then the operator $\Rightarrow$ is applied. When expression (2) is evaluated, the order of applying the logical operators is: *not* (on the left hand side), *and*, *or*, *not* (on the right hand side), $\Rightarrow$, and finally $\Leftarrow$. In expression (3), the universal quantifier and the existential quantifier are used in turn from left to right, therefore, the universal quantifier is first applied and then the existential quantifier is applied. Under this circumstance, the body of the quantified expression, $j > 0$ and $j > i$, is evaluated. This quantified expression states that for any integer i in type *int*, there must exist another positive integer j in the same type such that $j > i$. Since the type *int* in SOFL refers to the integer type in mathematics, it is infinite. Therefore, the quantified expression (3) evaluates to true. The reason why all the numeric types, as we will discuss in detail later in this chapter, refer to the

types in mathematics rather than those in a specific programming language is that we want to keep SOFL independent of any specific programming languages. This way will allow programmers to freely choose their favorite programming language to implement the specification.

Note that the order of the universal quantifier and the existential quantifier is significant. The change of their order would possibly change the evaluation result of the quantified expression. For example, if we exchange the universal quantifier and the existential quantifier in expression (3) above to form expression (4) below, it will evaluate to false, since it is impossible to have any positive integer j that is greater than all the integers.

(4) $\text{exists}[j: \text{int}] \text{forall}[i: \text{int}] \mid j > 0 \text{ and } j > i$

2.2.2 Compound Predicate Expressions

To ensure a good readability of predicate expressions, two kinds of comprehensible expressions are designed for use in predicate expressions. One is called *if-then-else* expression and the other is called *let* expression. We call a predicate expression using these expressions a *compound predicate expression*.

2.2.2.1 If-then-else Expression

An *if-then-else* expression is a mathematical function that can be used to define both the result when a condition is met and the result when it is not. For example, *if $x > y$ then $a = x - y$ else $a = y - x$* describes that if the condition $x > y$ is true, it will yield the result $a = x - y$; otherwise if the condition $x > y$ is false, it will yield $a = y - x$ as the result.

2.2.2.2 Let Expression

A *let* expression is a structured predicate expression in which *pattern identifiers* are declared and used to keep the expression concise. The syntax of the *let* expression is as follows:

$$\text{let } PI1 = E1, PI2 = E2, \dots, PIn = En \text{ in } Q(PI1, PI2, \dots, PIn)$$

In this *let* expression, $PI1, PI2, \dots, PIn$ are called pattern identifiers; $E1, E2, \dots, En$ are expressions; $PI1 = E1$ means that $PI1$ denotes $E1$; and the scope that the pattern identifiers can be used is the predicate expression $Q(PI1, PI2, \dots, PIn)$ after the reserved word *in*. That is, the $PI1$ in Q should be interpreted as $E1$, and the other pattern identifiers can be interpreted accordingly. For instance,

$$\text{let } A = x * y + 5, B = x + y - 10 \text{ in } x * B + A * y > x * A + y * B$$

is actually equivalent to the expression:

$$x * (x + y - 10) + (x * y + 5) * y > x * (x * y + 5) + y * (x + y - 10).$$

It is worth mentioning that a pattern identifier differs from a variable in the sense that the content of the pattern identifier does not change, always referring to the corresponding expression while the variable may change its value.

2.3 Data Types in SOFL

Data type, or simply type, is an important aspect of a software system. To enable a specification to describe complex expressions, various data types are provided in the SOFL specification language, including basic types, set types, sequence types, composite types, product types, map types, and union types. Each type defines a set of values together with a set of operators. Each operator shows how values can be processed to yield other values in the type. In this section, we briefly discuss these types, respectively.

2.3.1 Basic Types

The basic types include *numeric types*, *enumeration types*, *character type*, and *boolean type*.

2.3.1.1 Numeric Types

The numeric types include four specific types: natural numbers with zero *nat0*, natural numbers without zero *nat*, integers *int*, and real numbers *real*. Table 2.2 shows the type name and values and Table 2.3 shows all the arithmetic operators defined on the numeric types.

Let $x = 9$, $y = 4.5$, $z = 3.14$, $a = -4$, and $b = 3$. Below are examples of using the operators.

$$\begin{aligned} -z &= -3.14 \\ \text{abs}(a) &= 4 \\ \text{floor}(y) &= 4 \\ x + z &= 12.14 \\ x - y &= 4.5 \\ a * b &= -12 \\ x/y &= 2.0 \\ a \text{ div } b &= -2 \\ a \text{ rem } b &= 2 \text{ (quotient} = -2) \\ x \text{ mod } b &= 0 \end{aligned}$$

$$x * b = 729.$$

Note that we take the rule for guaranteeing the remainder to be positive in the integer division and the operation for the remainder.

In addition, there are also relational operators defined on the numeric types. Some of the operators are shared by almost all the formal notations in the literature while some other operators are unique to SOFL. Table 2.4 shows all the relational operators provided in SOFL. For example, $x < y < z$ means that $x < y$ and $y < z$; $x <= y <= z$ means $x <= y$ and $y <= z$.

2.3.1.2 Enumeration Types

An enumeration type is characterized by all the values of the type being explicitly given. Syntactically, an enumeration type takes the format:

$$\{ \langle E1 \rangle, \langle E2 \rangle, \dots, \langle En \rangle \}.$$

where $\langle E1 \rangle, \langle E2 \rangle, \dots, \langle En \rangle$ are the values of the enumeration type and each value is represented by a string enclosed in angle brackets. The reason why the angle brackets are used in expressing a value of an enumeration type is that

Table 2.2 Numeric type names and values

Type name	Type values
<i>nat0</i>	{0, 1, 2, ...}
<i>nat</i>	{1, 2, 3, ...}
<i>int</i>	{..., -2, -1, 0, 1, 2, ...}
<i>real</i>	{..., -2.5, -1.3, 0, 1.3, 2.5, ...}

Table 2.3 Operators defined on numeric types

Operator	Name	Type
$-x$	Unary minus	$real \rightarrow real$
$abs(x)$	Absolute value	$real \rightarrow real$
$floor(x)$	Floor	$real \rightarrow int$
$x + y$	Addition	$real \times real \rightarrow real$
$x - y$	Subtraction	$real \times real \rightarrow real$
$x * y$	Multiplication	$real \times real \rightarrow real$
x / y	Division	$real \times real \rightarrow real$
$x \text{ div } y$	Integer division	$int \times int \rightarrow int$
$x \text{ rem } y$	Remainder	$int \times int \rightarrow nat0$
$x \text{ mod } y$	Modulus	$nat0 \times nat0 \rightarrow nat0$
$x ** y$	Power	$real \times real \rightarrow real$

Table 2.4 Relational operators defined on numeric types

Operator	Name	Type
$x < y$	Less than	$real \times real \rightarrow bool$
$x > y$	Greater than	$real \times real \rightarrow bool$
$x \leq y$	Less than or equal	$real \times real \rightarrow bool$
$x \geq y$	Greater than or equal	$real \times real \rightarrow bool$
$x < y < z$	Less-between	$real \times real \rightarrow bool$
$x \leq y \leq z$	Less-equal-between	$real \times real \rightarrow bool$
$x \geq y \geq z$	Greater-equal-between	$real \times real \rightarrow bool$
$x = y$	Equal	$real \times real \rightarrow bool$
$x \neq y$	Unequal	$real \times real \rightarrow bool$

the string of the value, such as $E1$, might be misunderstood as a variable in formal specifications if the angle brackets were not used.

There are only two operators defined on numeration types. One is equality = and the other inequality $\neq$. For example, $\langle E1 \rangle = \langle E2 \rangle$ means $\langle E1 \rangle$ is the same as $\langle E2 \rangle$ and $\langle E1 \rangle \neq \langle E2 \rangle$ means $\langle E1 \rangle$ is different from $\langle E2 \rangle$.

Below are some possible enumeration types.

- $\{\langle Monday \rangle, \langle Tuesday \rangle, \langle Wednesday \rangle, \langle Thursday \rangle, \langle Friday \rangle\}$
- $\{\langle Cat \rangle, \langle Dog \rangle, \langle Horse \rangle, \langle Cow \rangle\}$
- $\{\langle Manchester \rangle, \langle York \rangle, \langle London \rangle, \langle Belfast \rangle, \langle Oxford \rangle, \langle Hiroshima \rangle, \langle Tokyo \rangle\}$

In a SOFL specification, an enumeration type can be used directly to declare a variable, but we can also first declare an identifier for the enumeration type in the type section of a module and then use the type identifier to declare variables. For instance, we can have the following type declaration in the type section:

$Weekday = \{\langle Monday \rangle, \langle Tuesday \rangle, \langle Wednesday \rangle, \langle Thursday \rangle, \langle Friday \rangle\}.$

Then, we can use the type identifier $Weekday$ to declare a variable wherever it is needed in the specification, for example,

$workingday: Weekday .$

2.3.1.3 Character Type

The character type contains all the characters that can be directly input to computer from an ordinary keyboard. We use *char* to represent the character type. Conforming to the convention in programming languages, a single character is expressed in a single quotation. For example, below are some possible characters.

- 'a'
- '+'
- '#'
- 'X'

There are only two operators defined on the character type, which are equality = and inequality <>. For instance, 'a' = 'a' is true and '#' <> 'X' is also true, while 'a' = '+' is false and 'X' <> 'X' is also false.

2.3.1.4 Boolean Type

Boolean type contains two logical values *true* and *false*. We use *bool* to represent the type, that is, *bool* = {*true*, *false*}. As discussed before, there are five logical operators defined on boolean type, they are *not*, *and*, *or*, *=>*, and *<=>*, as explained in Table 2.1. Since this type is well known in computer science, detailed discussion on this type is omitted.

2.3.2 Set Types

A set is an unordered collection of distinct elements. A set type defines a set of sets. A set type is expressed as

set of elemT

where *elemT* is a type called element type. Such a set type actually defines the power set of the element type *elemT*. That is, *set of elemT* = {*X* | *X* ⊆ *elemT*}.

In a module specification, a type identifier can be declared in the type section of the module. For example,

setT = *set of elemT*

declares the type identifier *setT* that denotes the set type *set of elemT*. Therefore, *setT* can be used to declare variables wherever it is needed.

Two constructors and several other operators are defined on set types. Table 2.5 shows two constructors, known as set enumeration and set comprehension, and Table 2.6 shows all the other operators. The constructors are characterized by describing how elements can be used to construct sets. The set enumeration indicates that a group of elements can be used to construct a set by enclosing them in braces. A set comprehension shows that a set can be constructed by defining the element expression involving bound variables and the property all the bound variables must satisfy.

Table 2.5 Constructors defined on set types

Constructor	Name
$\{x1, x2, \dots, xn\}$	Set enumeration
$\{e(x1, x2, \dots, xn) \mid x1 : T1, x2 : T2, \dots, xn : Tn \& P(x1, x2, \dots, xn)\}$	Set comprehension

Table 2.6 Operators defined on set types

Operator	Name	Type
$a \text{ inset } A$	Membership	$elemT \times set \text{ of } elemT \rightarrow bool$
$a \text{ notin } A$	Non-membership	$elemT \times set \text{ of } elemT \rightarrow bool$
$card(A)$	Cardinality	$set \text{ of } elemT \rightarrow nat0$
$get(A)$	Member access	$set \text{ of } elemT \rightarrow elemT$
$union(A, B)$	Union	$set \text{ of } elemT \times set \text{ of } elemT \rightarrow set \text{ of } elemT$
$inter(A, B)$	Intersection	$set \text{ of } elemT \times set \text{ of } elemT \rightarrow set \text{ of } elemT$
$diff(A, B)$	Difference	$set \text{ of } elemT \times set \text{ of } elemT \rightarrow set \text{ of } elemT$
$power(A)$	Power set	$set \text{ of } elemT \rightarrow set \text{ of } set \text{ of } elemT$
$dunion(S)$	Distributed union	$set \text{ of } set \text{ of } elemT \rightarrow set \text{ of } elemT$
$dinter(S)$	Distributed intersection	$set \text{ of } set \text{ of } elemT \rightarrow set \text{ of } elemT$
$A = B$	Equality	$set \text{ of } elemT \times set \text{ of } elemT \rightarrow bool$
$A <> B$	Inequality	$set \text{ of } elemT \times set \text{ of } elemT \rightarrow bool$
$subset(A, B)$	Subset	$set \text{ of } elemT \times set \text{ of } elemT \rightarrow bool$
$psubset(A, B)$	Proper subset	$set \text{ of } elemT \times set \text{ of } elemT \rightarrow bool$

The former explicitly displays all the elements of the set while the latter defines all the elements implicitly. The following examples illustrate how the two constructors can be used.

- $\{5, 9, 10, 50\}$, a set enumeration of four integers.
- $\{x + y \mid x : nat0, y : nat0 \& 1 < x * y < 10\}$, a set comprehension defining a set whose elements are defined by the element expression $x + y$, where x and y are both natural numbers and also satisfy the property $1 < x * y < 10$.

Apart from the constructors, there are other operators defined on set types. Table 2.6 gives all the other operators on set types. Below are some examples showing how these operators are used.

- $7 \text{ inset } \{4, 5, 7, 9\} <=> true$
 $3 \text{ inset } \{4, 5, 7, 9\} <=> false$
- $7 \text{ notin } \{4, 5, 7, 9\} <=> false$
 $3 \text{ notin } \{4, 5, 7, 9\} <=> true$
- $card(\{5, 7, 9\}) = 3$

- $get(\{5, 15, 25\}) = 15$ (possibly), get is a nondeterministic operator whose result can be any element of the given set.
- $union(\{5, 15, 25\}, \{15, 20, 25, 30\}) = \{5, 15, 25, 20, 30\}$
- $inter(\{5, 7, 9\}, \{7, 10, 9, 15\}) = \{7, 9\}$
- $diff(\{5, 7, 9\}, \{7, 10, 9, 15\}) = \{5\}$
- $dunion(\{\{5, 10, 15\}, \{5, 10, 15, 25\}, \{10, 25, 35\}\}) = \{5, 10, 15, 25, 35\}$
- $dinter(\{\{5, 10, 15\}, \{5, 10, 15, 25\}, \{10, 25, 35\}\}) = \{10\}$
- $power(\{5, 15, 25\}) = \{\}, \{5\}, \{15\}, \{25\}, \{5, 15\}, \{15, 25\}, \{5, 25\}, \{5, 15, 25\}$

2.3.3 Sequence and String Types

A sequence is an ordered collection of elements, which is similar to a set in the sense both a sequence and a set represent a collection of elements, but there are two important differences between them. The order of the appearance of elements in a sequence is significant but not so in a set. The duplication of an element in a sequence is allowed but forbidden in a set.

A sequence type defines a set of sequences and is expressed as

seq of elemT

where *elemT* is the element type. Such a sequence type defines a set of all possible sequences that are constituted from the elements of *elemT*. That is, $seq\ of\ elemT = \{s | s\ \text{is a sequence of elements of } elemT\}$.

A sequence type identifier can be declared in the type section of a module. For example,

$seqT = seq\ of\ elemT$

declares *seqT* as a sequence type identifier representing the type *seq of elemT*. Therefore, *seqT* can be used to declare variables wherever it is required.

The *string* type is a special sequence type in the sense that it contains only the sequences of characters, which are usually called strings. In other words, a string is a sequence of characters and the *string* type denotes a set of strings. Note that *string* is a built-in type in SOFL, therefore, the user of SOFL can use it directly to declare variables in specifications.

Two constructors and several other operators are defined on sequence types. Since *string* is a special sequence type, all the constructors and operators defined on sequence types are applicable to strings in the *string* type. Table 2.7 gives the two constructors and Table 2.8 shows all the other operators.

Table 2.7 Constructors defined on sequence types

Constructor	Name
$[x1, x2, \dots, xn]$	Sequence enumeration
$[e(x1, x2, \dots, xn) x1 : T1, x2 : T2, \dots, xn : Tn \ \& \ P(x1, x2, \dots, xn)]$	Sequence comprehension

Table 2.8 Operators defined on sequence types

Operator	Name	Type
$len(s)$	Length	$seq\ of\ elemT \rightarrow nat0$
$s(i)$	Sequence application	$seq\ of\ elemT \times nat \rightarrow elemT$
$s(i, j)$	Sub-sequence	$seq\ of\ elemT \times nat \times nat \rightarrow seq\ of\ elemT$
$hd(s)$	Head	$seq\ of\ elemT \rightarrow elemT$
$tl(s)$	Tail	$seq\ of\ elemT \rightarrow seq\ of\ elemT$
$elems(s)$	Elements	$seq\ of\ elemT \rightarrow set\ of\ elemT$
$inds(s)$	Indexes	$seq\ of\ elemT \rightarrow set\ of\ nat$
$conc(s1, s2)$	Concatenation	$seq\ of\ elemT \times seq\ of\ elemT \rightarrow seq\ of\ elemT$
$dconc(S)$	Distributed concatenation	$seq\ of\ seq\ of\ elemT \rightarrow seq\ of\ elemT$
$s1 = s2$	Equality	$seq\ of\ elemT \times seq\ of\ elemT \rightarrow bool$
$s1 <> s2$	Inequality	$seq\ of\ elemT \times seq\ of\ elemT \rightarrow bool$

The sequence enumeration shows a group of ordered elements enclosed in square brackets. A sequence comprehension defines a sequence by specifying the element expression involving bound variables and the property the bound variables must satisfy. The following examples illustrate how the two constructors can be used.

- $[5, 15, 15, 5, 35]$, a sequence of integers.
- $['u', 'n', 'i', 'v', 'e', 'r', 's', 'i', 't', 'y']$, a sequence of characters.
- $[i * j | i : nat, j : nat \& 1 \leq i + j \leq 3]$, a sequence comprehension.

It is worth mentioning that every sequence defined using a sequence comprehension is ensured (by the SOFL language) to keep its elements in ascending order if the elements are numeric values. Further, a sequence of characters is usually expressed as a string. For instance, the sequence of characters just mentioned above can be expressed as “university”. Compared with the original sequence representation, this string has a much better readability.

Note that the index of a sequence in SOFL starts from 1, not 0. Below are some examples showing how the operators given in Table 2.8 can be used. Let $s1 = [4, 9, 10]$, $s2 = [\{3, 9\}, \{6\}]$, $s3 = [10, 9, 4, 25]$, $s4 = \text{“university”}$, and $s = [[5, 15, 25], [10, 20, 30, 40], [2, 4, 6, 8, 10]]$. Then, we have

- $len(s1) = 3$
 $len(s2) = 2$
- $s1(3) = 10$
 $s4(5) = 'e'$
- $s3(2, 3) = [9, 4]$
- $hd(s1) = 4$
 $hd(s2) = \{3, 9\}$

- $tl(s3) = [9, 4, 25]$
- $elems([5, 10, 5, 10, 15]) = \{5, 10, 15\}$
 $elems(s4) = \{'u', 'n', 'i', 'v', 'e', 'r', 's', 't', 'y'\}$
- $inds(s2) = \{1, 2\}$
- $conc(s1, s3) = [4, 9, 10, 10, 9, 4, 25]$
- $dconc(s) = [5, 15, 25, 10, 20, 30, 40, 2, 4, 6, 8, 10]$.

2.3.4 Composite Types

A composite type represents a set of objects. Each object usually contains several fields, each describing an attribute of the object. A composite object is like a structure in C and a class in Java (only to some extent). A composite type identifier can be declared in the type section of a module using the following format:

$$\begin{aligned}
 ComT &= composed\ of \\
 &\quad f_1 : T_1 \\
 &\quad f_2 : T_2 \\
 &\quad \dots \\
 &\quad f_n : T_n \\
 &\quad end
 \end{aligned}$$

where $ComT$ is the composite type identifier to be declared and each f_i ($i = 1, \dots, n$) is a field variable declared with type T_i . This declaration states that $ComT$ represents a composite type containing objects each of which has n fields: $f_1, f_2, \dots, f_n$.

The type identifier $ComT$ can then be used to declare variables. For instance,
 $co : ComT$

declares the variable co with type $ComT$, thus co will denote a specific composite object with the n fields.

There is one constructor and several operators defined on composite types, as shown in Tables 2.9 and 2.10, respectively. The constructor is called *make function*. Applying it to a composite type, say $ComT$, by providing the initial value for each field, for example, value $v1$ for field f_1 , a specific composite object will be constructed, which is represented by $mk_ComT(v1, v2, \dots, vn)$.

There are four operators defined on composite types. The field select operator $co.f_1$ represents the field f_1 of the object co . The modification operator *modify*

Table 2.9 The constructor defined on composite types

Constructor	Name
$mk_ComT(v1, v2, \dots, vn)$	Make function

Table 2.10 The operators defined on composite types

Operator	Name
$co.f_1$	Field select
$modify(co, f_1 \rightarrow v1, \dots, f_n \rightarrow vn)$	Field modification
$co1 = co2$	Equality
$co1 <> co2$	Inequality

is applied to a composite object and some of its fields to modify the current value of the fields to a specified value. For example, $modify(co, f_1 \rightarrow v1, f_3 \rightarrow v3)$ denotes a composite object derived by modifying the value of field f_1 to $v1$ and the value of f_3 to $v3$ for object co .

The examples below show how the make function and the operators on composite objects can be used. Let *Account* be declared as a composite type below.

```

Account = composedof
    account_no : nat
    password : nat
    balance : real
end

```

Let *account* be a variable declared with *Account*:

```
account: Account .
```

Then, we can have

- $account = mk_Account(1073548, 1234, 5000)$
- $account.password$
- $account.blance$
- $modify(account, password \rightarrow 4321)$
- $account <> modify(account, password \rightarrow 4321)$

2.3.5 Product Types

A product type defines a set of tuples with a fixed length. Let $T1, T2, \dots, Tn$ be n types. Then, a product type T can be declared as

```
 $T = T1 * T2 * \dots * Tn.$ 
```

The declared type T can then be used to declare variables. For example,

```
 $a: T;$ 
```

declares that the variable a represents a value in type T .

A value of T , called a tuple, is created using the make-function:

Table 2.11 The constructor defined on product types

Constructor	Name
$mk_T(v1, v2, \dots, vn)$	Make function

Table 2.12 The operators defined on product types

Operator	Name
$a(i)$	Tuple application
$modify(a, 1 \rightarrow v1, \dots, n \rightarrow vn)$	Tuple modification
$a1 = a2$	Equality
$a1 <> a2$	Inequality

$mk_T(v1, v2, \dots, vn)$.

This make function is an overloaded operator since its syntax is the same as the make function defined on composite types. The only difference here is that T is a product type, not a composite type. Using the make function, a tuple can be created. For example, $mk_T(v1, v2, \dots, vn)$ represents a tuple in SOFL, which is composed of values $v1, v2, \dots, vn$, where $v1$ denotes a value in $T1$, $v2$ in $T2$, ..., and vn in Tn .

Note that a tuple is syntactically similar to a sequence, but there are two differences. One is that a tuple has a fixed length while the length of a sequence is flexible. Another difference is that the types for each value in a tuple may be different while all the values in a sequence must come from the same type. Further, a tuple is also different from a composite object in the sense that all the values of a composite object need to be referred by using the corresponding field name while the values of a tuple need to be referred by using their position (i.e., index).

Tables 2.11 and 2.12 give the constructor and other operators, respectively, on product types. Since the way to use them is similar to that of the corresponding operators on sequence types and composite types, we omit the detailed discussion here.

2.3.6 Map Types

A map is a function whose domain and range are both finite. A map type defines a set of maps and a map type identifier can be declared in the format:

$$mapT = map\ T1\ to\ T2,$$

where $mapT$ is the map type identifier to be declared, $T1$ is called domain type, and $T2$ the rang type. $mapT$ represents all the possible maps from $T1$ to $T2$. It can be used to declare map variables, such as

$$m1, m2: mapT.$$

Table 2.13 The Cconstructors defined on map types

Constructor	Name
$\{a1 \rightarrow b1, a2 \rightarrow b2, \dots, an \rightarrow bn\}$	Map enumeration
$\{e1(a) \rightarrow e2(b) \mid a : T1, b : T2 \& P(a, b)\}$	Map comprehension

Similar to set types and sequence types, two constructors and several other operators are defined on map types. Table 2.13 shows two constructors, map enumeration and map comprehension, and Table 2.14 shows all the other operators.

In the map enumeration, $a1 \rightarrow b1$ is called a maplet, stating that $a1$ in the domain is mapped to $b1$ in the range. The map comprehension defines the set of pairs $e1(a) \rightarrow e2(b)$, where $e1(a)$ is an expression involving variable a and $e2(b)$ another expression involving variable b and property $P(a, b)$ must be satisfied by a and b .

Since map is a well known concept in mathematics, we only explain the operators with the syntax unique to SOFL. $dom(m)$ denotes the domain of map m and $rang(m)$ the range of m . $domrt(s, m)$ represents the submap of map m whose domain is a subset of s and $rngrt(m, s)$ the submap of m whose range is a subset of s . $domrb(s, m)$ denotes the submap of m whose domain is a subset of the domain of m that is derived by removing all the elements of s from the domain of m and $rngrb(m, s)$ the submap of m whose range is a subset of the range of m that results from removing all elements of s from the range of m . $override(m1, m2)$ is an operation for a union of two maps $m1$ and $m2$ with the restriction: if a maplet in map $m2$ shares the first element with a maplet in $m1$, the resulting map only includes the maplet in $m2$ as its element. $comp(m1, m2)$ is the operation of map composition that forms another map from the two given maps $m1$ and $m2$ with the effect that $comp(m1, m2)$ shares the domain with $m1$ and the range with $m2$. $inverse(m)$ denotes the inverse of map m where m is required to be a one-to-one map. That is, if m is not a one-to-one map, $inverse(m)$ will be undefined.

Let us look at some examples that illustrate the ways to use the constructors and operators. Let $m1 = \{5 \rightarrow 10, 10 \rightarrow 20, 15 \rightarrow 30\}$, $s1 = \{5, 10\}$, $m2 = \{10 \rightarrow 5, 15 \rightarrow 50, 4 \rightarrow 20\}$. Then, we have

- $\{3 \rightarrow 'a', 8 \rightarrow 'b', 10 \rightarrow 'c'\}$
- $\{x \rightarrow y \mid x : \{5, 10, 15\}, y : \{10, 20, 30\} \& y = 2 * x\}$
- $m1(5) = 10$
- $m1(10) = 20$
- $dom(m1) = \{5, 10, 15\}$
- $rng(m1) = \{10, 20, 30\}$
- $domrt(s1, m1) = \{5 \rightarrow 10, 10 \rightarrow 20\}$
- $rngrt(m1, s1) = \{5 \rightarrow 10\}$
- $domrb(s1, m1) = \{15 \rightarrow 30\}$
- $rngrb(m1, s1) = \{10 \rightarrow 20, 15 \rightarrow 30\}$
- $override(m1, m2) = \{10 \rightarrow 5, 15 \rightarrow 50, 4 \rightarrow 20, 5 \rightarrow 10\}$
- $comp(m1, m2) = \{5 \rightarrow 5\}$

Table 2.14 The operators defined on map types

Operator	Name	Type
$m(a)$	Map application	$map\ T1\ to\ T2 \times T1 \rightarrow T2$
$dom(m)$	Domain	$map\ T1\ to\ T2 \rightarrow set\ of\ T1$
$rng(m)$	Range	$map\ T1\ to\ T2 \rightarrow set\ of\ T2$
$domrt(s, m)$	Domain restriction to	$set\ of\ T1 \times map\ T1\ to\ T2 \rightarrow map\ T1\ to\ T2$
$rngrt(m, s)$	Range restriction to	$map\ T1\ to\ T2 \times set\ of\ T2 \rightarrow map\ T1\ to\ T2$
$domrb(s, m)$	Domain restriction by	$set\ of\ T1 \times map\ T1\ to\ T2 \rightarrow map\ T1\ to\ T2$
$rngrb(m, s)$	Range restriction by	$map\ T1\ to\ T2 \times set\ of\ T2 \rightarrow map\ T1\ to\ T2$
$override(m1, m2)$	Override	$map\ T1\ to\ T2 \times map\ T1\ to\ T2 \rightarrow map\ T1\ to\ T2$
$comp(m1, m2)$	Map composition	$map\ T1\ to\ T2 \times map\ T2\ to\ T3 \rightarrow map\ T1\ to\ T3$
$inverse(m)$	Map inverse	$map\ T1\ to\ T2 \rightarrow map\ T2\ to\ T1$
$m1 = m2$	Equality	$map\ T1\ to\ T2 \times map\ T1\ to\ T2 \rightarrow bool$
$m1 <> m2$	Inequality	$map\ T1\ to\ T2 \times map\ T1\ to\ T2 \rightarrow bool$

- $inverse(m1) = \{10 \rightarrow 5, 20 \rightarrow 10, 30 \rightarrow 15\}$
- $m1 = m2 \iff false$
- $m1 <> m2 \iff true$

2.3.7 Union Types

A union type is the union of several disjoint constituent types. As we have discussed before, a sequence or a set can only represent a collection of elements of the same type. If we want to form a sequence or set of elements of different types, we must use a union type as the element type of the corresponding sequence or set type.

Let $T1, T2, \dots, Tn$ be n types. Then, a union type T constituted from these types is declared in the format:

$$T = T1|T2|\dots|Tn.$$

A value of T can come from one of the constituent types $T1, T2, \dots, Tn$. It is important to keep $T1, T2, \dots, Tn$ disjoint so that any value of type T can be precisely determined to belong to which constituent type.

As Table 2.15 shows, there are only three operators defined on union types. Let x be a value of union type T . The *is* function, $is_Ti(x)$, yields true if the type of x is Ti and false otherwise, where Ti can be any of the constituent types $T1, T2, \dots, Tn$. Below is a simple example that illustrates how a union type can be declared. Let the three types *Color*, *Key*, and *Digits* be declared as follows:

Table 2.15 The operators defined on union types

Operator	Name
$is_Ti(x)$	<i>is</i> function
$a1 = a2$	Equality
$a1 <> a2$	Inequality

$Color = \{< Red >, < Blue >, < Yellow >\}$

$Key = char$

$Digits = set\ of\ nat.$

Then, the union type *Hybrid* can be declared as:

$Hybrid = Color \mid Key \mid Digits,$

where the three types *Color*, *Key*, and *Digits* are disjoint. Applying the operators defined on union types, we can have the following results:

- $is_Color(< Blue >) <=> true$
- $< Yellow > = \{2, 5, 8\} <=> false$
- $'a' <> < Red > <=> true.$

2.4 Other Components in SOFL

To facilitate both high level software specification and detailed design, apart from the components discussed above, SOFL also provides other components, including *function definitions* in modules, *classes* as the user defined types, *explicit process specifications*, and *comments* for explaining formal expressions and variable declarations in natural language. Since these components are not often used in this book, we omit the their detailed introduction. The readers who are interested in these features of the SOFL specification language can refer to the SOFL book [1].

Reference

1. S. Liu. *Formal Engineering for Industrial Software Development Using the SOFL Method*. Springer-Verlag, ISBN 3-540-20602-7, 2004.

Chapter 3

Informal Specification



A specification is a document defining what to be done by a potential software system. An important role of the specification is to serve as a vehicle for the communication between the developer and the end-user (or simply the user) and between the developers on the same development team. For this reason, the readability of specifications is extremely important.

Several factors contribute to the readability of specifications, including the language used to write the specification, the structure of the specification, and the precision of expressions used in the specification. Apart from facilitating communications, writing specifications is also an effective means to help the developer gain a good understanding of the user's requirements and therefore help prevent faults (by preventing the related defects), in particular requirements-related faults.

The Agile-SOFL method advocates the use of a hybrid specification to define the user's requirements before the implementation of the required system is undertaken. It also offers a *three-step hybrid specification approach* to constructing the hybrid specification from an *informal specification*. In this chapter, we introduce the informal specification by describing its structure and the issues to be taken into account when the specification is constructed. In the next chapter, we will present the hybrid specification. In Chap. 5, we will discuss the three-step hybrid specification approach and how it can be used to prevent requirements-related faults.

3.1 The Structure

An informal specification refers to the specification written in natural language and is used to document the user's requirements at an abstract level. To ensure a good readability and coverage of information, an informal specification is composed of three sections: *Functions*, *Data*, and *Constraints*.

The *Functions* section is intended to list up the desired functions or services to be provided by the software under construction. Each function should be briefly described, serving as an item to remind the developer of the corresponding requirement, but should also be understandable enough for both the user and the developer. We will see some examples later in this chapter. The *Data* section should list up the data items required to realize the functions given in the Functions section. In general, these items are possibly used by more than one function and carry important information the system needs to process. The *Constraints* section presents necessary constraints on functions, data items, or both. A constraint usually states some service the desired functions should not provide or some condition that the functions should not violate.

3.2 Example

Let us take a simplified Stock Trading System (STS) as an example to illustrate the structure and the content of each section of the informal specification. For the sake of readability, the STS is deliberately designed to have only four functions, as shown below.

Informal specification

I. *Functions*

1. Register customer (name, age, bank account, address, email address, telephone number, and purchased stocks)
2. Register stock (name, price, unit for sale, limit for selling to one customer)
3. Purchase stock (the purchased stock details should be recorded in the customer's information and the payment should be made using the customer's bank account)
4. Sell stock (the money resulted from the sale of stock should be put into the customer's bank account).

II. *Data*

1. Customer list (F_1, F_3, F_4)
2. Stock list (F_2, F_3, F_4)
3. Bank accounts (F_3, F_4).

III. *Constraints*

1. For each kind of stock, a customer can only buy at most 1000 units (F_3, D_2).
2. A stock name is unique (D_2).
3. The currency used in the system is only Japanese yen (JPY) (F_3, F_4, D_2).

In the *Functions* section, four desired services are briefly described. The STS should be able to register a customer with the detailed attributes, such as name, age, bank account, address, email address, telephone number, and purchased stocks, and to register a stock, including its name, price, unit for sale, and limit for selling to one customer. It is also required to provide the functions for dealing with purchasing and selling a stock, respectively.

In the *Data* section, three data items, which are possibly shared by more than one function given in the *Functions* section, are presented, including Customer list, Stock list, and Bank accounts. The Customer list is intended to record all registered customers' information; the Stock list is designed to record all the registered stocks' information; and the Bank accounts is used to contain all customers' bank account information. To enhance the understandability of the specification, the connection between the data items given in this section and the functions given in the *Functions* section will be helpful. Since data items must be used by some functions, pointing out clearly each data item is possibly used by which function will be useful for refining the informal specification to a more precise hybrid specification later on. The way to show such a connection for a data item is to use parentheses to enclose the numbers of all the functions that potentially use this data item after the description of the data item. For example, Customer list (F_1, F_3, F_4) indicates that the data item Customer list is possibly used by F_1 (i.e., Function 1), F_3 (Function 3), and F_4 (Function 4).

In the *Constraints* section, three specific constraints are provided. The first constraint is on the amount of purchasing stocks: a customer can buy at most 1000 units of the same kind of stock. Since this constraint may need to be considered for designing the function 3 (i.e., Purchase stock) and the data item 2 (i.e., Stock list), we show this information in the parentheses after the constraint description, that is, (F_3, D_2), where D stands for Data. The second constraint states that each stock has a unique name. Since it needs to be considered in designing the data item 2, we need to add (D_2) after the constraint description. The third constraint is on the currency to be used in the STS. Since it may need to be taken into account for designing function 3 and function 4, we add (F_3, F_4) after the constraint description.

3.3 Completeness

When constructing an informal specification, the developer should always try to achieve its completeness. If an informal specification is complete, it should cover all the desired functions, data, and constraints, although each aspect may not be described precisely. The reason why the completeness of the informal specification is so important is that the informal specification will be treated as the user's requirements based on which the subsequent development activities will be performed. The software product resulting from the project is unlikely to meet the user's expectations if it fails to address essential aspects of the user's requirements.

However, ensuring the completeness may not be guaranteed during the informal specification phase in practice for some reasons (e.g., the user may not be able to express clearly what he or she wants). It might need a continuous evolution during the whole development process. The important point here is that the developer must make his or her best efforts in obtaining a complete coverage of the user's requirements in the informal specification at abstract level, even if it may fail in some circumstances.

Chapter 4

Hybrid Specification



A hybrid specification is an appropriate combination of *semi-formal specification*, *GUI specification*, and *formal specification*. In Agile-SOFL, a semi-formal specification is characterized by formal data description and informal logical description. A GUI (Graphical User Interface) specification focuses on the item descriptions of each GUI page and their connections to other GUI pages. A formal specification is written in the SOFL formal notation, making both the data definitions and logical expressions precise. The principle underlying the combination is basically to write a semi-formal specification for the whole system, but for critical operations, their semi-formal specification should be refined into a formal specification to achieve high precision for fault prevention. In addition, a GUI specification needs to be provided to describe the necessary items and structures on important GUI pages to indicate how the user of the system under construction should interact with the system.

The rest parts of this chapter will focus on the detailed discussions of each kind of specification in a hybrid specification mentioned above, including their structure and characteristics.

4.1 Semi-formal Specification

A semi-formal specification is a well-balanced document defining the user's requirements in a style that properly combines informal expressions with formal expressions, covering the aspects of functions, data, and constraints.

4.1.1 *Structure and Characteristics*

Basically, a semi-formal specification is a collection of modules in SOFL, with the following characteristics:

- All the data-related aspects in a module are described formally, that is, using the SOFL formal notation to write the data-related aspects. The data-related aspects include constant declarations, type declarations, and variable declarations. The variable declarations include both the store variable declarations given in the var section of a module and the input and output variable declarations given in the signature of a process specification.
- All logical expressions are given informally, that is, using natural language to write the logical expressions. Logical expressions in a module may be used to represent invariants given in the inv section and the pre- and post-conditions of a process specification. Several techniques for writing informal logical expressions will be detailed in the next chapter.
- A condition data flow diagram (CDFD) can be used to describe how the relevant processes in a module are connected in terms of data flows and data stores to form a subsystem, which is part of the whole system under construction.

A semi-formal specification has two important roles to play. One is to serve as a vehicle for communication between the user and the developer. The other is to prepare for further formalization at either an abstract level for more precise specification or a detailed level for implementation. Since it is usually difficult to properly and clearly describe the invariants and the pre- and post-conditions of processes without a precise and accurate understanding of the data structures to be used in the system, we emphasize the importance of formally defining all the data-related aspects in a semi-formal specification. As a matter of fact, this is similar to the situation when a program code is constructed. Without determining the structure of data represented by variables to be used in the code, it is almost impossible to write the algorithm in the code, simply because the statements (e.g., assignment statements) involved in the algorithm must refer to the related variables in order to properly express how the data are processed. Further, invariants and the pre- and post-conditions of processes need to be expressed in logical expressions, but since logical expressions are relatively difficult to write and properly writing them requires the developer to have high skills in mathematical abstraction, it will be more cost-effective in practice to informally write all the parts in relation to logical expressions in a module. This style not only allows the developer to easily write the semi-formal specification, but also allows both the user and the developer to easily communicate with each other via the semi-formal specification.

4.1.2 Process Specification

To facilitate both the user and the developer to easily understand the semi-formal specification, how to express the pre- and post-conditions of a process specification is crucial. In traditional model-based formal notations, such as VDM-SL, Z, and B-Method, both pre- and post-conditions are given in logical expressions, but little suggestion on the style of the logical expressions is provided. In the Agile-SOFL method, we propose to use *Functional Scenario Form* (FSF) to replace the pre- and post-conditions in defining the functionality of a process, see examples in Sect. 4.1.5. In fact, such an FSF style process specification is equivalent to the pre-post style specification in the sense that the pre-condition is *true* and the post-condition is an FSF.

An FSF of a process is a disjunction of *Functional Scenarios* (FS), $FS_1 || FS_2 || \dots || FS_n$, where the symbol $||$ is used to mean the logical operator *or* (mathematically $\vee$). Each $FS_i = T_i \ \&\& \ D_i$ ($i = 1, \dots, n$) is a conjunction of a *test condition* T_i and a *defining condition* D_i , where we use the symbol $\&\&$ to represent the logical operator *and* (i.e., mathematically $\wedge$) to syntactically separate the test condition T_i and the defining condition D_i for readability. Test condition T_i must be a conjunction of basic predicates (atomic predicates, their negations, or quantified predicates) or a disjunction of such conjunctions; it also contains no output variables of the process (i.e., it can only contain input variables and external variables denoting their initial values, called *initial external variables* (e.g., $\sim x$), as well as bound variables declared in quantified predicate expressions). To allow for concise expression of the FSF, T_i can be either a logical expression or the keyword *others*. The keyword *others* is unique but optional to use in a functional scenario; it represents all other possible conditions other than the already given test conditions in the FSF. That is, $others \Leftrightarrow \text{not}(FS_1 \text{ or } FS_2 \text{ or } \dots \text{ or } FS_{n-1})$, provided that the functional scenario is $FS_1 || FS_2 || \dots || FS_{n-1} || others: D_n$. Defining condition D_i must contain the output variables of the process and must not contain any atomic predicate involving only input variables or initial external variables. In other words, the defining condition is only composed of atomic predicates containing output variables (of course, input variables are also allowed to be used in such an atomic predicate because the predicate is intended to describe the relation between the input and the output). A functional scenario defines an independent subfunction of the process: when the input of the process satisfies test condition T_i , the output must be defined by defining condition D_i .

Note that the symbols $||$ and $\&\&$ cannot be used as the logical operator *or* and *and*, respectively, within all test conditions and defining conditions of functional scenarios. The adoption of the symbols $||$ and $\&\&$ in FSFs in Agile-SOFL aims to ensure a good readability of FSFs but their semantics are the same as the logical operator *or* and *and*, respectively.

Another extension of SOFL in Agile-SOFL is to allow the declarations of *local* variables (or temporary variables). A local variable in a process specification is a variable that can only be used within the process specification, including test conditions

and defining conditions. Such a variable is intended to help “bridge” expressions in test conditions and defining conditions. The scope of using each local variable is one functional scenario, see the example below. Multiple local variables may be declared in a process specification. The format of local variable declarations in a process specification is illustrated by an example as follows.

process *ProcessName*(*input variables declaration*) *output variables declarations*

```
...
local lv1: Ty1;
      lv2: Ty2
FSF:
 $T_1(lv1) \ \&\& \ D_1(lv1)$ 
||
 $T_2(lv1, lv2) \ \&\& \ D_2(lv1, lv2)$ 
||
 $others \ \&\& \ D_n(lv2)$ 
end_process.
```

In this specification, all the local variables *lv1*, *lv2*, ..., *lvn* are all declared with a type (e.g., *Ty1*, *Ty2*) after the keyword “local” and all the declarations are separated by a semicolon. Note that the local variable *lv1* are used in two functional scenarios, but due to the scope rule for local variables, *lv1* can only be interpreted locally to one functional scenario. For instance, *lv1* used in $T_1(lv1)$ can only refer to the same variable in $D_1(lv1)$; it has nothing to do with the *lv1* appearing in another functional scenario $T_2(lv1, lv2) \ \&\& \ D_2(lv1, lv2)$. In Chap. 5, we will use some example to explain its usage in a process specification.

4.1.3 Condition Data Flow Diagram (CDFD)

As mentioned previously, CDFD is used to describe the data dependence between processes defined in the same module, which can show how the processes work together to fulfil the overall functionality of the module. In Agile-SOFL, drawing a CDFD for a module is optional. If the module specification is clear enough to show its overall functionality and the functionality of its all components (e.g., processes), drawing a CDFD for the module will not be required. Only when the module specification cannot clearly convey the idea of how the processes involved work together to provide the overall functionality of the module, it can be beneficial to draw the corresponding CDFD.

4.1.4 Hierarchy of Modules

To avoid the complexity of one level module and its associated CDFD, a hierarchy of modules can be constructed by means of process decomposition. That is, if necessary, a process defined in the module can be decomposed into a low-level module, possibly

with a low level CDFD, that spells out the details of how the process transforms its input to output and how to update the related data stores. Repeatedly applying the process decomposition technique, a hierarchy of modules together with a hierarchy of CFFDs will be built. Such a hierarchy must conform to several important rules. First, each module is associated with only one CDFD or no CDFD. Second, each process can only be decomposed into one low-level module. Finally, not every process needs to be decomposed.

4.1.5 Example of Semi-formal Specification

We use part of the simplified stock trading system (STS) to show what a semi-formal specification looks like and to illustrate the issues in writing it. The specification of the STS is composed of two modules, named *Registration* and *TradeStocks*, respectively. The module *Registration* is concerned with the definition of the functions in relation to customer and stock registrations, respectively, and the module *TradeStocks* is used to define the functions related to operations on stock trading. To keep the discussion simple enough for readers to understand the characteristics of a semi-formal specification, we only use the module *Registration* below as an example to help explain the relevant issues.

```

module Registration;
type
  CustomerInformation = composed of
    id: IDstring
    passw: Password
    name: string
    tel_no: TelephoneNumber
    email_add: EmailAddress
    address: string
    owned_stocks: seq of PurchasedStock
  end;
IDstring = string; /*containing exactly 10 characters*/
TelephoneNumber = seq of nat0 /*containing exactly 11 digits*/
Password = seq of nat0; /*the password contains exactly 6 digits*/
EmailAddress = LocalPart * At * DomainName;
LocalPart = string;
At = {@};
DomainName = string;
StockInformation = composed of
  s_id: IDstring /*containing exactly 10 characters*/
  s_name: string
  company: Company
  unit_price: real
  max_sale_units: nat0

```

```

        /*the stock units that can be sold for one customer. */
        stock_units_for_sale: nat0
        /*the stock units available for sale. */
        end;
    PurchasedStock = composed of
        stock_info: StockInformation
        no_of_units: nat0
        buy_unit_price: real
        currently_evaluated_unit_price: real
        current_total_value: real
        end;
    Company = composed of
        name: string
        representative: PersonName
        address: string
        tel: TelephoneNumber
        email: EmailAddress
        end;
    PersonName = FirstName * MiddleName * FamilyName |
        FirstName * FamilyName;
    FirstName = string;
    MiddleName = string;
    FamilyName = string;
    var
    customer_list: seq of CustomerInformation;
    stock_list: seq of StockInformation;
    inv
    1. Every element in the type IDstring contains exactly 10 characters;
    2. Every telephone number in the type TelephoneNumber has exactly 11 digits;
    3. Every customer on the customer_list has a unique id;
    4. Every stock on the stock_list has a unique s_id;
    5. Every stock on the stock_list has a unique s_name;
    behav CDFD_No.1;
    process Register_Customer(customer: CustomerInformation) mesg: bool
    ext wr customer_list: seq of CustomerInformation
    FSF:
    The input customer is not included in the ~customer_list &&
    the customer_list contains the customer and mesg = true
    ||
    The input customer is included in the ~customer_list &&
    the customer_list is the same as ~customer_list and mesg = false
    end_process;
    process Register_Stock(stock: StockInformation) res: bool
    ext wr stock_list: seq of StockInformation
    FSF:

```

```

The input stock does not exist in the ~stock_list &&
the stock_list contains the stock and res = true
||
The input stock exists in the stock_list &&
the stock_list is the same as ~stock_list and res = false
end_process
end_module;

```

In this module, a list of type identifiers are declared in the type section, including *CustomerInformation*, *IDstring*, *TelephoneNumber*, and *PersonName*. Each of these identifiers represents a specific type and can be used to declare variables wherever necessary in the specification. In addition, the var section shows the declarations of the two store variables *customer_list* and *stock_list*. These type identifiers and store variables are all formally declared in the sense that their syntax strictly conforms to the well-defined grammar of the SOFL specification language and carries the corresponding well-defined semantics.

Further, five invariants are defined in the inv section. The first two invariants, “Every element in the type *IDstring* contains exactly 10 characters” and “Every telephone number in the type *TelephoneNumber* has exactly 11 digits”, are type invariants, specifying a constraint on all values of the corresponding type. The other three invariants, “Every customer on the *customer_list* has a unique *id*”, “Every stock on the *stock_list* has a unique *s_id*”, and “Every stock on the *stock_list* has a unique *s_name*” are all state invariants, specifying a constraint on the corresponding store variable *customer_list* and *stock_list*, respectively.

This module also specifies two processes, *Register_Customer* and *Register_Stock*. All the input variables, output variables, and external variables of each process are formally declared and its FSF conforms to the well-defined format, but the test condition and defining condition of each functional scenario involved is described in English. For readability, it is important to keep the symbol && for separating the test condition and the defining condition in a functional scenario. The process *Register_Customer* registers the given *customer* in the *customer_list* that is a sequence of *CustomerInformation* and outputs a message *mesg* to indicate the success or failure of the registration. The process *Register_Stock* has a similar function of registration but the data for registration is *stock* rather than *customer*. Since the FSF of each process is comprehensible enough, their further explanations are omitted here.

The CDFD shown in Fig. 4.1 is associated with the module, describing how the specified processes work together to fulfil the overall function the module is supposed to provide for the whole system. In this particular module, the CDFD does not show a clear data flow dependence between the two processes occurring in the CDFD. This is possible since the module provides two separate functional services for registering different data in different stores. These two services are modeled by separate processes in the same module because they all fulfil the task of registration of necessary data, albeit with different nature. Considering the balance of the positive and negative effects of CDFDs, drawing a CDFD for the module is optional. If the module specification is clear enough to convey its functionality and the functionality

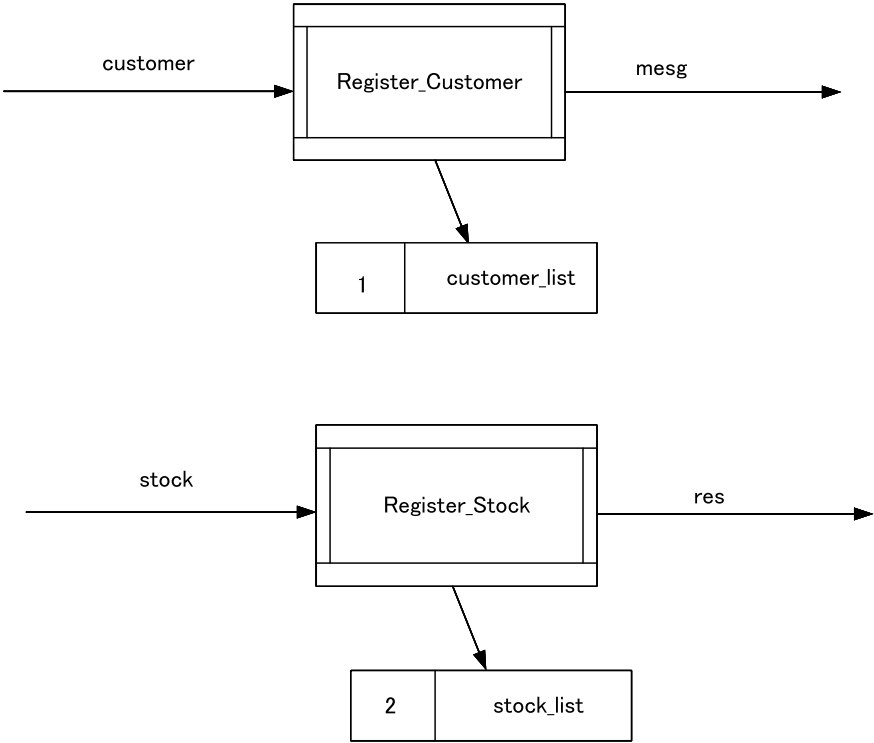


Fig. 4.1 No.1: The CDFD for the module Registration

of the involved components, the CDFD can be omitted. According to this criterion, the CDFD of the module *Registration* can actually be omitted. Moreover, since the module is simple, there is no need to decompose any processes to build a hierarchy of modules.

4.2 User Interface Specification

Human-machine interface (HMI), *user interface* (UI), and *graphical user interface* (GUI) are all terms to mean a user interface that facilitates humans to communicate or interact with a system, where the system can be a hardware system, software system, or the combination of both. In this book, we concentrate on software systems. Since more and more software systems adopt a GUI and even a textual interface can be regarded, or made, as a special case of graphical interface, we use GUI to generally mean a user interface for software systems in this book.

4.2.1 Focus of GUI Specification

A GUI specification focuses on the definition of what elements are used in each GUI page and how each page is connected to other pages. A GUI page is a screen expression or view that contains a collection of elements organized in a certain structure.

It is important to understand the difference between a GUI specification and a GUI design. A GUI design is actually a refinement of a GUI specification in the sense that not only elements are provided in each page, but the attributes of each element, such as its position, color, size, and shape, need to be decided. How to organize all the elements in a certain structure to achieve a good usability of the system also needs to be well planned. Therefore, a GUI specification is usually much simpler than a GUI design in general.

Agile-SOFL advocates that the GUI specification should be provided during the specification phase but the GUI design should be taken into account in the coding phase because the designed GUI can be quickly demonstrated through the execution of the corresponding code for confirmation between the stakeholders.

4.2.2 GUI Elements

GUI elements can be classified into four categories: *message elements*, *data elements*, *functional elements*, and *control elements*. A message element provides a piece of useful information to facilitate the user to learn about GUI pages. For instance, “Welcome to the Stock Trading System” can be a message element, indicating that the user is using the Stock Trading System. A data element is intended to let the user provide a piece of necessary information for the system, such as the textual items *name*, *age*, *address*, *id*, and *password*. A functional element is designed to represent a system operation through which the corresponding functional service will be offered to the user. For instance, in an automated teller machine (ATM) system, the buttons named *withdraw*, *deposit*, and *transfer* are all possible functional elements associated with the corresponding operations in the system. Such an element usually corresponds to a process in a module of the semi-formal specification in the way that the process specification defines what to be offered by the functional service. A control element is used to control the change of GUI from one page to another. Typical examples of such an element is a button labeled *next*, *return*, or *delete*. Clicking on the button *next*, the present GUI page will be changed to the next page. Clicking on the button *return*, the current page will be changed to the preceding page. Clicking on the button *delete*, the data just typed in will be eliminated.

4.2.3 Structure of GUI Specification

A GUI specification is composed of a set of GUI pages and a GUI module specification. Each GUI page is a graphical view, containing a collection of GUI elements. The GUI module specification is like a module specification introduced in the semi-formal specification previously; it contains a set of process specifications. Each process models either a functional element or a control element. The process specification for a functional element defines what data elements are used or produced by what functional element and the process specification for a control element describes how the current GUI page is changed to other pages.

Compared with the style of a process specification in a semi-formal specification, the process specification in a GUI module specification is even more informal in the sense that all the input and output data elements may not be formally declared using any well-defined type. They simply refer to the corresponding GUI elements in the GUI pages. This style helps keep the GUI module specification intuitive, simple, and comprehensible, which is important in practice.

Although any tools can be used to draw GUI pages, the Microsoft PowerPoint can be a convenient tool for this purpose because of its rich functions in both drawing of GUI elements and animation of PowerPoint slides that can comprehensibly simulate transitions of GUI pages. In this book, our GUI examples are all given in PowerPoint slides.

4.2.4 Example of GUI Specification

Let us use the simplified stock trading system STS as an example to illustrate the ideas of GUI specification. Figures 4.2, 4.3, and 4.4 show the three GUI pages used in the GUI specification. On Page 1 in Fig. 4.2, “Welcome to the Stock Trading System” is a message element while all the other four, “Register Customer”, “Register Stock”, “Purchase Stock”, and “Sell Stock”, are functional elements. On Page 2 in Fig. 4.3, “Provide the customer’s information” is a message element, but all the other six are data elements. Page 3 in Fig. 4.4 contains the similar GUI elements.

To understand how the three pages are connected and work together to facilitate the user to interact with the system, we build a GUI module. For brevity, we only give necessary parts of the module contents below.

```

module GUI;
process Register_Customer_Page1(Register Customer) Page 2
FSF:
Click on the button Register Customer &&
the GUI Page 1 is changed to Page 2.
comment

```



Fig. 4.2 Page 1: The Stock Trading System GUI

The screenshot shows the second page of the Stock Trading System GUI, titled "Provide the customer's information". At the top, a light blue rectangular box contains the title in bold blue font. Below the title, there are six input fields, each preceded by a label in blue font. The labels and their corresponding input fields are: "ID:" followed by a short rectangular box; "Password:" followed by a rectangular box; "Name:" followed by a rectangular box; "Telephone Number:" followed by a long rectangular box; "Email address:" followed by a rectangular box; and "Postal address:" followed by a rectangular box. All input fields are light blue with a thin black border. At the bottom center of the page, the text "Page 2" is displayed in bold black font.

Fig. 4.3 Page 2: The Stock Trading System GUI

Fig. 4.4 Page 3: The Stock Trading System GUI

This process corresponds to the *Register_Customer* process specified in the module *Registration*.

end_process;

process *Register_Stock_Page1*(Register Stock) Page 3

FSF:

Click on the button Register Stock &&
the GUI Page 1 is changed to Page 3.

comment

This process corresponds to the *Register_Stock* process specified in the module *Registration*.

end_process;

...

process *Input_Customer_Information_Page2*(

ID, Password, Name, Telephone number,
Email address, Postal address)

Page 4

FSF:

Type in specific values for the input GUI data elements required on Page 2 &&
the GUI Page 2 is changed to Page 4 (omitted for brevity).

end_process;

```

process Input_Stock_Information_Page3(
    ID, Company, Unit price,
    Maximum sale units, Stock units for sale)
    Page 5

```

FSF:

Type in specific values for the input GUI data elements required on Page 3 && the GUI Page 3 is changed to Page 5 (omitted for brevity).

```

end_process
end_module

```

When writing a process specification in the GUI module, a special format is adopted to ensure a good readability of the specification. The process name should be designed by the developer. In general, it does not appear on any GUI page but it may correspond to some GUI element. The last part of the process name must provide a GUI page number (e.g., Page 1) to indicate on which GUI page the process is applied. For example, the process *Register_Customer_Page1* corresponds to the button named “Register Customer”, although its name does not appear on Page 1. Further, if the process requires to operate on a data element (e.g., ID), it must be written as an input parameter of the process. If there is a need to express multiple choices of input parameter groups, the symbol | must be used to separate the input parameter group, similar to what we write for an ordinary process specification in a semi-formal specification. If the process causes the change of the current GUI page to another page, say Page 2, then the Page to be changed to (e.g., Page 2) must be written as an output variable. The process *Input_Customer_Information_Page2* is a good example to illustrate these features of a process specification. It has six input parameters, including ID, Password, and Name. It states that once the specific values for the six input parameters are provided, the current GUI page will be changed to Page 4, which is omitted for brevity.

Since a GUI for a software system is rather simple compared with the system itself, Agile-SOFL advocates that only one GUI module is constructed to specify the operations on all the GUI pages and their connections. However, if a large-scale and/or complex system requires a complex GUI specification, more than one GUI module can be constructed to allow each module specification to concentrate on one group of GUI pages and to let all the module specifications cover all the GUI page groups.

4.2.5 Formal Specification

For safety-critical operations, security-critical operations, and business-critical operations of a large-scale software system that are difficult to be clearly specified informally, we suggest that they should be specified using the SOFL formal language in the Agile-SOFL method, if the development environment allows. Such a formalization can significantly help resolve the potential ambiguities in informal specifications

and aid the developer to deepen his or her understanding of the desired functionality of the relevant processes. It can also set up a firm foundation for automated code testing and verification, see Chaps. 8 and 9 for details. Of course, this principle can also be applied to other formal notations.

Formalization can be applied to two components in a module of a semi-formal specification. One is the invariants given in the *inv* section and the other is process specifications. By formalization we mean that the invariant and the process specification are all written in the SOFL language to replace the informal expressions. Thus, all the target invariants and processes will be specified using predicate expressions.

In the STS system, we choose to formalize two invariants and one process as an example. One invariant is “Every telephone number in the type *TelephoneNumber* has exactly 11 digits”, which is a type invariant and another is “Every customer on the *customer_list* has a unique id”, which is a state invariant. The process is *Register_Stock*. Their formal specifications are given below.

```

inv
  forall[tel: TelephoneNumber] | len(tel) = 11;
  forall[c1, c2: elems(customer_list)] | c1 <> c2 => c1.id <> c2.id;

process Register_Stock(stock: StockInformation) res: bool
  ext wr stock_list: seq of StockInformation
  FSF:
    stock notin elems(~stock_list) &&
    stock_list = conc(~stock_list, [stock]) and res = true
  ||
    stock inset elems(~stock_list) &&
    stock_list = ~stock_list and res = false
  end_process

```

In this formal specification, we use several operators, such as *notin*, *elems*, *conc*, and *inset*, which are defined in the SOFL specification language. Their definition and explanation can be found in Chap. 2.

Chapter 5

Three-Step Hybrid Specification Approach



As described in the previous chapter, hybrid specification is a well-balanced document to define what to be done by the potential system, but writing it properly to accurately and completely reflect the user's requirements is not necessarily a simple job. In this chapter, we describe a *three-step hybrid specification approach* (TSHSA) to facilitate developers to specify requirements using the Agile-SOFL method. The TSHSA is characterized by supporting interactions between the developer and the user, the user collaboration, and responding to changes during the specification process.

As illustrated in Fig. 5.1, the three steps in the TSHSA are known as *informal specification*, *semi-formal specification* + *GUI specification*, and *formal specification*. The informal specification results from studying the domain knowledge and communications between the developer and the user. It is intended to briefly but completely state requirements for the system to be developed. Of course, the completeness may not be achieved at one time and iteration of interactions between the developer and the user is usually inevitable. The semi-formal specification and the GUI specification are obtained by extending and clarifying the informal specification. During this process, the developer usually needs to collaborate with the user to make right decisions in interpreting the informal specification into the semi-formal specification and the GUI specification. The formal specification is a further clarification of the informal expressions in the semi-formal specification. To achieve the agility of the whole development project, it is suggested only to apply it to critical parts of the whole system specification.

The TSHSA explained above is designed for software projects starting from scratch, but many of the projects in software industry may not fall into this category. They may be constructed based on existing software components or small systems. Nevertheless, even in this circumstance, the TSHSA can be applied, perhaps more efficiently, in the way that the functions, data, and constraints for the existing components are incorporated into the informal specification and the semi-formal specification. Since the developer is supposed to be familiar with the components, utilizing them in the specifications to be written should be much easier than obtaining them from the user.

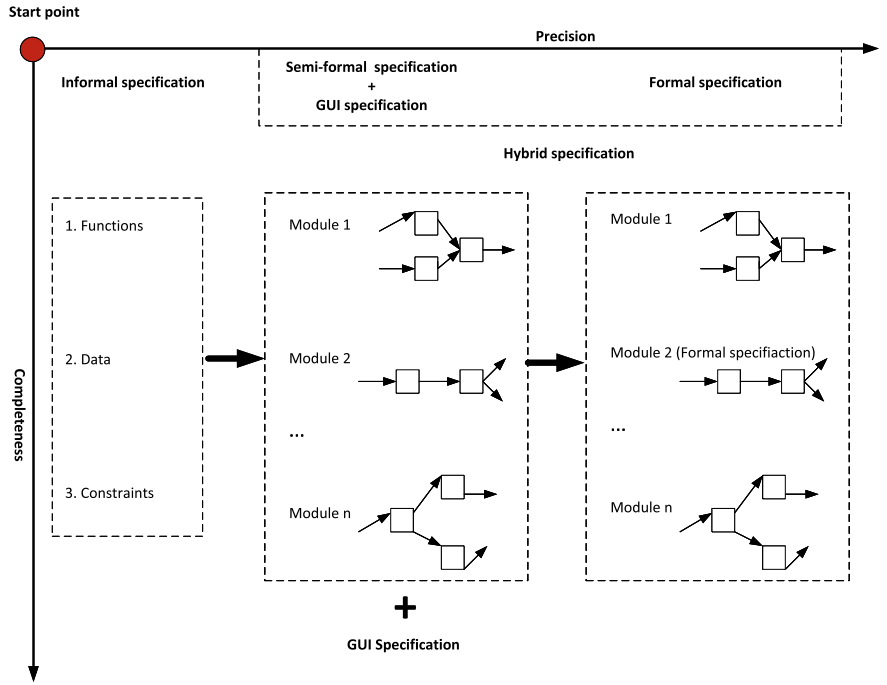


Fig. 5.1 An illustration of the three-step hybrid specification approach

Below we will discuss in detail the three kinds of specifications separately, including how the specification can be written, how its contents can be derived based on the preceding specification, and what quality each specification should try to achieve.

5.1 Informal Specification

An informal specification is the first document for building a software system that is intended to reflect the user’s requirements. As mentioned previously, it usually results from communications between the developer and the user, from learning the domain knowledge, or from both. To efficiently obtain the user’s requirements, it is quite natural and effective to ask the user what services are expected to supply by the potential system. Such a service usually corresponds to a function implemented by an operation in the system. Therefore, when building an informal specification, we treat the job of learning about the desired functions as the most important thing to start with. We then proceed to analyze and determine what data items are needed to realize the functions in the system. To ensure that both the functions and data items work properly, we need to examine and learn what potential constraints need to be defined on the relevant functions and/or data items.

We call this way of thinking and working to build an informal specification “market-driven approach” (MDA). The “market” in the MDA refers to the functions. Clearly understanding the desired functions for the system is like to understand the need of the “market”. Whatever data items and constraints required to build appropriate functions in the “market” will need to be described properly in the corresponding section of the specification.

It is ideal to document all the desired requirements in the informal specification, but unfortunately this is rather hard to realize in practice, since learning new things (e.g., requirements) is usually an evolutionary process. We must take an iterative process to fulfil the goal but some techniques as discussed below may help reach the goal efficiently.

5.1.1 Describing Functions

In principle, all the required functions should be documented in the *Functions* section of the informal specification. When describing the functions, we need to pay attention to the following points for a good readability:

- Each function description must be concise.
- Each function should be assigned a number to distinguish from others.
- A complex function can be broken down into simpler functions, but the relation between the complex function and its decomposition must be clearly indicated using the hierarchical numbering system, as explained in Chap. 3.

An important merit of writing the specification is to help prevent requirements-related faults. If a desired function is not documented in the specification, the software to be constructed will probably encounter errors. If a desired function is acquired but defined wrongly in the specification, the software will also likely to encounter errors. To help prevent these kinds of errors, we should try to include the following functions in the *Functions* section:

- input and output functions
- processing functions
- “symmetric” functions.

In general, a software system needs to interact with its user in operation, therefore providing functions for input and output are usually necessary. Further, the system must process data in order to provide the desired services for the user. For this purpose, processing functions need to be defined. To ensure that the system has a full coverage of the functions, trying to provide “symmetric” functions can help fulfil this goal. By “symmetric” functions, we mean the functions that provide the complementary but “opposite” services. That is, if the system has one function performing some task, it must also possess another function performing the “opposite” task. For instance, in the STS given in the previous chapter, “registration” verse “cancellation”

(e.g., customer registration and cancellation); in an ATM system, “withdraw” verse “deposit”; and in a library system, “borrow book” verse “return book”.

5.1.2 Describing Data

Data are the target for the functions to process, transform, and/or arrange in the system. If a necessary data item is not acquired, the system will encounter errors. If a necessary data item is acquired but defined wrongly in the specification, the system will encounter errors as well. Therefore, in the *Data* section, we should try to describe all the necessary data items for the specified functions. Since there are many data items concerned with the specified functions, in this section we only focus on those that must be shared by more than one function given in the *Functions* section. The data items that are used as input and/or output parameters of a function will be properly defined in the corresponding specification of the function in the semi-formal specification to be constructed later. It seems extremely difficult to come up with a general technique for capturing all the necessary data items, but as long as the developer keeps thinking what data items are required to fulfil the specified functions, a good solution would likely be found.

5.1.3 Describing Constraints

Many functional services are provided under certain constraints. For example, the same customer can only buy at most 1000 stocks of the same kind in the STS described in the previous chapter. Some systems may also need restrictions on data items. For example, the age of a person must be greater than zero. In the *Constraints* section, all the important constraints on both the functions and the data items should be presented.

5.1.4 Building Connections

In general, functions use data items under certain constraints to provide expected services. Therefore, building connections between functions, data items, and constraints is helpful in understanding how they should cooperate to work properly in the system. Trying to build such connections can also help the developer prevent potential errors to be possibly committed during the descriptions of the functions, data items, and constraints.

The way to build the connections is to utilize the “reference” system as explained in Chap. 3. If a data item d is used by the function numbered 1, we will write d (F_1) when d is described in the *Data* section. If the same data item d is used by several

different functions numbered, for example, 1, 2, 4, then the description of d will be d (F_1, F_2, F_4). Similarly, for a constraint description, we can do the same thing. Since a constraint may also be defined on a data item given in the *Data* section, we can extend this method to allow the constraint to be connected to the corresponding data item. In this case, F should be replaced by D. For example, if a constraint c has something to do with the function numbered 1 and the data item numbered 3, the c description will be written as c (F_1, D_3). If constraint c is connected to more than one function and data item, the references of all the functions and data items can be written in parentheses separated by commas.

Note that there is no need to provide similar references to data items and constraints in a function description given in the *Functions* section, since the references from the data items and the constraints allow the developer clearly understand the connection between functions, data items, and constraints. This omission of references from function descriptions can also help save time and cost in writing good quality informal specifications.

5.1.5 Example

Using the guidelines given above, we extend the simplified stock trading system (STS) with more necessary functions and decompositions of complex functions below.

Informal specification

I. Functions

- 1 Register customer (name, age, bank account, address, email address, telephone number, and purchased stocks)
- 2 Cancel customer
- 3 Register stock (name, price, unit for sale, limit for selling to one customer)
- 4 Cancel stock
- 5 Purchase stock
 - 5.1 Check the condition for purchasing stock
 - 5.2 Record the purchased stock in the customer's account on the Customer list
 - 5.3 Update the stock data on the Stock list
 - 5.4 Perform the payment from the customer's bank account and Provide a receipt for the customer
- 6 Sell stock (the money resulted from the sale of stock should be put into the customer's bank account).

II. Data

- 1 Customer list (F_1, F_3, F_4)
- 2 Stock list (F_2, F_3, F_4)
- 3 Bank accounts (F_3, F_4).

III. Constraints

- 1 For each kind of stock, a customer can only buy at most 1000 units of the stock (F_3, F_5, D_2)
- 2 A stock name is unique (D_2)
- 3 The currency used in the system is only Japanese yen (JPY) (F_3, F_4).

Following the guidelines for describing functions, we provide six functions in the *Functions* section. All the six functions involve interactions between the user and the system. Functions 1 and 2 as well as 3 and 4 are “symmetric” functions, respectively, and functions 5 and 6 can be regarded as processing functions. Since function 5 is a little complex, it is decomposed into five subfunctions, numbered 5.1 to 5.4, respectively. Moreover, in the *Data* and *Constraints* sections, we list the most important data items that are shared by the functions and constraints we can think of for now.

It is worth pointing out that the lists of functions, data items and constraints may not be complete for implementing the STS system and how to ensure a complete list may affect the agility and practicality of the Agile-SOFL method. If we require that the informal specification should record all the desired functions, data items, and constraints, then the developer will inevitably need to come back from the subsequent steps of specification (e.g., semi-formal specification, formal specification) to improve the informal specification. The advantage of this practice is to help maintain the consistency between the informal specification and the hybrid specification to be written later, but the downside is to consume more time and labor for the evolution of the informal specification, therefore affecting the agility of the development.

In Agile-SOFL, we adopt an alternative way to address this problem. The completeness of the functions, data items, and constraints will be gradually provided during the construction of the hybrid specification in the clarifying requirements phase. In other words, we treat the informal specification as the first version of the system specification, which may be incomplete, and try to achieve the completeness of the requirements ultimately in the final hybrid specification. In this way, the developer does not need to come back to improve the informal specification. To ensure a good traceability of requirements, building trace links between the elements (e.g., functions, data items) of the informal specification and those of the hybrid specification is important. Such trace links are expected to reflect the idea of which element of the informal specification is represented (or “implemented”) by which elements of the hybrid specification. For the elements of the hybrid specification that do not have the corresponding elements in the informal specification, we must clearly build a trace link from the developer to them, meaning that they are added by the developer during the clarification of the informal specification into the hybrid specification. If the developer does this based on the interaction with the user, then the details of such an action should be clearly recorded to help enhance the readability of the hybrid specification and the traceability of the requirements. It will also help clearly indicate the responsibility of the relevant developer in the development team.

5.2 Semi-formal Specification + GUI Specification

The second step in the TSHSA is to build a semi-formal specification together with a GUI specification. The reason for building these two kinds of specifications in the same step is that they are closely associated with each other and their constructions interact with each other in the sense of providing necessary data items and reasons for improving the functional specifications. Writing GUI specification helps the user-developer collaboration in understanding the desired functions and data items precisely. It also provides a clear basis for properly specifying the related functions in the semi-formal specification.

The second step should start with writing the semi-formal specification. During this process, the developer will be able to identify the functions that need to directly communicate with the user of the system. Such functions should then be well considered in the GUI specification because the GUI is responsible for interactions with the user. In the meanwhile, during the GUI specification, the necessary input and output data items for each function can be clarified, which can then help complete the specification of the related functions.

5.2.1 *Techniques for Semi-formal Specification*

As described in Chap. 4, a semi-formal specification is a collection of modules together with the corresponding CDFDs. We discuss techniques for building each individual module and its elements in this subsection.

5.2.1.1 Building Modules

Writing the semi-formal specification should take advantage of the informal specification by doing five things. First, grouping the related functions, data items, and constraints into one module in the semi-formal specification. In such a module, each function in the informal specification should be represented by a process in the SOFL language; each data item should be represented by a store variable declared in the var section of the module; and each constraint can be represented by an invariant in the inv section of the module or incorporated into a process specification. Second, formally declaring all the necessary constants, types, and store variables in a module using the SOFL language. Third, specifying invariants in the inv section in natural language (i.e., informally). Four, specifying every process in pre- and post-conditions but both conditions should be written in natural language. Finally, constructing a CDFD for the module to define the data flow dependence relation between the specified processes in the same module, if necessary.

Experience suggests that the “market-driven approach” can be effective in helping us build a module. Using this approach, we first try to work out the necessary

processes in the module. Then, try to write the specification for each process during which the necessary constant identifiers and type identifiers will be declared in the corresponding sections, respectively. After these declarations, we need to consider what invariants on types and store variables should be defined in the *inv* section. On the basis of these descriptions, we can come back to concentrate on the writing of the pre- and post-conditions of the process specification. During this process, the relevant declarations might be modified to suit the need for completing the process specification properly.

5.2.1.2 Writing Process Specifications

The readability of a process specification is extremely important because the specification is intended for humans to understand rather than for machine to execute. We discuss how good readability can be achieved by means of functional scenario form, appropriate reference of variables, comprehensible syntactical expressions, and the modularity of pre- and post-conditions.

Functional Scenario Form

As stated in Chap. 4, a functional scenario form (FSF) is a disjunction of functional scenarios. Each functional scenario (FS) is a conjunction of a test condition and a defining condition, defining an independent behavior of the process: when the test condition is satisfied by the input, the output of the process will be defined by the defining condition (i.e., the output must satisfy the defining condition).

The reason why FSF is comprehensible is that it describes all the possible “use cases” of the process at the logical level and each FS focuses on one “use case”. This concept is familiar to ordinary customers and developers in practice, therefore helping achieve a good readability.

The general format of a process specification used in Agile-SOFL is given as follows:

```

process ProcessName(input variable declarations)
    output variable declarations
ext wr/rd store variables
FSF
 $T_1 \ \&\& \ D_1$ 
||
 $T_2 \ \&\& \ D_2$ 
||
...
||
 $T_n \ \&\& \ D_n$ 
end_process.
```

Variable References

Table 5.1 The tabular format of FSF in a process specification

Test conditions	FS1	FS2	...	FSn
T_1	✓			
T_2		✓		
...				
T_n				✓
Defining conditions				
D_1	✓			
D_2		✓		
...				
D_n				✓

Since a process provides its functionality through dealing with the related input, output, and external data items, a clear expression of how the variables declared in the process signature are used in the FSF becomes the key to ensure a good readability of the process specification. In other words, if we actively use the declared variables in the FSF, it will help improve its readability. We call this technique *variable-reference technique* and it will be further illustrated in the example to be given below.

Comprehensible syntactical expression

The FSF of a process specification is usually described in textual format, but it can also be given in the tabular format as shown in Table 5.1, which may have a better readability for some people. The first column of the table is divided into two parts: test conditions and defining conditions. Both conditions are given in natural language. From the second column to the right, each column describes a single functional scenario by marking the test condition and its corresponding defining condition with a tick ✓. For example, the first functional scenario FS1 is T_1 & D_1 .

Of course, that a tabular format is more comprehensible than a textual format may be controversial and there is no well-recognized scientific evidence to support this claim. If one likes the tabular format, one can adopt it in his or her own project. If not, one can keep using the textual format, or any other format he or she prefers to use. It is just a matter of personal preference.

5.2.2 Example of Semi-formal Specification

Let us take the simplified STS introduced in Chap. 3 as an example to explain how the above techniques can be used to construct a semi-formal specification based on the corresponding informal specification. Applying the techniques for writing modules and process specifications, we obtain the semi-formal specification given below, which is an extension of the partial semi-formal specification given in Chap. 4.

The whole STS specification includes three modules: *Registration*, *TradeStocks*, and *Purchase_Stock_Decom*. The module *Registration* is concerned with the definition of the functions in relation to customer and stock registrations, respectively, and the module *TradeStocks* is used to define the functions related to operations on stock trading. *Purchase_Stock_Decom* is the decomposition of the process *Purchase_Stock* specified in the module *TradeStocks*. We use the textual format for most process specifications in the modules but use the tabular format only for one process in the module *TradeStocks* to show an example of its usage.

```

module Registration;
type
  CustomerInformation = composed of
    id: IDstring
    passw: Password
    name: string
    tel_no: TelephoneNumber
    email_add: EmailAddress
    address: string
    owned_stocks: seq of PurchasedStock
  end;

IDstring = string; /*containing exactly 10 characters*/
TelephoneNumber = seq of nat0 /*containing exactly 11 digits*/
Password = seq of nat0; /*the password contains exactly 6 digits*/
EmailAddress = LocalPart * At * DomainName;
LocalPart = string;
At = {@};
DomainName = string;
StockInformation = composed of
  s_id: IDstring /*containing exactly 10 characters*/
  s_name: string
  company: Company
  unit_price: real
  max_sale_units: nat0
  /*the stock units that can be sold for one customer. */
  stock_units_for_sale: nat0
  /*the stock units available for sale. */
  end;

PurchasedStock = composed of
  stock_info: StockInformation
  no_of_units: nat0
  buy_unit_price: real
  currently_evaluated_unit_price: real
  current_total_value: real
  end;

```

```

Company = composed of
    name: string
    representative: PersonName
    address: string
    tel: TelephoneNumber
    email: EmailAddress
end;

PersonName = FirstName * MiddleName * FamilyName |
    FirstName * FamilyName;

FirstName = string;
MiddleName = string;
FamilyName = string;

var
customer_list: seq of CustomerInformation;
stock_list: seq of StockInformation;
inv
1. Every element in the type IDstring contains exactly 10 characters;
2. Every telephone number in the type TelephoneNumber has exactly 11 digits;
3. Every customer on the customer_list has a unique id;
4. Every stock on the stock_list has a unique s_id;
5. Every stock on the stock_list has a unique s_name;
behav CDFD_No.1;
process Register_Customer(customer: CustomerInformation) mesg: bool
ext wr customer_list: seq of CustomerInformation
FSF:
The input customer is not included in the  $\sim$ customer_list &&
the customer_list contains the customer and mesg = true
||
The input customer is included in the  $\sim$ customer_list &&
the customer_list is the same as  $\sim$ customer_list and mesg = false
end_process;
process Cancel_Customer(id: IDstring, name: string) notice: string
ext wr customer_list: seq of CustomerInformation
FSF:
The input id and name exist in the  $\sim$ customer_list &&
the customer_list does not include the customer whose
id and name are the same as the input id and name and
notice = "Success!"
||
The input id and name do not exist in the  $\sim$ customer_list &&
the customer_list is the same as  $\sim$ customer_list and
notice = "The customer is not registered in the system!"
end_process;
process Register_Stock(stock: StockInformation) res: bool
ext wr stock_list: seq of StockInformation

```

FSF:

The input *stock* does not exist in the *~stock_list* &&
the *stock_list* contains the *stock* and *res = true*

||

The input *stock* exists in the *stock_list* &&
the *stock_list* is the same as *~stock_list* and *res = false*
end_process

process *Cancel_Stock(s_id: IDstring, company: Company) notice1: string*
ext wr *stock_list: seq of StockInformation*

FSF:

The input *s_id* and *company* exist in the *~stock_list* &&
the *stock_list* contains no stock whose id and company
equal the input *s_id* and *company*, respectively and
notice1 = “The cancellation of the stock is successful!”

||

The input *s_id* and *company* do not exist in the *~stock_list* &&
the *stock_list* is the same as *~stock_list* and
notice1 = “The stock is not registered in the system!”
end_process
end_module;

This module results from grouping functions 1, 2, 3, and 4, data items 1 and 2, and constraint 2 given in the informal specification. The correspondence between the informal specification elements and the semi-formal specification elements is summarized in Table 5.2 in which **IS_elements** and **SFS_elements** stand for informal specification elements and semi-formal specification elements, respectively. This correspondence can be treated as the trace links for the informal specification elements in the system. All the other type identifier declarations given in the type section are necessary for writing the semi-formal specification and they are derived during applying the “market-driven approach”. Note that the type identifier declarations may have introduced more type identifiers than necessary in order to ensure a good readability. For example, the type identifier declarations

PersonName = *FirstName* * *MiddleName* * *FamilyName* |
FirstName * *FamilyName*;

FirstName = *string*;
MiddleName = *string*;
FamilyName = *string*

can be simplified into the following:

PersonName = *string* * *string* * *string* |
string * *string*,

but this declaration may be more difficult to understand than the previous ones. Striking a good balance between the simplicity and understandability is important in writing specifications but how it can be achieved usually depends on the practitioner’s experience and engineering judgement.

Apparently, when writing a process specification, we need to work out the specific input, output, and external variables necessary for the process and to choose right

Table 5.2 The correspondance between the informal specification elements and the semi-formal specification elements

IS_elements	SFS_elements
Function 1	Process <i>Register_Customer</i>
Function 2	Process <i>Cancel_Customer</i>
Function 3	Process <i>Register_Stock</i>
Function 4	Process <i>Cancel_Stock</i>
Data item 1	Store variable <i>customer_list</i>
Data item 2	Store variable <i>stock_list</i>
Constraint 2	Invariant 5

types to declare them, respectively. We also need to precisely understand the required functionality of the process and figure out how it can be expressed properly in the FSF. Since all these detailed information is not provided in the informal specification, the developer is required to create the details based on his or her understanding of the user’s requirements (through communications or interactions with the user) and experience of building similar software systems before.

To make each process specification comprehensible enough, we apply the variable-reference technique discussed previously. That is, whenever we want to express a condition that needs to refer to some data item represented by an input variable, output variable, or external variable, we use the variable in the expression. Let us take the process *Register_Stock* as an example. In the first functional scenario of the FSF of this process, that is,

The input *stock* does not exist in the *~stock_list* &&
the *stock_list* contains the *stock* and *res = true*,
we use the related variables *stock*, *~stock_list*, *stock_list*, and *res* when we need to express some ideas related to them in the functional scenario. This variable-reference technique is highly effective in ensuring the readability of process specifications according to our experience. Readers are recommended to find out the effect by themselves through practice.

The four processes given in this module do not have a strong data flow dependency with each other but interact with the data stores *customer_list* and *stock_list*. In this case, the CDFD associated with the module may be omitted. However, since drawing the CDFD may help the developer confirm the appropriateness of the process specifications and the CDFD may provide a helpful guideline for stakeholders to read and understand what the module is expected to do, drawing a CDFD for the module is highly recommended if it is possible. Figure 5.2 shows the CDFD for the module *Registration*.

The details of the module *TradeStocks* are given below.

```
module TradeStocks;  
type  
  Registration.IDstring; /*Within this module, we will use IDstring to refer  
to this type defined in the module Registration*/  
var  
  Registration.stock_list /*we will use stock_list to refer to this external vari-  
able declared in the module Registration*/
```

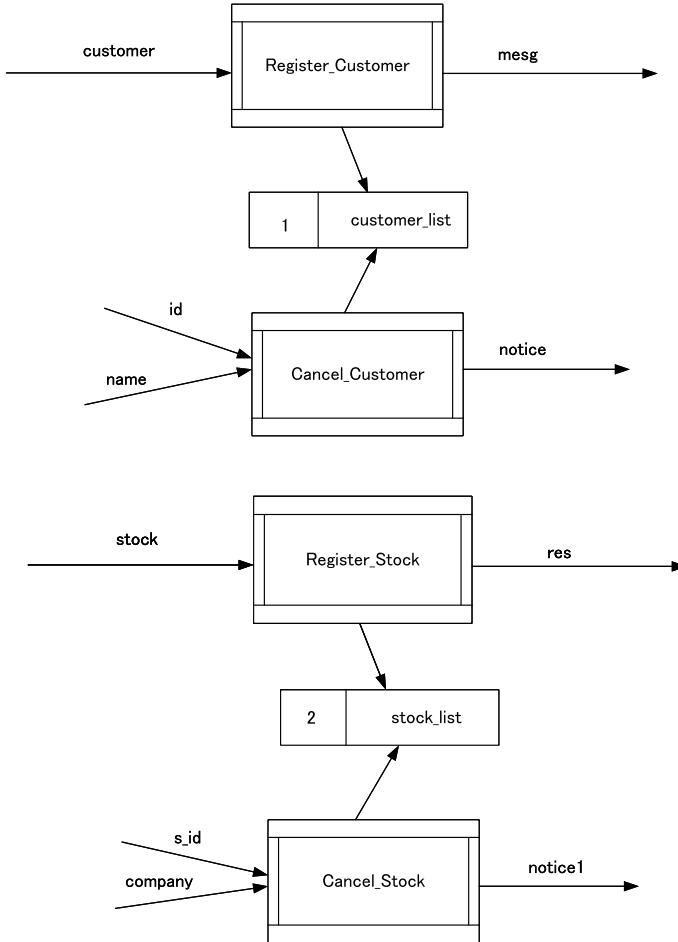


Fig. 5.2 No. 1: the CDFD for the module *Registration*

Registration.customer_list /*we will use *customer_list* to refer to this external variable*/

```

process Purchase_Stock(stock_id: IDstring, number_of_units: nat0)
  ps_mesg: string
ext wr stock_list
  wr customer_list
  decom Purchase_Stock_Decom /*indicating that this process is decomposed
into the module called Purchase_Stock_Decom*/
end_process;
process Sell_Stock(stock_id: IDstring, customer_id: IDstring,
  number_of_units: nat0)

```

```

                ss_mseg: string
    ext wr stock_list
        wr customer_list
    FSF:

```

Test conditions	FS1	FS2
The <i>stock_id</i> exists in the owned stock list of the customer with the <i>customer_id</i>	✓	
The <i>stock_id</i> does not exist in the owned stock list of the customer with the <i>customer_id</i>		✓
Defining conditions		
the <i>stock_list</i> is updated by adding the <i>number_of_units</i> properly and the <i>customer_list</i> is updated by deleting the stocks with the <i>stock_id</i> and the customer's balance is updated by adding the money resulting from selling the stocks to the customer's balance	✓	
both the <i>stock_list</i> and the <i>customer_list</i> are all unchanged		✓

```

end_process;
end_module;

```

This module includes two processes, *Purchase_Stock* and *Sell_Stock*, and two store variables, *Registration.stock_list* and *Registration.customer_list*. *Purchase_Stock* is decomposed into a low level module called *Purchase_Stock-Decom*, which is given later in this section. For this reason, there is no need to provide the FSF part for this process because its functionality will be defined by its decomposition. *Sell_Stock* is rather simple, therefore, its FSF is given in detail. To illustrate how the tabular format can be used to express the FSF of a process, we use it for the process *Sell_Stock*.

As far as the two store variables are concerned, they are actually declared in the module *Registration*. In Agile-SOFL, we use *module.variable* to mean the import of the *variable* declared in the *module* into the current module so that the *variable* can be used directly in the current module. For example, *Registration.stock_list* imports the variable *stock_list* declared in the module *Registration* into the current module *TradeStocks*. Therefore, it can be directly used in process specifications of this module, such as `ext wr stock_list` given in the specification of the process *Sell_Stock*. The same principle can also be applied to the import of type identifiers. For instance, in the type section of the module *TradeStocks*, *Registration.IDstring* means that the type identifier *IDstring* declared in the module *Registration* can be used directly to declare variables wherever necessary. For instance, in the specification of process *Purchase_Stock*, the variable *stock_id* is declared directly using the type *IDstring*, that is, *stock_id: IDstring*.

Further, since the two processes in this module do not have a strong data flow dependence, the corresponding CDFD of the module is omitted.

The process *Purchase_Stock* is decomposed into the module *Purchase_Stock-Decom* given below. The contents of the module are provided mainly based on the functions 5.1 to 5.4, data 1 to 3, and constraints 1 and 3 in the informal specification given in Sect. 5.1.5.

```

module Purchase_Stock_Decom;
const
  stock_purchase_max_unit = 1000; /*the greatest number of stock of one kind
for a customer to buy once */
type
  Registration.IDstring;
  BankAccount = composed of
    a_id: IDstring
    a_password: nat0 * nat0 * nat0 * nat0
    name: string
    balance: nat0
  end;
  SuppliedInformation = composed of
    s_stock_id: IDstring
    s_customer_id: IDstring
    s_number_of_stocks_to_buy: nat0
  end;
  Receipt = composed of
    r_customer_id: IDstring
    r_stock_id: IDstring
    r_cost: nat0
  end;

var
  Registration.customer_list;
  Registration.stock_list;
  bank_accounts: seq of BankAccount;
  behav CDFD_No.2;
  process Check_Condition(
    stock_id: IDstring,
    customer_id: IDstring,
    number_of_stocks_to_buy: nat0)
    supplied_info: SuppliedInformation | failure_mesg: string
  ext rd customer_list
    rd stock_list
    rd bank_accounts
  FSF:
    The stock_id exists in the stock_list and
    the customer_id is included in the customer_list and
    the number_of_stocks_to_buy <= stock_purchase_max_unit and
    the customer's bank account balance >= the cost of purchasing the stock &&
    The fields s_stock_id, s_customer_id, and s_number_of_stocks_to_buy of
    supplied_info are equal to the input values stock_id, customer_id, and
    number_of_stocks_to_buy, respectively.
  ||
  The stock_id exists in the stock_list and
  the number_of_stocks_to_buy > stock_purchase_max_unit &&

```

```

failure_mesg is equal to "The number of the stocks to buy is over the limit!"
||
The stock_id exists in the stock_list and
the customer's bank account balance < the cost of the requested number of stocks
&&
failure_mesg is equal to "The customer's account balance is not sufficient for
buying the requested number of stocks!"
||
The stock_id does not exist in the stock_list &&
failure_mesg is equal to "The stock to be purchased is not registered!"
||
The customer_id does not exist in the customer_list &&
failure_mesg is equal to "The customer is not registered!"
end_process;
process Handle_Stock(supplied_info: SuppliedInformation)
    supplied_info1: SuppliedInformation,
    cost: nat0
ext wr customer_list
    wr stock_list
FSF:
true &&
customer_list is updated by adding the stock purchased for
the customer with the customer id supplied_info.s_customer_id and
stock_list is updated by removing the stock with the supplied_info.s_stock_id
purchased by the customer with the supplied_info.s_customer_id and
the cost is calculated and supplied_info1 = supplied_info
end_process;
process Perform_Payment(supplied_info1: SuppliedInformation, cost:
nat0)
    receipt: Receipt
ext wr bank_accounts
FSF:
There exists a unique bank account in bank_accounts such that its id is the same
as the supplied_info1.s_customer_id &&
The bank_accounts is updated by reducing the cost from the account balance
of the customer with the same id as supplied_info1.s_customer_id in the
bank_accounts and the receipt is produced by reporting the customer id, name,
stock id, and the cost.
||
others && receipt = nil and bank_accounts remains unchanged.
end_process;
end_module.

```

Note that the process *Handle_Stock* has only one functional scenario in its FSF and the test condition of the scenario is *true*, indicating no specific constraint on the input variables. The process *Perform_Payment* has a similar situation.

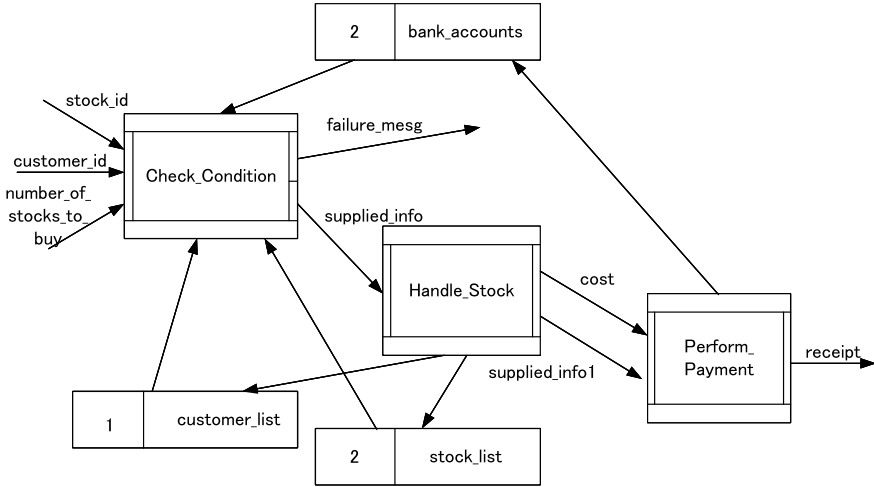


Fig. 5.3 No. 2: the decomposition of the process *Purchase_Stock*

This module is constructed by applying the “market-driven” approach. We declare a constant identifier *stock_purchase_max_unit* to represent the maximum stock units for purchasing mentioned in the constraint 1 in the informal specification. We also declare necessary type identifiers, such as *BankAccount* and *SuppliedInformation*, and three store variables: *Registration.customer_list*, *Registration.stock_list*, and *bank_accounts*. The functions 5.1 to 5.4 in the informal specification are represented and specified by the three processes: *Check_Condition*, *Handle_Stock*, and *Perform_Payment*, where functions 5.2 and 5.3 are merged into process *Handle_Stock*. As mentioned before, the basic idea is to use one process to specify one function given in the informal specification, but this idea can be flexibly applied in practice by merging several functions into one process or breaking up one function into several processes or properly combining these two strategies.

Since the three processes in this module has a strong data flow dependence, we draw the corresponding CDFD to express the dependence relation, as shown in Fig. 5.3.

5.2.3 Techniques for GUI Specification

As described in Chap. 4, a GUI specification should focus on the definition of the elements used in each GUI page and how pages are connected with each other. When writing a GUI specification, we face the natural questions: how can we get to know what elements should be included in each page and how can each page be connected

to other pages. There is no magic, but writing the semi-formal specification could provide opportunities for the developer to think about this issue.

When specifying the semi-formal specification, we need to pay attention to the processes representing the functions involving interactions with the user of the system. Then, we try to work out what input data items should be used for such a process and what output data items should be produced by the process. After having an initial idea, we try to figure out how the input and the output data items can be represented properly in a related GUI page and confirm their accuracy by consulting with the user on the GUI page.

As far as the GUI page transition is concerned, we can work out the details by defining a process for each desired transition. By trying to write a semi-formal specification for the process we can gradually figure out what we need to do on the current GUI page and what other page with what data items should be changed to. Specifically, we model the operation of changing the current GUI page to another page as a process and try to specify its functionality to clearly show how the current page can be changed to another.

For example, suppose the GUI page given in Fig. 4.2 in Chap. 4 is the current page. If we expect that clicking on the button “Register Customer” will make the transition from the current page to the GUI page in Fig. 4.3, we should specify a process in the module called GUI, for example, *Register_Customer_Page1*, to show what action is necessary to change to page 2. From the specification of this process given in Sect. 4.2.4 in Chap. 4, we can see that this process needs to take an action on the button Register Customer in order to make the transition from page 1 to page 2.

The above techniques can be used to effectively construct the module *GUI* and its corresponding GUI pages. The GUI pages can be designed with any suitable tool and the Microsoft PowerPoint can be a convenient choice. In fact, writing the related semi-formal specification can provide some ideas on the details of the elements for each GUI page. How each page can be changed to another page in the run time could be figured out by trying to design and specify a process responsible for the page transition. During this process, interactions between the developer and the user may be unavoidable for making right decisions.

5.3 Formal Specification

The formal specification step aims to formalize the critical invariants and critical processes in modules in the semi-formal specification. To this end, we must first select the critical invariants and processes for formalization, but how to determine them from the semi-formal specification is a challenge. While this usually needs to be done based on the practitioners’ engineering judgement, a general criterion to use for the decision is whether the data items involved in the invariant and/or the process represent critical data (e.g., money, password, engine control signal) that are likely to fail the operation of the whole system (e.g., ATM, automobile) to lead

to disastrous consequences (e.g., loosing money, leaking confidential information, causing catastrophic disasters).

5.3.1 *Preparing Skills for Formalization*

Once an invariant or a process is understood as a critical one, we can clarify its definition by refining its informal expression given in the semi-formal specification into a formal one. During this clarification, some further decisions may need to be taken to ensure that the formal expressions accurately reflect the user's requirements. Perhaps the most difficult issue for many practitioners is how a given informal expression can be properly transformed into a formal expression.

While this is a difficult question to answer in writing a pre-post style specification, it would be much easier to handle in writing an FSF style specification since an FSF gives us a clear instruction for writing the specification: concentrate on formalizing one functional scenario and then proceed to another one until the whole FSF is completed. To help the formalization of each functional scenario, the author's experience suggests that the following three things should be helpful: (1) mastering the specification language (e.g., SOFL), in particular the operators defined on each data type (e.g., basic types, set types, sequence types, composite types), (2) getting used to thinking and expressing conditions in predicate logic, (3) gaining experience through practice.

5.3.2 *Handling Quantified Predicate Expressions*

When quantified predicate expressions (or simply quantified expressions) are used in an FSF style specification, we face a challenge of how to properly separate the test condition and the defining condition in a functional scenario. In the universally quantified expression $\text{forall}[x: T] \mid P(x)$, x is a bound variable, taking values from its range T , and the predicate expression $P(x)$ is its scope. In other words, x can only be used in $P(x)$. Similarly, in the existentially quantified expression $\text{exists}[x: T] \mid P(x)$, x is a bound variable and $P(x)$ is its scope.

Suppose we want to express an idea in a functional scenario of a process specification that if input x exists in a sequence a represented by an external variable, then the output p will be defined to represent the position of the first appearance of x in a . In traditional pre-post style specification, we may write it in the post-condition as $\text{exists}[i: \text{inds}(a)] \mid a(i) = x \text{ and } p = i$, where i is the bound variable and the conjunction $a(i) = x \text{ and } p = i$ is its scope. However, when converting this expression into a functional scenario, we cannot simply write it as

$$\text{exists}[i: \text{inds}(a)] \mid a(i) = x \ \&\& \ p = i,$$

because the bound variable i occurring in the defining condition $p = i$ is used outside its scope $a(i) = x$, therefore making no good sense. In order to avoid such a mistake, we need to write the above idea properly. The following is a possible expression:

$x \text{ inset } elems(a) \ \&\& \ p = hd([i: inds(a) \mid a(i) = x]).$

This functional scenario states that when x is a member of sequence a , p will be defined as the position of its first appearance in a . In this expression, we use several operators, such as sequence comprehension $[i: inds(a) \mid a(i) = x]$, the head operator hd , indexes operator $inds$, and elements operator $elems$, which are all defined on sequence types, as briefly introduced in Chap. 2.

We may utilize a local variable to simplify the expression of the functional scenario if we want to express the idea: if there exists an unique x in sequence a , then the output p will be defined to represent the position of its appearance in a . The simplified functional scenario can be the one as follows:

$exists![i: inds(a) \mid a(i) = x \text{ and } pos = i] \ \&\&$
 $p = pos,$

where pos is declared as a local variable of the process whose FSF includes this functional scenario as its part. Another example of using a local variable can be found in the formal specification of the process *Perform_Payment* given below.

5.3.3 Example of Formal Specification

Let us take the process *Perform_Payment* as an example to show how a semi-formal process specification can be formalized into a formal one. Since all the variables in the semi-formal specification of the process have been precisely declared, what we need to do here is to focus on the formalization of the FSF. Below is a possible formal specification for this process.

process *Perform_Payment*(supplied_info1: SuppliedInformation, cost: nat0)

receipt: Receipt

ext wr *bank_accounts*

local *pos*: nat

FSF:

$exists![i: inds(\sim bank_accounts)] \mid$

$\sim bank_accounts(i).a_id = supplied_info1.s_customer_id \text{ and } pos = i \ \&\&$

$len(bank_accounts) = len(\sim bank_accounts) \text{ and}$

$bank_accounts(pos) = modify(\sim bank_accounts(pos),$

$balance - \rightarrow \sim bank_accounts(pos).balance - cost) \text{ and}$

$bank_accounts(1, pos - 1) = \sim bank_accounts(1, pos - 1) \text{ and}$

$bank_accounts(pos + 1, len(bank_accounts)) =$

$\sim bank_accounts(pos + 1, len(\sim bank_accounts))$

and

$receipt = mk_Receipt(supplied_info1.s_customer_id,$

```

    supplied_info1.s_stock_id, cost)
||
  others && receipt = nil and bank_accounts = bank_accounts
end_process;

```

The first functional scenario of the FSF of this process has a rather complex defining condition, which specifies that the sequence *bank_accounts* is updated by reducing the *cost* from the balance of the bank account on *bank_accounts* with the *a_id* same as the *supplied_info1.s_customer_id* and keeping all the rest contents of the bank accounts unchanged. It also defines the condition that the output *receipt* must satisfy.

Comparing with the corresponding semi-formal specification of the process *Perform_Payment*, its formal specification given above looks more detailed and complex, but it is more precise than the semi-formal specification, therefore helping remove ambiguities.

Chapter 6

Specification-Based Implementation



When implementing the software system under development in a specific programming language (e.g., Java, C#, C++, or Python), the specification should be utilized as an important reference, simply because it is supposed to have defined the desired requirements. How the specification can play a positive role in helping the programmer efficiently construct a good quality code becomes an interesting issue.

In this chapter, we discuss two techniques for implementation based on the hybrid specification provided, which are *module-based incremental implementation* and *iterative coding for individual operations*, respectively. These techniques can help achieve both agility and quality of implementations and may be flexibly applied in practice as long as they fit into the development situation. To ensure that all the relevant elements given in the specification are properly implemented in the code, we also suggest a way to build trace links from the specification to the code. Such trace links may also help the evolution of the code for its maintenance.

6.1 Module-Based Incremental Implementation

As described in the preceding chapters, a hybrid specification is composed of a semi-formal specification, a GUI specification, and a formal specification of critical processes. When implementing a system based on its hybrid specification, we face three tasks. The first is to decide what in the specification should be chosen for implementation. The second is to determine in what order the chosen parts should be implemented. The third is to make a decision on how a chosen part should be implemented in the code.

6.1.1 Strategy for Module Selection

Since a hybrid specification is a collection of inter-related modules and each module specifies an independent function of the entire functionality of the system, it is quite natural and efficient to choose module as the unit for implementation. That is, we first choose one module to implement in the code and then proceed to another module. Repeat this process until all the modules in the specification are implemented. The question is how we should choose the modules to implement in turn.

To achieve the agility and the quick release of the software product, we should first choose the modules that are concerned with interactions between the user and the system. This will allow the communication between the developer and the user to be more effective in implementing right functions. Then, we can proceed to implement other modules in the specification.

Specifically, we take the following priority to implement the modules:

1. The GUI module.
2. The modules defining the functions related to the functional elements specified in the GUI module.
3. Other modules.

Figure 6.1 illustrates the strategy for incremental implementation, where the label H stands for high priority, M for middle priority, and L for low priority. Firstly implementing the GUI module will facilitate the interactions and exchange of ideas between the developer and the user. Therefore, the user's feedback and involvement in the implementation process can be effectively and efficiently utilized. To help the user continue his or her involvement in making decisions and validating the implemented software, the next modules to be chosen for implementation should be those that define the functions in relation to the functional elements specified in the GUI module. This kind of module is called GUI-related module. The reason for early implementation of the GUI-related modules is that the user will be able to see early how the software behavior in runtime. The user's quick feedback about the implemented functions will further help the developer implement right functions. The other modules in the specification specify the functions necessary for fulfilling the overall goal of the system rather than directly supporting interactions with the user, which are called *internal modules*, therefore, they can be implemented later.

6.1.2 Module Implementation

When implementing a module in an object-oriented programming language, such as Java, C#, or C++, we need to choose an appropriate structure for it in the code. Considering the feature and the role of a module in specifications, it is usually effective to transform a module into a class, either static class or instance class. In this book, we only focus on the instance class implementation.

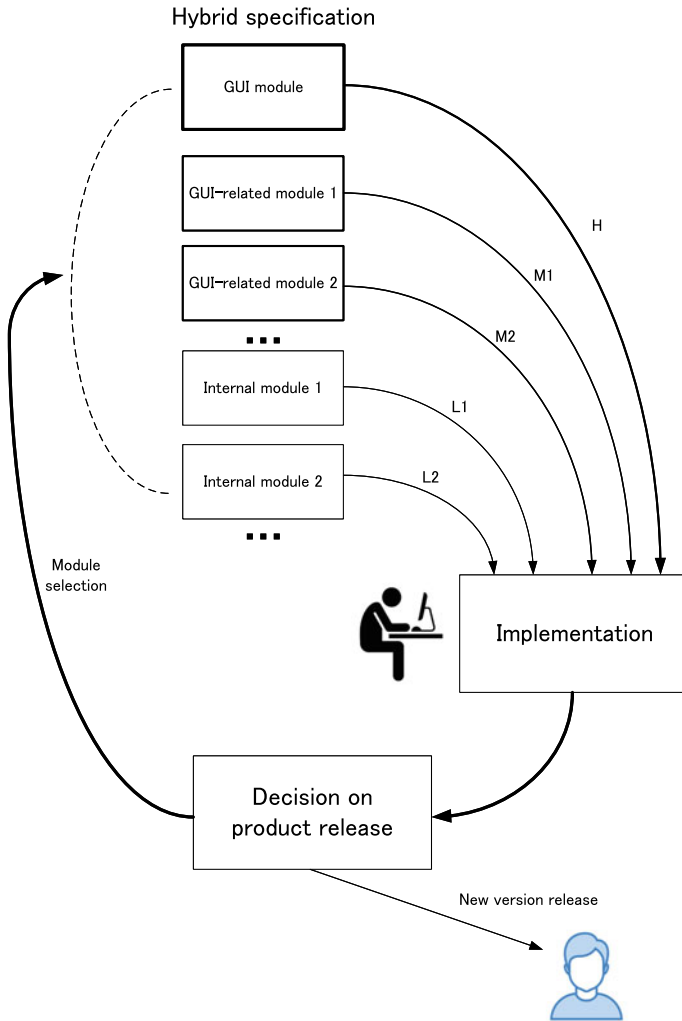


Fig. 6.1 Illustration of the incremental implementation strategy

As described in Chap. 2, a module may contain constant identifier declarations, type identifier declarations, store variable declarations, invariants, CDFD, process specifications, and function specifications. The module can be transformed into a class in a Java code by applying the “market-driven” approach. To this end, we can take the following guidelines:

- Define a class in the code to implement the module selected from the specification.
- Define a constructor of the class in the code to implement the *Init* process of the module.
- Try to define a method in the class to implement a process in the module.

- Try to define a method in the class to implement a function in the module.
- Define a method in the class to implement the CDFD of the module if any.
- Declare an instance variable to implement a store variable declared in the module.
- Declare a final variable to implement a constant identifier declared in the module.

These guidelines are determined based on the underlying principle that each element in the module and its implementation in the code play the same or similar role in realizing the functionality of the module.

In principle, each process should be implemented by a method in the class, but this may not be appropriate in some situations. In that case, the developer may have to come up with some better idea about how the process specification can be properly implemented in the code. There is no unique solution for this problem and different arrangement should be provided for different situations.

Note that we do not explicitly mention how the type identifier declarations in the module should be dealt with during the module transformation. In fact, since the declared types are used to declare variables in the module, when the corresponding variables are implemented as variable declarations in the code, we will have to determine the type for each of the variables based on the type used for the same variable in the module. In general, the type for implementation should be chosen in the way that it offers a capacity to represent all the possible values of the corresponding type in the module. For example, for a variable of a set type in the module, we can choose array, Vector, or ArrayList to declare the corresponding variable in the code.

6.2 Iterative Coding

When constructing the algorithm of a specific method in the class to implement a process specification in the module, we can take the *iterative coding strategy* to support evolutionary coding.

Specifically, when making a progress in writing code, the programmer needs to iteratively go back some lines of the code, known as a *program segment*, to review the program segment, as illustrated in Fig. 6.2. The purpose of the review has threefold. One is to confirm whether the semantics and the style of the program segment is desirable with respect to the specification and the coding standards required. The second is to check whether the program segment is robust enough or not. A program segment is robust if it does not allow any potential exceptions to cause unrecoverable errors. An effective way to ensure the robustness is to properly use the exception handling mechanism, *try* and *catch* mechanism in Java, for example. Note that a robust program segment is not necessarily correct with respect to the specification; it can only ensure that the program segment will not cause unrecoverable errors, such as runtime crash and non-termination, but may not ensure that every acceptable input will result in a desirable output. The third is to check whether the algorithm used for the program segment is efficient enough or not. The efficiency can be extremely

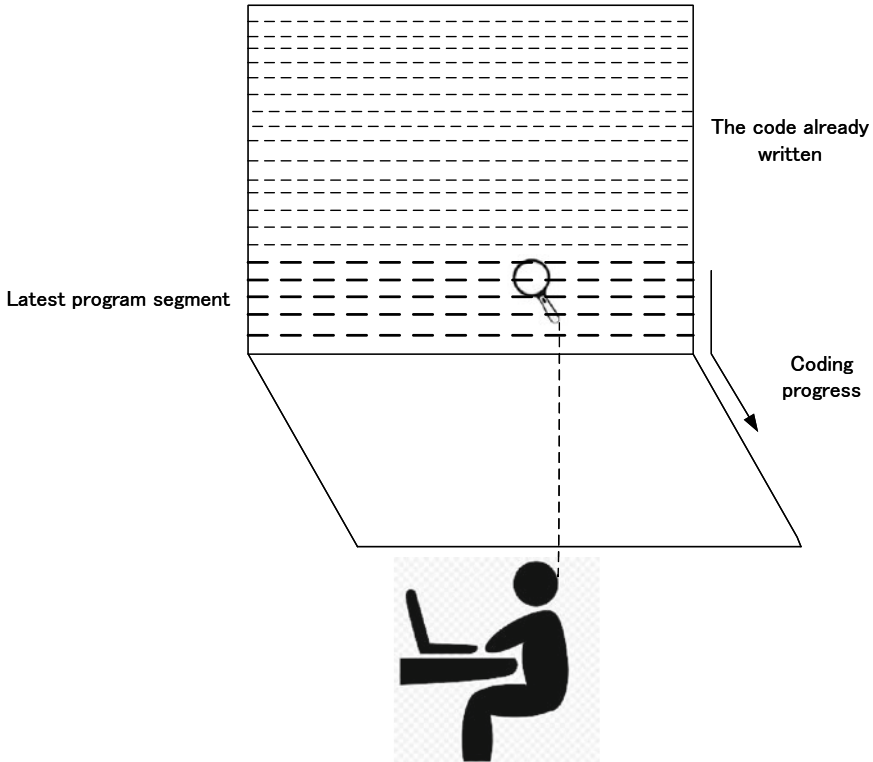


Fig. 6.2 Illustration of iterative coding

important for some kinds of systems, such as real-time systems, but may not be critical for some other systems, such as entertainment software.

It is usually quite difficult to provide a uniform technique for dealing with any of these three kinds of issues because it involves too many specific situations each of which may needs a different technique to deal with properly. In practice, the programmer’s engineering judgement plays an important role in this regard. Further research needs to be done to work out systematic and tool-supported methods for dealing with each of these three problems.

Table 6.1 Format of trace link table

Specification elements	Code elements	Comments
Module	Classes	The classes implement the module
Constant identifier	Final variable	The final variable implements the constant
Store variable	Instance variable	The instance variable implements the store variable
Invariant	Methods	The invariant is implemented in the methods
Process	Methods	The methods implement the process
Function	Methods	The methods implement the function

6.3 Building Trace Links

After a specification is implemented into code, it is important to ensure that every meaningful elements given in the specification is properly implemented in the code, simply because the specification is supposed to reflect the user’s requirements for the software under construction. One way to build such trace links is to connect the meaningful specification elements to the corresponding items (e.g., variable, type, method, class) in the code. Such trace links must be easy to build by ordinary practitioners in order to ensure its practicality. In this section, we show a tabular approach to building trace links from a specification to its corresponding code.

6.3.1 Trace Links Table

A table can be used to record the trace links from the specification to its code. Table 6.1 shows the format of the trace link table. The left column shows the elements of the specification; the middle column shows the elements of the code that should be linked to the corresponding specification element on the left; and the right column can be used as comments to explain what the link actually means. Such a trace link table should usually be built by the programmer who carries out the implementation since he or she has the responsibility for the implementation quality. By building the table, he or she will be “forced” to confirm whether all the meaningful elements in the specification are implemented properly in the code.

6.3.2 *Comprehensibility of Trace Link Table*

The trace link table must be ensured to be comprehensible enough, especially when the table becomes large. To this end, we must provide a good readability for the table. How can we do this? One possible way is to start with the specification to work out how each specification element is implemented in the code. That is, we need to figure out what elements in the code are used to implement what specification elements.

To efficiently construct such trace links, one technique is to focus on each module and its elements included. That is, we take module as the basic unit to work out the trace links, and repeat the same process until all the modules in the specification are covered. For each module, we need to build the trace links for every declared data item and specified process and function, as illustrated in Table 6.1. For each data item, it is necessary to indicate what code element is used to implement it. Each invariant is usually implemented by one or more methods, therefore, it is necessary to show what methods in the code actually contribute to the implementation of the invariant. Each process and function can be implemented by one or more methods in some classes, therefore, the corresponding connections need to be recorded in the trace link table. Of course, more details about how the code element implements the specification data item can be provided in the table, but to be practical, a simple connection would be good enough to help the developer track the relationships between specification elements and the corresponding code elements.

Chapter 7

Specification-Based Inspection



Given a code implementing a specification, how to check whether the code contains bugs without the need to execute it. There are two types of bugs. One is requirements-related bugs and the other is implementation-related bugs. A requirement-related bug is a syntactical expression that prevents the code from correctly implementing the corresponding requirements. For instance, if the requirement $a + b$ is implemented as $a * b$, $a * b$ can be regarded as a requirement-related bug. An implementation-related bug is a syntactical expression or a situation of lacking necessary syntactical expressions that violate the rules or restrictions of using the programming language. For example, $a[i]$ is part of a bug when index i exceeds the length of array a ; the other part of the bug is the lack of a statement that ensures that index i will never exceed the length of array a .

The technique for code inspection introduced in this chapter is known as *formal specification-based inspection* (FSBI) and is aimed at finding requirements-related bugs. To fulfil this goal, implementation-related bugs are also expected to be identified. The question is how to do this effectively and efficiently. The latest version of the FSBI technique was reported in [1] and has been used in many students projects.

7.1 Essential Ideas of FSBI

The FSBI focuses on how an operation specification in pre- and post-conditions is used to help inspect the implementation (code). The essential ideas of the FSBI have broad applications because the pre-post notion is widely adopted in well-known formal notations and techniques, such as VDM-SL [2], Z [3], B [4], SCR [5], and OCL [6], and their extensions or variations, such as SOFL [7] and Alloy [8]. Since Agile-SOFL process specification uses a highly comprehensible style, our discussions in this chapter and throughout this book will be based on Agile-SOFL process specifications.

Conceptually, without loss of generality, we use $S(S_{iv}, S_{ov})[S_{pre}, S_{post}]$ to denote a pre-post process specification for process S , where S_{iv} denotes the set of all the input variables whose values are not changed by the process, S_{ov} the set of all the output variables whose values are produced by the process, and S_{pre} and S_{post} are the pre- and post-conditions of S , respectively. In the case of an FSF process specification, S_{pre} is *true* and S_{post} is an FSF.

Let P be a code implementing S . To ensure that P implements S correctly, the proof obligation is

$$\forall_{\sigma \in \Sigma} \cdot S_{pre}(\sim\sigma) \Rightarrow S_{post}(\sim\sigma, P(\sim\sigma)) \quad (7.1)$$

where Σ denotes all the possible initial and final states of S . In this quantified expression, S is perceived as an abstraction of P , which generally defines a relation between initial state $\sim\sigma$ before the execution of P and final state $\sigma (= P(\sim\sigma))$ after the execution. P is treated as a function mapping the initial state $\sim\sigma$ to the final state σ . Obligation (1) requires that, for any initial state $\sim\sigma$ satisfying the pre-condition S_{pre} , the final state σ resulting from the execution of P will satisfy the post-condition S_{post} .

Formally discharging the proof obligation based upon Hoare logic [9] or Dijkstra's weakest pre-condition calculus [10] would be desirable, but it is usually complex and challenging to do so in practice. The biggest challenge lies in the process of finding appropriate loop invariants and the difficulty will be exacerbated when the code uses complex data structures. In the FSBI, we take a lightweight but rigorous approach to deal with this challenge. Instead of formally verifying the proof obligation, we will try to identify bugs that prevent the code from satisfying the obligation. To do so effectively and efficiently, we will take the "divide and conquer" principle to focus on the examination of whether the implementation of each individual functional scenario contains any bugs or not.

We use FSF style process specification as the basis for our discussions. To check whether the code properly implements the specification, we focus on checking whether each functional scenario given in the specification is properly implemented in the code. In principle, if the code correctly implements the specification, its every functional scenario must be correctly implemented by one or more code paths, and vice versa. Here, each code path (or path) defines a possible execution to produce a final state from some initial state. The Definition 7.1 defines the goal and principle of the FSBI below.

Definition 7.1 Let specification $S = f_1 \vee f_2 \vee \dots \vee f_n$ and program $P = p_1 \vee p_2 \vee \dots \vee p_n$. Then, we say that an inspection of P against S is successful iff a function M satisfying the following condition is constructed:

$$\begin{aligned} M : S &\rightarrow \text{power}(P) \\ (\forall_{f \in S} \exists_{q \in \text{power}(P)} \cdot q \neq \emptyset \wedge M(f) = q) &\wedge (\forall_{p \in P} \exists_{f \in S} \cdot p \in M(f)) \end{aligned}$$

where we treat specification S as a set of the functional scenarios involved in the FSF specification and code P as a set of the paths involved in the code definition. We also use $power(P)$ to denote the power set of P and $M(f) = q$ ($q \subseteq P$) means that the set of paths q correctly implements scenario f . Remember that each functional scenario f is a conjunction of test condition T and defining condition D , that is, $f = T \wedge D$. As described in detail in Chap. 4, T contains no output variables while D must contain the output variables of the process.

Formally, the meaning of the equation $M(f) = q$ in the above definition is given in Definition 7.2 below.

Definition 7.2 Let $f \in S$ be a scenario and $q \in power(P)$ be a set of paths. Then, $M(f) = q$ iff the following condition holds:

$$\begin{aligned} & (\forall_{\sigma, \sigma \in \Sigma} \cdot T(\sim \sigma) \wedge (\exists_{p \in q} \cdot \sigma = p(\sim \sigma)) \Rightarrow D(\sim \sigma, \sigma)) \wedge \\ & (\forall_{p \in q} \cdot (\exists_{\sigma, \sigma \in \Sigma} \cdot T(\sim \sigma) \wedge \sigma = p(\sim \sigma) \wedge D(\sim \sigma, \sigma))), \end{aligned}$$

where a path p is treated as a partial function from states to states. A set of paths q is said to correctly implement a scenario f , denoted as $M(f) = q$, iff (1) any final state σ that is produced by a single path p in q from an initial state $\sim \sigma$ satisfying the test condition T satisfies defining condition D (D may contain both initial and final state variables) and (2) any path p in q must produce a final state σ satisfying D based upon some initial state $\sim \sigma$ satisfying T . The relation between q and T and D can also be expressed as a Hoare triple [9] $\{T\}q\{D\}$ with a simple extension to the semantics: when predicate T holds and the execution of any path in q terminates, the predicate D will hold after the execution of the path.

To ensure that the FSBI method works effectively, the FSF style process specification must be a *well-formed* specification as defined in Definition 7.3 below.

Definition 7.3 Let the FSF of process specification S be $(T_1 \wedge D_1) \vee (T_2 \wedge D_2) \vee \dots \vee (T_n \wedge D_n)$. If S satisfies the condition $(\forall_{i, j \in \{1, \dots, n\}} \cdot (i \neq j \Rightarrow (T_i \wedge T_j \Leftrightarrow false))) \wedge (T_1 \vee T_2 \vee \dots \vee T_n \Leftrightarrow true)$, then S is called a well-formed process specification.

A well-formed specification ensures that for every input there must exist a test condition that is satisfied by the input and therefore the output of the process can be defined by the corresponding defining condition, provided that it is satisfiable. It can also ensure that every input satisfying a test condition must lead to an output that satisfies the defining condition given in the same functional scenario. A well-formed specification enables us to thoroughly assess the implemented code. In this case, every path within the code should align with a functional scenario outlined in the specification. Ensuring code correctness in line with the proof obligation (1), as previously outlined, mandates that every code path contributes to the realization of some functional scenario defined in the specification.

For example, a process called *Conditional_Division* is specified as follows:

```

Conditional_Division  ({x, y, ~z}, {z})
[true,
 ~z ≠ 0 ∧ x/~z ≥ y/~z && z = x/~z
 ||
 ~z ≠ 0 ∧ x/~z < y/~z && z = y/~z
 ||
 ~z = 0 && z = 0
 ],

```

where $Conditional_Division_{iv} = \{x, y, \sim z\}$ is the input variable set, $Conditional_Division_{ov} = \{z\}$ is the output variable set, the pre-condition is *true*, the post-condition in FSF is

```

~z ≠ 0 ∧ x/~z ≥ y/~z && z = x/~z ||
~z ≠ 0 ∧ x/~z < y/~z && z = y/~z ||
~z = 0 && z = 0

```

and all the variables involved are of *real* type.

In this FSF, the three test conditions are:

- (1) $\sim z \neq 0 \wedge x/\sim z \geq y/\sim z$
- (2) $\sim z \neq 0 \wedge x/\sim z < y/\sim z$
- (3) $\sim z = 0$

It is apparent that these three test conditions satisfy the condition given in Definition 7.3, therefore, the specification of the process *Conditional_Division* is a well-formed specification.

As discussed in the article [1], the principle of the FSBI method has the capability to ensure the correctness of the code with respect to its specification provided that the set of paths for each functional scenario is finite. However, this assumption is not often true of many codes in practice. That is, a functional scenario may be implemented by infinite or a large number of paths in the code. In this case, the inspection principle can be employed as a theoretical guidance for the inspector to use a set of “branch sequences” that can be generated with the strategy discussed in Sect. 7.2 to reveal bugs in the code.

A branch sequence serves as a representative of all the related code paths in the code. The expected effect is that bugs on all the relevant code paths are uncovered by inspecting their branch sequence. To effectively implement the inspection principle, establishing a systematic inspection process is crucial. This process will define a clear engineering procedure for conducting inspections and serve as a guideline for developing supporting tools.

7.2 Inspection Process

The inspection process supporting our inspection method is illustrated in Fig. 7.1. The process consists of five major steps: Obtain Functional Scenarios, Derive code Paths, Link Scenarios to Paths, Inspect Paths to Find Bugs, and Create

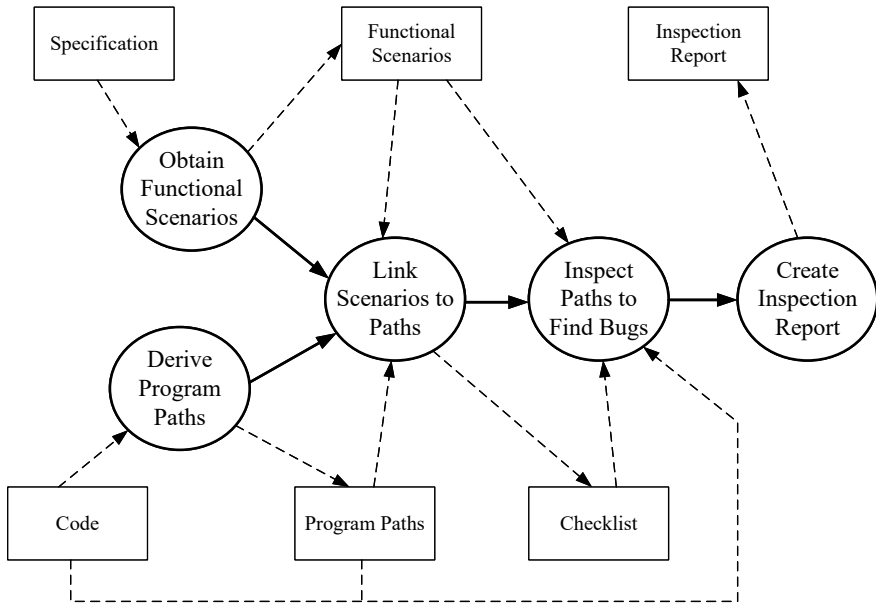


Fig. 7.1 The process of formal specification-based inspection

Inspection Report. Each step describes an operation and is represented by a circle in the figure. Each data item, such as “Specification” or “Functional Scenarios”, is represented by a box. An arrow from a box to a step means that the data item of the box is an input to the step, and an arrow from a step to a box means that the data item of the box is an output of the step. An arrow from one step to another shows the flow of taking steps.

The **Obtain Functional Scenarios** step is easy for an FSF specification; it simply collects all the functional scenarios defined in the specification and forms a functional scenario set. For a pre-post process specification, this step first converts it into an FSF and then forms the set of functional scenarios. Liu et al. has published an algorithm in [11] that can convert a pre-post process specification into an equivalent FSF. The essential idea of the algorithm includes five steps. First, convert the post-condition (e.g., S_{post}) into a disjunctive normal form (DNF) using a standard algorithm [12]. Second, each conjunction of the DNF is transformed into a conjunction of a guard condition and a defining condition where a guard condition contains no output variables of the operation. Third, all the conjunctions with the same guard condition are merged into a conjunction of the guard condition and the disjunction of their defining conditions, and all the conjunctions with the same defining condition are merged into a conjunction of the disjunction of the guard conditions and the defining condition. Fourth, form a conjunction of the pre-condition (e.g., S_{pre}) and each merged logical expression to build a functional scenario. The final step is to form an FSF by constructing the disjunction of all the functional scenarios obtained in the previous step.

The **Derive Program Paths** step takes the target code of inspection as input and derives all necessary code paths as the target of the inspection. Since it is generally difficult to statically enumerate all the paths of a code, we actually focus on the derivation of all “branch sequences”. A branch sequence is a sequence of combination of branch conditions (in true and false, respectively) and basic statements formed from the beginning to the end of the code. A basic statement is any statement different from conditional statement and iterative statement (e.g., while-do, for, do-while). They include assignment, return, break, continue, print, and function call. In general, a branch is a part of a real code path. When a code does not include any loop, all the branch sequences derived from the code will be the same as all the code paths derived. However, when a loop is included, each branch sequence may be only a part (or subpath) of a code path. For a sequential structure such as $C_1; C_2$, we produce a branch sequence denoted by $[C_1, C_2]$, which may be a part of a longer branch sequence. For an *if-then* conditional structure such as *if* (e) C , we produce two sequences $[e, C]$ and $[!e]$, respectively, where $!e$ denotes the negation of the logical expression e . For an *if-then-else* conditional structure such as *if* (e) C_1 *else* C_2 , we generate two sequences $[e, C_1]$ and $[!e, C_2]$, respectively. For a while loop such as *while* (e) C , we produce two sequences $[e, C]$ and $[!e]$, respectively, to represent the two possible choices of the execution of the loop: when the loop condition e is true and false, respectively. Variations of the three kinds of structures discussed above, such as *for* and *switch* structures in some programming languages (e.g., Java), can be dealt with using the same strategies in branch sequence derivation.

For convenience, we will use terms, such as *code path* (or *path*) and *branch sequence* interchangeably in the discussions in this chapter and the subsequent chapters.

The **Link Scenarios to Paths** step takes both the obtained scenarios and paths as input to generate an *inspection target list* (ITL) and a *checklist*. The ITL is usually generated manually and it contains a set of inspection targets. Each target, denoted by (f, q) , indicates that a specific path set q of the code needs to be inspected against the scenario f in the specification. The checklist is related to the ITL: it contains a set of questions which can be derived automatically from the inspection targets in the ITL. For example, we can derive the following questions from the target (f, q) : “Is each symbol in f correctly implemented by q ?”, “Is each atomic condition in f correctly implemented by q ?”, “Are the test condition and the defining condition of f correctly implemented by q ?”, and “Is the whole scenario f correctly implemented by q ?”. These questions will facilitate a four-level analysis in an inspection, which is to be discussed later. Note that the phrase “correctly implemented” used above, and in the rest of this chapter, means that the implementation conforms to the specification.

The **Inspect Paths to Find Bugs** step uses the checklist and the functional scenarios as guidelines to examine the related paths. Since the paths are located in the code and analyzing them usually requires the inspector to refer to the related contextual information (e.g., variable declarations), the code itself also needs to be available for this step. The analysis of paths is aimed at answering all the questions on the checklist. For example, to answer the above questions derived from the target (f, q) , the analysis can be done at four levels: (1) *the symbol level*, (2) *the atomic*

condition level, (3) *the condition level*, and (4) *the scenario level*. The four-level analysis is an application of the “divide and conquer” review principle [13]: check components first and then their combinations. We will discuss how to carry out the four-level analysis when we apply it to a specific code for the operation *Withdraw* of an ATM system in Sect. 7.3.4.

The final step, **Create Inspection Report**, produces a document to report all the bugs revealed during the inspection and to provide comments concerning any suspicious statements or conditions on the paths. The bugs detected need to be corrected and the comments serve as a reminding message for further clarification or confirmation by the individuals concerned (e.g., programmer, analyst, inspector, or all of them).

7.3 An Example

Let us use a simple example to illustrate the FSBI process. The example we use in this section is a simplified operation for selecting acceptable academic papers submitted to a conference. For the sake of convenience in discussions, we call this system *Paper Selection System* (PSS).

7.3.1 Specification of PSS

An organized conference usually call for papers from the research communities. Only regular papers can be submitted and each submitted paper needs to be reviewed by three reviewers who are usually program committee members or qualified reviewers. Each reviewer may evaluate the paper into one of the seven possible status: *Strong Accept*, *Accept*, *Weak Accept*, *Neutral*, *Weak Reject*, *Reject*, and *Strong Reject*. The reviewer also needs to recommend the paper as one of the three paper types: *Regular Paper*, *Short Paper*, and *Workshop Paper*. Here we assume that only one workshop is organized as the satellite event of the conference. There are several rules that allow a paper to be accepted for presentation at the conference and publication in its proceedings. A formal module specification is written below using Agile-SOFL. In the module *PaperSelectionSystem*, a process known as *Paper_Selection* is specified to define all the rules for selecting acceptable papers.

```

module   PaperSelectionSystem

type
    Review_Report    =   composed of
                        evaluation: Evaluation
                        recom: Paper_Type_Recom
                        end;
    Evaluation        =   { < Strong Accept >, < Accept >,
                        < Weak Accept >, < Neutral >,
                        < Weak Reject >, < Reject >,
                        < Strong Reject > }
```

```

Paper_Type_Recom    = { < Regular Paper >, < Short Paper >,
                        < Workshop Paper >, < Undecided > }
Decision           = composed of
                        result: { < Accept >, < Reject >, < Pending > }
                        paper_type: Paper_Type_Recom
                        end;

```

```

process Paper_Selection(reviews: seq of Review_Report) decision:
Decision

```

```

  FSF:

```

```

    /*Rule 1_FS*/

```

```

    len(reviews) = 3 and

```

```

    (exists[i, j: inds(reviews)] | i <> j =>

```

```

        reviews(i).evaluation inset { < Strong Accept >, < Accept > } and
        reviews(j).evaluation inset { < Strong Accept >, < Accept > }) &&
        decision = mk_Ddecision( < Accept >, < Regular Paper > )

```

```

    ||

```

```

    /*Rule 2_FS*/

```

```

    len(reviews) = 3 and

```

```

    (exists[i, j, k: inds(reviews)] | i <> j and j <> k =>

```

```

        reviews(i).evaluation = < Weak Accept > and
        reviews(j).evaluation = < Weak Accept > and
        reviews(k).evaluation = < Weak Accept > )) &&
        decision = mk_Ddecision( < Accept >, < Regular Paper > )

```

```

    ||

```

```

    /*Rule 3_FS*/

```

```

    len(reviews) = 3 and

```

```

    (exists[i: inds(reviews)] |

```

```

        reviews(i).evaluation inset { < Strong Accept >, < Accept > }) and
        (exists[j, k: inds(reviews)] | i <> j and j <> k =>
            reviews(j).evaluation = < Neutral > and
            reviews(k).evaluation = < Neutral > )) &&
        decision = mk_Ddecision( < Accept >, < Regular Paper > )

```

```

    ||

```

```

    /*Rule 4_FS*/

```

```

    len(reviews) = 3 and

```

```

    (exists[i, j, k: inds(reviews)] | i <> j and j <> k =>

```

```

        reviews(i).evaluation = < Weak Accept > and
        reviews(j).evaluation = < Weak Accept > and
        reviews(k).evaluation = < Neutral > )) &&
        decision = mk_Ddecision( < Accept >, < Short Paper > )

```

```

    ||

```

```

    /*Rule 5_FS*/

```

```

    len(reviews) = 3 and

```

```

    (exists[i, j, k: inds(reviews)] | i <> j and j <> k =>
      reviews(i).evaluation = < Weak Accept > and
      reviews(j).evaluation = < Neutral > ) and
      reviews(k).evaluation = < Neutral > )) &&
      decision = mk_Decision( < Accept >, < WorkshopPaper > )
  ||
  /*Rule 6_FS*/
  len(reviews) < 3  &&  decision = mk_Decision( < Pending >, <
  Undecided > )
  ||
  /*Rule 7_FS*/
  others && decision = mk_Decision( < Reject >, < RegularPaper > )
end_process
end_module

```

The process *Paper_Selection* takes a sequence of three review reports as input and produces a decision on the related paper (the paper does not need to be mentioned explicitly in the process specification) using seven functional scenarios. Since each functional scenario defines one rule for making the decision on the paper, we use comment to provide a name for each functional scenario, called Rule_1, Rule_2, ..., Rule_7, respectively.

7.3.2 Implementation of PSS

The module specification can be implemented in a programming language. In our case, we use Java to express the code implemented. To make the description concise, we focus on the code implementing the process *Paper_Selection* and all other relevant parts, such as type declarations, are omitted in our discussion since they are assumed to be properly represented in the code. For instance, a composite type can be represented by a class declaration. The code implementing the process *Paper_Selection* is given below.

Implementation of the process *Paper_Selection*:

```

class Decision{
    String result = "";
    String paper_type_recom = "";
    Decision(String res, String recom) {
        result = res;
        paper_type_recom = recom;
    }
}

class PaperSelectionSystem {
    public Decision Paper_Selection(Review_Report[3]: reviews) {

```

```

/*Declare necessary local variables*/

1  String Strong_Accept = "";
2  int Strong_Accept_count = 0;
3  String Accept = "";
4  int Accept_count = 0;
5  String Weak_Accept = "";
6  int Weak_Accept_count = 0;
7  String Neutral = "";
8  int Neutral_count = 0;
9  String Weak_Reject = "";
10 int Weak_Reject_count = 0;
11 String Reject = "";
12 int Reject_count = 0;
13 String Strong_Reject = "";
14 int Strong_Reject_count = 0;
15 String Invalid = "";
16 int Invalid_count = 0;
17 Decision decision = new Decision("", "");
/*Find out what three reviews' evaluations are*/
18 for(i = 0; i < reviews.length; i++){
19   switch (reviews[i]) {
20     case "Strong Accept" : {Strong_Accept = reviews[i];
21                           Strong_Accept_count++;}
22                           break;
23     case "Accept" : {Accept = reviews[i];
24                     Accept_count++;}
25                     break;
26     case "Weak Accept" : {Weak_Accept = reviews[i];
27                           Weak_Accept_count++;}
28                           break;
29     case "Neutral" : {Neutral = reviews[i];
30                       Neutral_count++;}
31                       break;
32     case "Weak Reject" : {Weak_Reject = reviews[i];
33                           Weak_Reject_count++;}
34                           break;
35     case "Reject" : {Reject = reviews[i];
36                     Reject_count++;}
37                     break;
38     case "Strong Reject" : {Strong_Reject = reviews[i];
39                             Strong_Reject_count++;}
40                             break;
41     default: {Invalid = reviews[i];
42               Invalid_count++;} }
/*Anyway, let decision be Reject first. Later,
when the required conditions are met,
the decision will be updated to Accept.
In other words, this statement implements Rule 7.*/
43 decision = new Decision("Reject", "Regular");
/*Decide the status of the paper based on the reviewers' evaluations*/
/*Implementation of Rule 1*/
44 if (Strong_Accept.equals("Strong Accept") && Strong_Accept_count > 1) {

```

```

45  decision = new Decision("Accept", "Regular" );
46  return (decision);}
47  if (Accept.equals("Accept" ) && Accept_count > 1) {
48    decision = new Decision("Accept", "Regular" );
49    return (decision);}
50  if (Strong_Accept.equals("Strong Accept" ) && Strong_Accept_count <= 1) {
51    if (Accept.equals("Accept" ) ) {
52      decision = new Decision("Accept", "Regular" );
53      return (decision);} }
54  /*Implementation of Rule 2*/
55  if (Weak_Accept.equals("Weak Accept" ) && Weak_Accept.count = 3) {
56    decision = new Decision("Accept" , "Regular" );
57    return (decision);}
58  /*Implementation of Rule 3*/
59  if (Strong_Accept.equals("Strong Accept" ) ) {
60    if (Neutral.equals("Neutral" ) && Neutral.count = 2) {
61      decision = new Decision("Accept", "Regular");
62      return (decision);} }
63  if (!Strong_Accept.equals("Strong Accept")) {
64    if (Accept.equals("Accept")) {
65      if (Neutral.equals("Neutral")) {
66        decision = new Decision("Accept", "Regular");
67        return (decision);} } }
68  /*Implementation of Rule 4*/
69  if (Weak_Accept.equals("Weak Accept") && Weak_Accept.count = 2) {
70    if (Neutral.equals("Neutral")) {
71      decision = new Decision("Accept", "Short Paper");
72      return (decision);} }
73  /*Implementation of Rule 5*/
74  if (Weak_Accept.equals("Weak Accept")) {
75    if (Neutral.equals("Neutral") && Neutral.count = 2) {
76      decision = new Decision("Accept", "Workshop Paper");
77      return (decision);} }
78  /*Implementation of Rule 6*/
79  if (Strong_Accept.count + Accept.count + Weak_Accept.count +
80      Neutral.count + Weak_Reject.count + Reject.count +
81      Strong_Reject.count < 3) {
82    decision = new Decision("Pending", "Undecided");
83    return (decision);} } /*method*/ } /*class*/

```

7.3.3 Linking Scenarios to Paths in PSS

From the algorithm of the method Paper_Selection defined in the class PaperSelectionSystem, we derive eighty-one representative code paths in accordance with the principle of deriving representative paths discussed earlier in this chapter. These paths are linked to the corresponding functional scenarios, respectively. As a result of the linking, we form the inspection target as follows:

Inspection Target List:

Rule 1_FS: 27 paths

[1~19, 20, 21, 22, 44, 45, 46],
 [1~19, 20, 21, 22, 47, 48, 49],
 [1~19, 20, 21, 22, 50, 51, 52, 53],
 [1~19, 23, 24, 25, 44, 45, 46],
 [1~19, 23, 24, 25, 47, 48, 49],
 [1~19, 23, 24, 25, 50, 51, 52, 53],
 [1~19, 26, 27, 28, 44, 45, 46],
 [1~19, 26, 27, 28, 47, 48, 49],
 [1~19, 26, 27, 28, 50, 51, 52, 53],
 [1~19, 29, 30, 31, 44, 45, 46],
 [1~19, 29, 30, 31, 47, 48, 49],
 [1~19, 29, 30, 31, 50, 51, 52, 53],
 [1~19, 32, 33, 34, 44, 45, 46],
 [1~19, 32, 33, 34, 47, 48, 49],
 [1~19, 32, 33, 34, 50, 51, 52, 53],
 [1~19, 35, 36, 37, 44, 45, 46],
 [1~19, 35, 36, 37, 47, 48, 49],
 [1~19, 35, 36, 37, 50, 51, 52, 53],
 [1~19, 38, 39, 40, 44, 45, 46],
 [1~19, 38, 39, 40, 47, 48, 49],
 [1~19, 38, 39, 40, 50, 51, 52, 53],
 [1~19, 41, 42, 44, 45, 46],
 [1~19, 41, 42, 40, 47, 48, 49],
 [1~19, 41, 42, 50, 51, 52, 53],
 [1~19, -18, 44, 45, 46],
 [1~19, -18, 40, 47, 48, 49],
 [1~19, -18, 50, 51, 52, 53]

Rule 2_FS: 9 paths

[1~19, 20, 21, 22, 54, 55, 56],
 [1~19, 23, 24, 25, 54, 55, 56],
 [1=19, 26, 27, 28, 54, 55, 56],
 [1~19, 29, 30, 31, 54, 55, 56],
 [1~19, 32, 33, 34, 54, 55, 56],
 [1~19, 35, 36, 37, 54, 55, 56],
 [1~19, 38, 39, 40, 54, 55, 56],
 [1~19, 41, 42, 54, 55, 56],
 [1~19, -18, 54, 55, 56],

Rule 3_FS: 18 paths

[1~19, 20, 21, 22, 57, 58, 59, 60],
 [1~19, 20, 21, 22, 61, 62, 63, 64, 65],
 [1~19, 23, 24, 25, 57, 58, 59, 60],
 [1~19, 23, 24, 61, 62, 63, 64, 65],
 [1=19, 26, 27, 28, 57, 58, 59, 60],

[1=19, 26, 27, 28, 61, 62, 63, 64, 65],
 [1~19, 29, 30, 31, 57, 58, 59, 60],
 [1~19, 29, 30, 31, 61, 62, 63, 64, 65],
 [1~19, 32, 33, 34, 57, 58, 59, 60],
 [1~19, 32, 33, 34, 61, 62, 63, 64, 65],
 [1~19, 35, 36, 37, 57, 58, 59, 60],
 [1~19, 35, 36, 37, 61, 62, 63, 64, 65],
 [1~19, 38, 39, 40, 57, 58, 59, 60],
 [1~19, 38, 39, 40, 61, 62, 63, 64, 65],
 [1~19, 41, 42, 57, 58, 59, 60],
 [1~19, 41, 42, 61, 62, 63, 64, 65],
 [1~19, -18, 57, 58, 59, 60],
 [1~19, -18, 61, 62, 63, 64, 65]

Rule 4_FS: 9 paths

[1~19, 20, 21, 22, 66, 67, 68, 69],
 [1~19, 23, 24, 25, 66, 67, 68, 69],
 [1=19, 26, 27, 28, 66, 67, 68, 69],
 [1~19, 29, 30, 31, 66, 67, 68, 69],
 [1~19, 32, 33, 34, 66, 67, 68, 69],
 [1~19, 35, 36, 37, 66, 67, 68, 69],
 [1~19, 38, 39, 40, 66, 67, 68, 69],
 [1~19, 41, 42, 66, 67, 68, 69],
 [1~19, -18, 66, 67, 68, 69]

Rule 5_FS: 9 paths

[1~19, 20, 21, 22, 70, 71, 72, 73],
 [1~19, 23, 24, 25, 70, 71, 72, 73],
 [1=19, 26, 27, 28, 70, 71, 72, 73],
 [1~19, 29, 30, 31, 70, 71, 72, 73],
 [1~19, 32, 33, 34, 70, 71, 72, 73],
 [1~19, 35, 36, 37, 70, 71, 72, 73],
 [1~19, 38, 39, 40, 70, 71, 72, 73],
 [1~19, 41, 42, 70, 71, 72, 73],
 [1~19, -18, 70, 71, 72, 73]

Rule 6_FS: 9 paths

[1~19, 20, 21, 22, 74, 75, 76],
 [1~19, 23, 24, 25, 74, 75, 76],
 [1=19, 26, 27, 28, 74, 75, 76],
 [1~19, 29, 30, 31, 74, 75, 76],
 [1~19, 32, 33, 34, 74, 75, 76],
 [1~19, 35, 36, 37, 74, 75, 76],
 [1~19, 38, 39, 40, 74, 75, 76],
 [1~19, 41, 42, 74, 75, 76],
 [1~19, -18, 74, 75, 76]

Rule 7_FS: 8 paths

[1~19, 20, 21, 22, 43]

[1~19, 23, 24, 25, 43]
 [1~19, 26, 27, 28, 43]
 [1~19, 29, 30, 31, 43]
 [1~19, 32, 33, 34, 43]
 [1~19, 35, 36, 37, 43]
 [1~19, 38, 39, 40, 43]
 [1~19, 41, 42, 43]

Based on the above inspection target list, a checklist containing necessary questions can be formed. Since the checklist can be too long to present properly for all the inspection targets, we concentrate only on the inspection target in relation to Rule 3_FS. We only choose one path from many of this inspection target as an example to illustrate how the checklist can be formed and can be used for inspection of the path. To make this inspection target easy to understand, we show the corresponding content of the functional scenario and the linked path as follows:

Rule 3_FS:

```
len(reviews) = 3 and
(exists[i: inds(reviews)] |
  reviews(i).evaluation inset { < StrongAccept >, < Accept > }) and
(exists[j, k: inds(reviews)] | i <> j and j <> k ==>
  reviews(j).evaluation = < Neutral > and
  reviews(k).evaluation = < Neutral > )) &&
decision = mk_Decision( < Accept >, < Regular Paper > )
```

The related path ([1~19, 20, 21, 22, 57, 58, 59, 60]):

```
1  String Strong_Accept = "";
2  int Strong_Accept_count = 0;
3  String Accept = "";
4  int Accept_count = 0;
5  String Weak_Accept = "";
6  int Weak_Accept_count = 0;
7  String Neutral = "";
8  int Neutral_count = 0;
9  String Weak_Reject = "";
10 int Weak_Reject_count = 0;
11 String Reject = "";
12 int Reject_count = 0;
13 String Strong_Reject = "";
14 int Strong_Reject_count = 0;
15 String Invalid = "";
16 int Invalid_count = 0;
17 Decision decision = new Decision("", "");
/*Find out what three reviews' evaluations are*/
18 for(i = 0; i < reviews.length; i++){
19   switch (reviews[i]) {
20     case "Strong Accept": {Strong_Accept = reviews[i];
21                           Strong_Accept_count++;}
22     break;
57   if (Strong_Accept.equals("Strong Accept")) {
58     if (Neutral.equals("Accept") && Neutral.count = 2) {
59       decision = new Decision("Accept", "Regular");
60       return (decision); } }
```

Table 7.1 Questions derived from the functional scenario

Symbol level questions
1. Is the <i>reviews</i> implemented properly?
2. Is $\langle \text{StrongAccept} \rangle$ implemented properly?
3. Is $\langle \text{Accept} \rangle$ implemented properly?
4. Is $\langle \text{Neutral} \rangle$ implemented properly?
5. Is $\langle \text{Regular Paper} \rangle$ implemented properly?
6. Is <i>decision</i> implemented properly?
Atomic condition level questions
1. Is $\text{len}(\text{reviews}) = 3$ implemented properly?
2. Is $\text{reviews}(i).\text{evaluation} \text{ inset } \{ \langle \text{StrongAccept} \rangle, \langle \text{Accept} \rangle \}$ implemented properly?
3. Is $\text{reviews}(j).\text{evaluation} = \langle \text{Neutral} \rangle$ implemented properly?
4. Is $\text{reviews}(k).\text{evaluation} = \langle \text{Neutral} \rangle$ implemented properly?
5. Is $\text{decision} = \text{mk_Decision}(\langle \text{Accept} \rangle, \langle \text{Regular Paper} \rangle)$ implemented properly?

Table 7.2 Questions derived from the functional scenario

Condition level questions
1. Is the test condition $\text{len}(\text{reviews}) = 3 \text{ and } \dots \text{reviews}(k).\text{evaluation} = \langle \text{Neutral} \rangle$ implemented properly?
2. Is the defining condition $\text{decision} = \text{mk_Decision}(\langle \text{Accept} \rangle, \langle \text{Regular Paper} \rangle)$ implemented properly?
Scenario level question
1. Is the whole functional scenario Rule 3_FS implemented properly?

Based on the functional scenario, we can form the questions given in Tables 7.1 and 7.2. These questions are used as a part of the whole checklist for the inspection.

7.3.4 Inspecting Paths

Four-level inspections can be done to check the selected branch sequence based on the checklist given above. For the symbol-level, atomic condition level, and condition level inspections, we adopt a technique called the *question and answer* technique.

Taking each question from the checklist, the inspector tries to answer it by checking and confirming the related branch sequence in the code. Trying to answer such questions helps the inspector detect bugs.

For the scenario level analysis, we use *branch-sequence* analysis. The purpose of the technique is to find bugs by reading through the branch sequence and trying to understand their potential behaviors.

7.3.5 Creating Inspection Report

During the inspection of all the related branch sequences, all the bugs found need to be recorded properly for creating a final inspection report. For each bug identified, the following pieces of information about the related code must be recorded in the report: the *file name*, *class name*, *method name*, *code line number*, *the erroneous text (statement or condition)*, and possibly a *suggested correction*.

In spite of the potential benefits of the FSBI method, applying it in practice can be challenging. All the activities involved in the FSBI procedure would take time if they are performed manually. Apparently, the solution for this challenge is to build a software tool to support the method. There is no usable tool available yet and developing one needs further research.

References

1. S. Liu, Y. Chen, F. Nagoay, and J. McDermid. Formal Specification-Based Inspection for Verification of Programs. *IEEE Transactions on Software Engineering*, 38(5):1100–1122, September 2012.
2. C. B. Jones. *Systematic Software Development Using VDM*. 2nd edition, Prentice Hall, 1990.
3. J. Woodcock and J. Davies. *Using Z: Specification, Refinement, and Proof*. Prentice Hall, 1996.
4. J. R. Abrial. *The B-Book: Assigning Programs to Meanings*. Cambridge University Press, 1996.
5. C. L. Heitmeyer. Software Cost Reduction. In J. Marciniak, editor, *Encyclopedia of Software Engineering (2nd Ed.)*, pages 1374–1380, New York, 2002. John Wiley & Sons, Inc.
6. J. Warmer and A. Kleppe. *The Object Constraint Language: Getting Your Models Ready for MDA*. Addison-Wesley, 2003.
7. S. Liu. *Formal Engineering for Industrial Software Development Using the SOFL Method*. Springer-Verlag, ISBN 3-540-20602-7, 2004.
8. D. Jackson. Alloy: A Lightweight Object Modeling Notation. *ACM Transactions on Software Engineering and Methodology*, 11(2):256–290, 2002.
9. C. A. R. Hoare and N. Wirth. An Axiomatic Definition of the Programming Language PASCAL. *Acta Informatica*, 2(4):335–355, 1973.
10. E. W. Dijkstra and C. S. Scholten. *Predicate Calculus and Program Semantics*. Springer-Verlag, 1990.

11. S. Liu, T. Hayashi, K. Takahashi, K. Kimura, T. Nakayama, and S. Nakajima. Automatic Transformation from Formal Specifications to Functional Scenario Forms for Automatic Test Case Generation. In *9th International Conference on Software Methodologies, Tools and Techniques (SoMet 2010)*, pages 383–397, Yokohama city, Japan, Sept. 29- Oct. 1 2010. IOS International Publisher.
12. J. Dick and A. Faivre. Automating the Generation and Sequencing of Test Cases from Model-based Specifications. In *Proceedings of FME'93: Industrial-Strength Formal Methods*, pages 268–284, Odense, Denmark, April 1993. Springer-Verlag, LNCS 670.
13. D. L. Parnas and D. M. Weiss. Active Design Reviews: Principles and Practices. *Journal of Systems and Software*, 7(4):259–265, 1987.

Chapter 8

Specification-Based Testing



Testing is a necessary activity to evaluate the performance of a code and to detect errors and the related bugs included in the code. An error is an undesired behavior of the code and it is usually caused by a software defect or bug. A bug is a syntactical expression that will be likely, but not definitely, to cause errors during the runtime of the code if the code path containing the bug is traversed. Once an error is found by testing, the bug causing the error needs to be identified and removed. The operation for locating bugs and determining their nature is called debugging.

Testing and debugging are two closely related activities but they are used for different purposes. The former is for confirming whether any bug exists in the code under development while the latter is for determining the location and the nature of the bug whose existence has been confirmed by testing.

Testing usually takes three steps, which are (1) generating test cases, (2) running the code under testing, and (3) analyzing the test result to determine whether any error has been found. A test case usually comprises a test data and the expected result. A test data is a set of values for the input variables of the code and an expected result is either the set of values of the output variables or a condition that should be satisfied by the output values.

There are basically two types of testing approaches that are divided based on how test cases are generated. One is known as *structural testing* or *white-box testing*. In this approach, the code is treated as a white box, and all the structural details of the code can be seen clearly by the tester. Taking advantage of the structure of the code, test data are generated to meet some coverage criteria, such as statement coverage, branch coverage, branch sequence coverage, and path coverage. Another testing approach is called *functional testing* or *black-box testing*. Taking this approach, the code is treated as a black box whose structural details are transparent to the tester (i.e., the tester is supposed not to see the structural details for testing). Test data are usually generated based on the input parameters of the code (e.g., input parameters of a function in C or a method of a class in Java) and the corresponding functional description (e.g., specification).

Specification-based testing to be discussed in this chapter is a black-box testing method that adopts formal specification as the basis for test data generation and for test result analysis. We will first clarify the purpose of this method and then discuss how it can be done properly.

8.1 Specification-Based Testing for Validation

Many publications are discussing how to use specification-based testing to verify code against specification, but actually specification-based testing makes better sense in real world software development for validation of systems (including both specification and code) than for their verification. Why?

A formal specification is supposed to accurately and completely define the requirements the user has in mind, but this can hardly be true in software development reality. The reason is that a formal specification is usually written by an analyst based on his/her understanding of the user's requirements, but since the user and the analyst usually communicate with each other in natural language, possibly using some documents written in natural language, there is no guarantee that the analyst understands every aspect of the user's requirements accurately and completely. Furthermore, the analyst may also make mistakes in using formal notation to express conditions in formal specifications. Even for a skillful analyst, making mistakes in writing formal specifications is inevitable. This situation can be exacerbated when the scale and/or complexity of the specification becomes greater. Validating specifications against the user's perception of requirements is important but has been a challenge in software engineering.

When the required functions defined in a formal specification are implemented in a code, the code is expected to fulfil the defined functions correctly. Again, this cannot be guaranteed in reality. There are several possible reasons for this. One is that the programmer may misunderstand the specification due to the complexity of the formal expressions involved, although it is precise enough to possibly avoid ambiguity. As a matter of fact, the precision of a formal specification does not guarantee no ambiguity in interpreting the requirements defined. In general, a formal specification using pre- and post-conditions defines a relation between input and output, therefore is possible to be nondeterministic in defining the output based on input. For example, a functional scenario $x > 0 \ \& \ y > x + 1$ defines when input x is greater than zero, output y must be greater than $x + 1$. This involves nondeterminism because when x is greater than 0, there are many ways to produce y that meets the defining condition $y > x + 1$. For instance, $y = x + 2$, $y = x + 5$, and $y = x + 100$ are all satisfying the defining condition. Although each symbol, such as x , y , 0, and 1, has a precise meaning in the specification, the functional scenario can produce ambiguity for a programmer in understanding the required function (how greater should y be than $x + 1$?). One may argue that the analyst who writes the defining condition does not care how greater y should be than $x + 1$, therefore, any algorithmic solution making y satisfy the defining condition should be acceptable. However, nobody in realistic software

development projects can ensure that the defining condition describes exactly what the user wants. Perhaps the user has not finished what he/she wants to express, but just because he/she lacks specific idea of how to complete what he/she wants, the specification is left nondeterministic.

Another possibility of introducing bugs into code is that the programmer may use the programming language inappropriately in the sense that some rules (e.g., a file cannot be opened twice before it is closed) associated with the use of the language or some mathematical constraints (e.g., the denominator cannot be 0 for a division) are violated in coding. In fact, such implementation-related bugs are often committed during coding.

Considering the above situations, it is necessary to check both the specification and the implemented code against the user's perception of requirements. Activities for such checks are usually known as *software validation*. Specification-based testing can be a natural and effective way for validation. Since the specification is intended to reflect the user's requirements, checking every possible functional scenario defined in the specification would help validate the specification. If such a check can also lead to the checking of some coverage of the code, such as branch sequence coverage discussed in Chap. 7, the code can also be validated. To fulfil this goal, we have developed an effective *functional scenario-based testing* (FSBT) method [1], aiming to test the implementation of a process based on its formal specification to systematically verify their consistency. From the next section in this chapter, we will discuss how this technique can be utilized to validate both the specification and its implementation.

8.2 Framework of FSBT

Figure 8.1 shows a framework of the FSBT. Since the FSBT is used to validate the program system, it must involve the judgements of the stakeholders (e.g., user, analyst, programmer). The prerequisite for the FSBT to work properly is the availability of a formal specification for the program to be tested.

As mentioned previously, the FSBT focuses on testing the code implementing a process based on its formal specification. If the specification is in the style of pre- and post-conditions, it will first need to be converted into an equivalent FSF style from which all functional scenarios defined can be easily obtained. If the process specification is given in the FSF style, then deriving the functional scenarios will be rather straightforward. Our discussions in the rest of this chapter and this book will be based the FSF style specification.

For the convenience in our discussions, we use $P(P_{iv}, P_{ov})[P_{FSF}]$ to represent the specification of process P , where P_{iv} is the set of all the input variables whose values are not changed by P , P_{ov} is the set of all the output variables whose values are produced or updated by the process, and P_{FSF} is a functional scenario form of P , which is a disjunction of all the functional scenarios defined in the process. For the sake of simplicity, we assume that P_{FSF} has incorporated the relevant state invariants declared in the *inv* section of the related module.

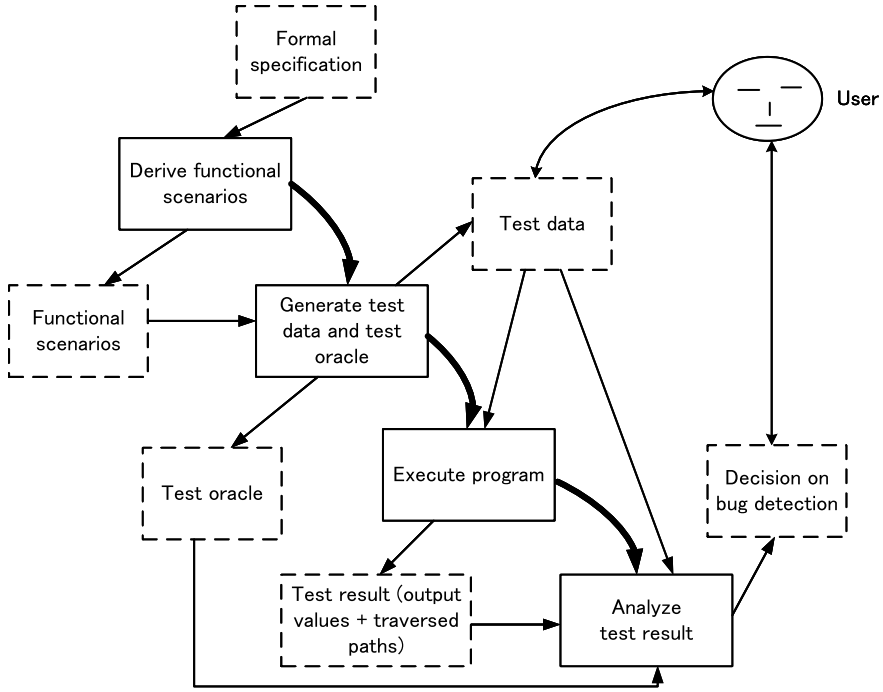


Fig. 8.1 The Framework of FSBT

Note that we use decorated variables, such as $\tilde{x}$, in a functional scenario to denote the initial value of the external variable x before the process and use x itself to denote its final value after the process. Thus, $\tilde{x}$ is treated as an input variable while x is treated as an output variable of the process; that is, $\tilde{x} \in P_{iv}$ and $x \in P_{ov}$.

Let us look at a specific process specification as an example to see how functional scenarios can be derived from it. The process *Check_Triangle* given below determines the type of the corresponding triangle based on the three input edges $d1$, $d2$, and $d3$.

```

process Check_Triangle( $d1, d2, d3$ : int)
     $t\_type$ : string
    FSF:
         $d1 = d2$  and  $d2 = d3$  &&
             $t\_type = \text{"equilateral triangle"}$  ||
         $d1 = d2$  or  $d1 = d3$  or  $d2 = d3$  &&
             $t\_type = \text{"isosceles triangle"}$  ||
         $d1 \neq d2$  and  $d1 \neq d3$  and  $d2 \neq d3$  &&
             $t\_type = \text{"other triangle"}$  ||
         $d1 \leq 0$  or  $d2 \leq 0$  or  $d3 \leq 0$  &&
             $t\_type = \text{"non-triangle"}$ 
    end\_process
  
```

From this specification we obtain the three elements of the process *Check_Triangle* as follows:

$$Check_Triangle_{iv} = \{d1, d2, d3\}$$

$$Check_Triangle_{ov} = \{t_type\}$$

$Check_Triangle_{FSF} = d1 = d2 \text{ and } \dots$ (the same as given above, omitted here for brevity).

Four functional scenarios are derived from the *FSF* of the process as follows:

- (1) $FS_1 : d1 = d2 \text{ and } d2 = d3 \ \&\& \ t_type = \text{"equilateral triangle"}$
- (2) $FS_2 : d1 = d2 \text{ or } d1 = d3 \text{ or } d2 = d3 \ \&\& \ t_type = \text{"isosceles triangle"}$
- (3) $FS_3 : d1 \neq d2 \text{ and } d1 \neq d3 \text{ and } d2 \neq d3 \ \&\& \ t_type = \text{"other triangle"}$
- (4) $FS_4 : d1 \leq 0 \text{ or } d2 \leq 0 \text{ or } d3 \leq 0 \ \&\& \ t_type = \text{"non-triangle"}$

The most important step is to generate test data and test oracle based on the derived functional scenarios. Each individual scenario serves as the basis for the generation of test data and test oracle, see the detailed discussions in Sects. 8.3 and 8.4. With the generated test data, the program can be executed. As a result, the output values and the traversed paths can be collected. The final step of testing is to analyze the test results based on the test oracle derived from the related functional scenario to determine whether bugs are found or not. From the next section, we will begin to discuss these two important issues, respectively.

8.3 Test Data Generation

Let the *FSF* of process *P* be $P_{FSF} = T_1 \ \&\& \ D_1 \ || \ T_2 \ \&\& \ D_2 \ || \ \dots \ || \ T_n \ \&\& \ D_n$. Let *C* be the code implementing process *P*. To ensure that *C* implements *P* correctly, the proof obligation to be discharged is

$$\forall_{\sigma \in \Sigma} \cdot \exists_{T \in \{T_1, T_2, \dots, T_n\}} \cdot T(\sim\sigma) \wedge D(\sim\sigma, C(\sim\sigma)) \quad (8.1)$$

where Σ denotes all the possible states of *P*. In this quantified expression, *P* is perceived as an abstraction of *C*, which generally defines a relation between the initial state $\sim\sigma$ before the execution of *C* and the final state σ ($= C(\sim\sigma)$) after the execution of *C*. *C* is treated as a function, mapping the initial state (the pre-state) $\sim\sigma$ to the final state (the post-state) σ . Condition (8.1) requires that, for any initial state $\sim\sigma$ of the code, there must exist a functional scenario *T* $\&\&$ *D* in the *FSF* whose test condition *T* is satisfied by $\sim\sigma$ and defining condition *D* is satisfied by the final state σ (together with the initial state $\sim\sigma$ because *D* is usually a condition of two states).

Theoretically, to discharge the proof obligation, formal proof must be carried out in order to be able to deal with all possible cases of initial state $\sim\sigma$, which is

generally both difficult to perform and unnecessary in realistic software development projects. The reason for the unnecessary is simple. When the code is completed, it usually contains many bugs. Therefore, trying to prove its correctness with respect to the corresponding specification must fail. Since such a proof is usually not easy to conduct, it is hardly cost-effective for realistic software projects. Another concern is even if the proof is successful, it does not necessarily ensure that the code contains no bugs due to the possibility that the process specification is not valid, as discussed in the beginning of Sect. 8.1.

A practical way to validate both the specification and the code based on the theoretical definition of the code correctness is to use specification-based testing. The reason is that it can dynamically evaluate the performance of the code and the whole testing process can be observed by the user (or relevant stakeholders). The question now is how the test data should be produced to ensure that all the important aspects of the user's requirements are tested. Below we will discuss the criteria for test data generation and the way to derive test oracle for test result analysis, respectively.

8.3.1 Test Data Generation Criteria

Given a process P and its implementation C , we need to test every functional scenario defined in the process specification and if possible, all the branch sequences of C , represented by $BS(C)$, in order to let the user confirm whether the program system behaves as expected. A test strategy reflecting this idea is given in **Criterion 1** below.

Criterion 1: Let $P_{FSF} = T_1 \ \&\& \ D_1 \ || \ T_2 \ \&\& \ D_2 \ || \ \dots \ || \ T_n \ \&\& \ D_n$ and C be the code implementing P . Let T_s denote a test set, which is a set of test data, generated from P for testing C . Then, T_s is required to satisfy all the three conditions:

- (1) $\forall_{i \in \{1, \dots, n\}} \exists_{td \in T_s} \cdot T_i(td)$
- (2) $\neg(T_1 \vee T_2 \vee \dots \vee T_n \Leftrightarrow true) \Rightarrow \exists_{td \in T_s} \cdot \neg(T_1 \vee T_2 \vee \dots \vee T_n)(td)$
- (3) $\forall_{bs \in BS(C)} \exists_{td \in T_s} \cdot traversed(bs, td)$.

We call this criterion *scenario-branch-sequence coverage* (SBSC). The criterion suggests that a generated test set T_s cover all the functional scenarios of the specification and all the branch sequences of the program. Specifically, the test set T_s is expected to meet 3 conditions: (1) for each functional scenario, there must exist a test data in T_s that satisfies the test condition of the functional scenario, (2) if the disjunction of all the test conditions does not cover the entire domain of the process (i.e., the disjunction is not equivalent to *true*), there must exist a test data in T_s that satisfies the negation of the disjunction of all the test conditions, and (3) for each branch sequence, there must exist a test data that makes the branch sequence traversed (denoted by $traversed(bs, td)$).

The distinct characteristic of this criterion compared with existing test data generation criteria reported in the literature is that it is concerned with both specification and code, requiring not only the functional scenarios defined in the specification are

tested, but the branch sequences implemented in the code are also covered. Conditions (1) and (2) together reflect the equivalence partitioning technique for generating test data [2, 3] on the basis of precisely defined test conditions of functional scenarios. Condition (3) reflects a unique coverage requirement. Meeting this criterion would lead to the exercise of all possible combinations of the branch conditions on all the program paths, therefore ensuring that all the “representative paths” of the code will be tested. Of course, some branch sequences may not be feasible to traverse. In that case, as long as the infeasible branch sequences are recognized, they should be excluded from the branch sequences required in the criterion.

Another criterion that requires the generated test set to cover more detailed parts of a test condition allows more detailed situations to be examined in testing.

Criterion 2: Let test set T_s satisfy **Criterion 1**. Let $Q_1 \vee Q_2 \vee \dots \vee Q_m$ be a disjunctive normal form (DNF) of test condition T of the given functional scenario $T \ \&\& \ D$. Then, T_s must also satisfy the condition

$$\forall_{Q \in \{Q_1, Q_2, \dots, Q_m\}} \exists_{td \in T_s} \cdot Q(td).$$

This criterion imposes further partition of the sub-domain defined by the test condition of a functional scenario involved in **Criterion 1**. Specifically, it requires that for each constituent clause of the DNF of the test condition, T_s contains a test data satisfying it. This means that at least one test data from each partitioned area of the sub-domain is required to be selected for the testing. Assuming that clause Q is the conjunction $R_1 \wedge R_2 \wedge \dots \wedge R_w$, where R_j ($j = 1, \dots, w$) is an atomic predicate or its negation, the predicate $Q(td)$ will be equivalent to $R_1(td) \wedge R_2(td) \wedge \dots \wedge R_w(td)$. Since any predicate expression can be converted into an equivalent DNF, this criterion is applicable to any test condition of any functional scenario.

Similar to existing criteria for testing case generation (e.g., branch coverage, path coverage), the two criteria above show a guideline for generating adequate test data. That is, they tell when test data generation should stop. Due to many different reasons, however, these criteria may not always be guaranteed to satisfy in practice (e.g., time constraints).

8.3.2 Example

Suppose the process *Check_Triangle* is implemented by a method of a class in Java as follows:

```
public String Check_Triangle(int d1, d2, d3) {
    String t_type = new String(" ");
    if (d1 == d2 && d2 == d3)
        {t_type = "equilateral triangle";}
    if (d1 == d2 || d1 == d3 || d2 == d3)
        {t_type = "isosceles triangle";}
    if (d1 != d2 && d1 != d3 && d2 != d3)
        {t_type = "other triangle";}
    if (d1 <= 0 || d2 <= 0 || d3 <= 0)
```

```

    {t_type = "non-triangle";}
return(t_type);
} //end of the code

```

This code contains the following branch sequences:

- (1) [(d1==d2 && d2==d3),
(d1==d2 || d1==d3 || d2==d3),
(d1!=d2 && d1!=d3 && d2!=d3),
(d1<=0 || d2<=0 || d3<=0)]
- (2) [(d1==d2 && d2==d3),
(d1==d2 || d1==d3 || d2==d3),
(d1!=d2 && d1!=d3 && d2!=d3),
~(d1<=0 || d2<=0 || d3<=0)]
- (3) [(d1==d2 && d2==d3),
(d1==d2 || d1==d3 || d2==d3),
~(d1!=d2 && d1!=d3 && d2!=d3),
(d1<=0 || d2<=0 || d3<=0)]
- (4) [(d1==d2 && d2==d3),
(d1==d2 || d1==d3 || d2==d3),
~(d1!=d2 && d1!=d3 && d2!=d3),
~(d1<=0 || d2<=0 || d3<=0)]
- (5) [(d1==d2 && d2==d3),
~(d1==d2 || d1==d3 || d2==d3),
(d1!=d2 && d1!=d3 && d2!=d3),
(d1<=0 || d2<=0 || d3<=0)]
- (6) [(d1==d2 && d2==d3),
~(d1==d2 || d1==d3 || d2==d3),
(d1!=d2 && d1!=d3 && d2!=d3),
~(d1<=0 || d2<=0 || d3<=0)]
- (7) [(d1==d2 && d2==d3),
~(d1==d2 || d1==d3 || d2==d3),
~(d1!=d2 && d1!=d3 && d2!=d3),
(d1<=0 || d2<=0 || d3<=0)]
- (8) [(d1==d2 && d2==d3),
~(d1==d2 || d1==d3 || d2==d3),
~(d1!=d2 && d1!=d3 && d2!=d3),
~(d1<=0 || d2<=0 || d3<=0)]
- (9) [~(d1==d2 && d2==d3),
(d1==d2 || d1==d3 || d2==d3),
(d1!=d2 && d1!=d3 && d2!=d3),
(d1<=0 || d2<=0 || d3<=0)]
- (10) [~(d1==d2 && d2==d3),
(d1==d2 || d1==d3 || d2==d3),
(d1!=d2 && d1!=d3 && d2!=d3),
~(d1<=0 || d2<=0 || d3<=0)]
- (11) [~(d1==d2 && d2==d3),
(d1==d2 || d1==d3 || d2==d3),
~(d1!=d2 && d1!=d3 && d2!=d3),
(d1<=0 || d2<=0 || d3<=0)]
- (12) [~(d1==d2 && d2==d3),
(d1==d2 || d1==d3 || d2==d3),
~(d1!=d2 && d1!=d3 && d2!=d3),
~(d1<=0 || d2<=0 || d3<=0)]

- (13) [$\sim(d1 == d2 \ \&\& \ d2 == d3)$,
 $\sim(d1 == d2 \ || \ d1 == d3 \ || \ d2 == d3)$,
 $(d1 != d2 \ \&\& \ d1 != d3 \ \&\& \ d2 != d3)$,
 $(d1 <= 0 \ || \ d2 <= 0 \ || \ d3 <= 0)$]
- (14) [$\sim(d1 == d2 \ \&\& \ d2 == d3)$,
 $\sim(d1 == d2 \ || \ d1 == d3 \ || \ d2 == d3)$,
 $(d1 != d2 \ \&\& \ d1 != d3 \ \&\& \ d2 != d3)$,
 $\sim(d1 <= 0 \ || \ d2 <= 0 \ || \ d3 <= 0)$]
- (15) [$\sim(d1 == d2 \ \&\& \ d2 == d3)$,
 $\sim(d1 == d2 \ || \ d1 == d3 \ || \ d2 == d3)$,
 $\sim(d1 != d2 \ \&\& \ d1 != d3 \ \&\& \ d2 != d3)$,
 $(d1 <= 0 \ || \ d2 <= 0 \ || \ d3 <= 0)$]
- (16) [$\sim(d1 == d2 \ \&\& \ d2 == d3)$,
 $\sim(d1 == d2 \ || \ d1 == d3 \ || \ d2 == d3)$,
 $\sim(d1 != d2 \ \&\& \ d1 != d3 \ \&\& \ d2 != d3)$,
 $\sim(d1 <= 0 \ || \ d2 <= 0 \ || \ d3 <= 0)$]

In these branch sequences, we use condition C (e.g., $(d1 == d2 \ \&\& \ d2 == d3)$) to denote that C evaluates to true and $\sim C$ (e.g., $\sim(d1 == d2 \ \&\& \ d2 == d3)$) to denote the case when C evaluates to false. Applying **Criterion 1**, we generate a test set T_s as shown in Table 8.1. In the table, **FS** denotes Functional Scenarios; **TD** Test Data; and **BS** Branch Sequences. Since branch sequences (1), (2), (5), (6), (7), (8), (9), (10), (15), and (16) are all infeasible to be traversed, they are excluded from the set of branch sequences required to be traversed. Apparently, the generated test set T_s covers all the four functional scenarios defined in the specification and all the feasible branch sequences (3), (4), (11), (12), (13), and (14) in the code.

8.3.3 Test Data Generation Algorithms

Since the test condition of a functional scenario is represented by a predicate expression, generating test data from it is actually equivalent to generating test data from

Table 8.1 A test set for testing the process Check_Triangle

FS	TD (d1, d2, d3)	BS
(1) FS_1	(3, 3, 3)	(4)
(2) FS_2	(5, 5, 10)	(12)
	(-5, -5, 10)	(11)
(3) FS_3	(8, 15, 20)	(14)
	(-8, 30, 50)	(13)
(4) FS_4	(-10, -10, -10)	(3)

predicate expressions. In this section, we discuss efficient algorithms for test data generation. These algorithms can be used to support automatic test data generation based on predicate expressions.

Since a predicate expression used for a test condition of a functional scenario can be an atomic predicate, a conjunction, or a disjunction, the methods and algorithms for generating test data from atomic predicates to be discussed in this section can also be applied to handle test data generation from their negations.

8.3.3.1 Generation from Atomic Predicates

An atomic predicate is a relation. As the negation of a relation can be transformed into another equivalent relation (e.g., *not* $x > y$ can be changed to $x \leq y$), the methods and algorithms for generating test data from atomic predicates to be discussed in this section can also be applied to handle test data generation from their negations.

In Agile-SOFL, variables used in relations can be declared with different types, such as numeric types, sequence types, set types, and composite types. Therefore, algorithms need to be provided to deal with test data generation from various relations involving variables of various types. We will discuss them in turn, starting from relations of numeric type and then extending to set types, sequence types, composite types, and map types, respectively.

Relations of Numeric Types

Let $Q(x_1, x_2, \dots, x_q)$ be a relation involving free variables $x_1, x_2, \dots, x_q$. The goal of test data generation from the relation is to select such a value for each variable that will satisfy the relation (i.e., make the relation evaluate to true). To fulfil this goal efficiently, the algorithms for test data generation must take the structure of the relation into account. We first try to deal with a simple case and then extend to more complicated cases where the algorithms for the simple case can be used.

Simple Relation

A simple relation contains only one variable, say x_1 , and takes the format: $x_1 \ominus E$, where $\ominus \in \{=, >, <, \geq, \leq, <>\}$ is a relational operator and E a constant expression (i.e., it can be evaluated to a specific constant of some numeric type). For instance, $10 + 8 * 9$ is such a constant expression. Table 8.2 shows the algorithms dealing with several kinds of simple relations. In these algorithms, δ denotes a non-zero natural number (e.g., 1, 2, 8).

Table 8.2 Algorithms for simple relations

No. of algorithms	$\ominus$	Algorithms of test data generation for x_1
(1)	$=$	$x_1 := E$
(2)	$\geq, <>$ or $>$	$x_1 := E + \delta$
(3)	$\leq$ or $<$	$x_1 := E - \delta$

Relations with a Single Variable in Both Operands

A little more complicated case than the previous one is that the relation contains only one variable x_1 and takes the format $E_1 \ominus E_2$, where E_1 and E_2 are both arithmetic expressions involving x_1 .

An algorithm, known as **Algorithm- R_1** , for generating test data from such a relation needs to take the following two steps:

Step 1: Transform the relation $E_1 \ominus E_2$ into an equivalent expression $x_1 \ominus E$ by means of appropriate algebraic manipulations to move variable x_1 to the left-hand side of the relational operator $\ominus$ and form the constant expression E on the right-hand side.

Step 2: Apply an appropriate test data generation algorithm for $x_1 \ominus E$ to generate test data.

For example, the two steps for generating test data from the relation $5 * x_1 > 3 * x_1 + 8$ are as follows:

Step 1: Transform $5 * x_1 > 3 * x_1 + 8$ into an equivalent relation $x_1 > 8 / 2$.

Step 2: Apply algorithm (2) in Table 8.2 to generate a test data $x_1 = 8/2 + \delta = 5$, where $\delta = 1$.

Relations with Multiple Variables in Both Operands

The most general case is that a relation $E_1 \ominus E_2$ contains multiple variables, such as $x_1, x_2, \dots, x_m$ ($m \leq q$), in both E_1 and E_2 . An algorithm, known as **Algorithm- R_m** , for generating a test data needs to take the following two steps:

Step 1: Randomly assign values from appropriate types to variables $x_2, \dots, x_m$ to transform the relation into the format $E_1 \ominus E_2$ where both E_1 and E_2 contain only a single variable, say x_1 .

Step 2: Apply **Algorithm- R_1** to generate a test data.

Let us look at the relation $5 * x_1 + 3 * x_2 - x_3 > 3 * x_1 + x_3$. Following the steps of **Algorithm- R_m** , we first randomly assign 5 to x_2 and 9 to x_3 , as an example, to convert the relation into $5 * x_1 + 3 * 5 - 9 > 3 * x_1 + 9$. Simplifying it, we obtain the relation $5 * x_1 + 6 > 3 * x_1 + 9$. Then, we apply **Algorithm- R_1** to generate a test data $x_1 = 2$.

Relations with Variables of Compound Types

It is worth noticing that the algorithms above are suitable for dealing with expressions of numeric types, but if the expression involves the operators defined on compound types, such as *set type*, we need appropriate test data generation algorithms to deal with those operators. The algorithms for generating test data from set-type relations, sequence-type relations, and composite-type relations are given below, respectively.

Algorithm-set: Let $q = 1$ and the relation $Q(x_1, x_2, \dots, x_q)$ becomes $Q(x_1)$, where x_1 is a variable denoting a single element of a set, or a set of elements. $Q(x_1)$ is a relation involving x_1 and an operator defined on set types, such as *inset* (membership), *notin* (non-membership), *card* (cardinality), *union* (union of sets), *inter* (intersection of sets), *diff* (difference between sets), *subset* (subset), *psubset* (proper subset), and *power* (power set). Then, the goal of the algorithm for generating

Table 8.3 Algorithms for test case generation from set type expressions

No. of algorithms	Relation $Q(x_1)$	Algorithms of generating test data for x_1
(1)	$x_1 \text{ inset } E_1$	$x_1 := \text{get}(E_1)$, E_1 is non-empty
(2)	$x_1 \text{ notin } E_1$	$x_1 := \text{get}(\text{Type}(x_1) \setminus E_1)$
(3)	$\text{card}(x_1) = w$	$x_1 := \{a_1, a_2, \dots, a_w\}$, where $x_1 : \text{set of } T$, and $a_k \in T$, $k = 1, 2, \dots, w$
(4)	$\text{union}(x_1, E_1) = E_2$	$x_1 := E_2 \setminus E_1$
(5)	$\text{inter}(x_1, E_1) = E_2$	$x_1 := E_2$
(6)	$\text{diff}(x_1, E_1) = E_2$	$x_1 := \text{union}(E_1, E_2)$
(7)	$\text{subset}(x_1, E_1)$	$x_1 := E_1$
(8)	$\text{psubset}(x_1, E_1)$	$x_1 := E_1 \setminus \{\text{get}(E_1)\}$
(9)	$x_1 = E_1$	$x_1 := E_1$
(10)	$x_1 <> E_1$	$x_1 := \{\text{get}(\text{Type}(x_1))\} \setminus E_1$
(11)	$\text{power}(x_1) = E_1$	$x_1 := \text{getLargest}(E_1)$

a test data from each kind of relation is to choose a set value that satisfies the relation. Table 8.3 shows the details of such algorithms in which E_1 and E_2 denote two specific sets, $\text{get}(E_1)$ represents an element obtained from set E_1 if E_1 is a non-empty set (otherwise, it becomes undefined), and w is a natural number.

In this table, x_1 in Algorithms (1) and (2) denotes an *element* of a set (e.g., E_1), while in the rest of algorithms it denotes a set whose type is *set of* T where T is the element type already defined. Algorithm (1) shows that a test data for x_1 can be generated by assigning any value from E_1 (as indicated by the assignment $x_1 := \text{get}(E_1)$). Algorithm (3) indicates that a test data for set x_1 to satisfy the condition $\text{card}(x_1) = w$ is to randomly select w elements from its element type T . Algorithm (4) states that a test data for set x_1 to satisfy the condition $\text{union}(x_1, E_1) = E_2$ (the union of x_1 and E_1 is equal to E_2) can be the difference set between E_2 and E_1 (i.e., $x_1 := E_2 \setminus E_1$). In the algorithm related to the relation $\text{power}(x_1) = E_1$, $\text{getLargest}(E_1)$ represents the largest subset of E_1 where E_1 is the power set of x_1 . The other algorithms in the table can be similarly interpreted, we therefore do not elaborate on them for brevity.

Note that the algorithms in the table are suitable for dealing with expressions involving only one input variable x_1 . However, if more than one input variable is involved, a similar measure to that of **Algorithm- R_m** can be taken.

Similarly, a group of algorithms are designed to deal with expressions involving operators on *sequences*. A sequence is a listing of ordered elements in which the duplication of elements is allowed. A sequence type contains a set of sequences; it can be declared in the format *seq of* T , where T , called element type, can be any type previously defined. These algorithms are summarized in **Algorithm-*seq*** below.

Algorithm-*seq*: Let $q = 1$ and the relation $Q(x_1, x_2, \dots, x_q)$ becomes $Q(x_1)$, where x_1 is a variable denoting a sequence of elements, $Q(x_1)$ is an expression

Table 8.4 Algorithms for test case generation from sequence type expressions

No. of algorithms	Relation $Q(x_1)$	Algorithms of generating test data for x_1
(1)	$len(x_1) = w$	$x_1 := [a_1, a_2, \dots, a_w]$ where $x_1: seq\ of\ T$, and $a_k \in T, k = 1, \dots, w$
(2)	$inds(x_1) = E_1$	$x_1 := [a_1, a_2, \dots, a_w]$ where $x_1: seq\ of\ T$, and $a_k \in T, k = 1, \dots, w$ $w = card(E_1)$
(3)	$elems(x_1) = E_2$	$x_1 := [a_1, a_2, \dots, a_w]$ where $a_k \in E_2, k = 1, \dots, w$, and $len(x_1) \geq card(E_2)$
(4)	$conc(x_1, E_3) = E_4$	$x_1 := [a_1, a_2, \dots, a_k]$ where $E_3 = [a_{k+1}, \dots, a_w]$, $1 \leq k < w$, and $E_4 = [a_1, a_2, \dots, a_w]$
(5)	$hd(x_1) = a_1$	$x_1 := [a_1, a_2, \dots, a_w]$ where $a_1, a_k \in T, k = 2, \dots, w$
(6)	$tl(x_1) = E_4$	$x_1 := [a_1, a_2, \dots, a_w]$ where $conc([a_1], E_4) = [a_1, a_2, \dots, a_w]$

involving x_1 and an operator defined on sequence types, such as *len* (length of sequence), *inds* (indexes set of sequence), *elems* (element set of sequence), and *conc* (concatenation of sequences). Then, the algorithm for generating a test data from each kind of relation aims to find a sequence value that satisfies the relation. Table 8.4 shows a set of test data generation algorithms for various relations in which E_1 denotes a set of natural numbers, E_2 a set value of appropriate type, E_3 and E_4 two specific sequences respectively, and w a natural number.

In this table, algorithm (1) shows that a test data can be generated for sequence x_1 to satisfy the relation $len(x_1) = w$ (the length of x_1 is w) by assigning x_1 a sequence of w elements taken randomly from the element type T . Algorithm (3) describes that a test data can be generated for sequence x_1 to meet the condition $elems(x_1) = E_2$ (the element set of sequence x_1 is equal to set E_2) by assigning x_1 a sequence of elements covering all of the elements of E_2 . Algorithm (5) states that a test data can be generated for sequence x_1 from the relation $hd(x_1) = a_1$ by assigning x_1 a sequence of elements whose first element is a_1 and the rest are selected randomly from their type. Since the other algorithms in the table can be interpreted similarly, their discussions are omitted.

Another important and frequently used data type is *composite type*. A composite type defines a set of composite objects, each being composed of several fields. Each field is represented by a variable that is declared with a certain type. The general

Table 8.5 Algorithms for test data generation from composite type expressions

No. of algorithms	Relation $Q(x_1)$	Algorithms of generating test data for x_1 whose field variables are $a_1, a_2, \dots, a_m$
(1)	$x_1 = mk_A(v_1, v_2, \dots, v_m)$	$x_1.a_j := v_j, j = 1, \dots, m$
(2)	$x_1.a_j = v_j$	$x_1.a_j := v_j, j = 1, \dots, m$
(3)	$x_1 = modify(y, a_1 \rightarrow v_1, a_2 \rightarrow v_2)$	$x_1 := y; x_1.a_1 := v_1; x_1.a_2 := v_2$
(3)	$x_1 = y$	$x_1 := y$
(4)	$x_1 <> y$	$x_1 := y; x_1.a_1 := y.a_1 + \delta, \delta > 0$

format of a composite type declaration is illustrated using the type A declaration below.

$$\begin{aligned}
 A &= \text{composed of} \\
 &\quad a_1: Type(a_1) \\
 &\quad a_2: Type(a_2) \\
 &\quad \dots \\
 &\quad a_m: Type(a_m) \\
 &\quad \text{end}
 \end{aligned}$$

This specifies that each composite object of type A has m fields: $a_1, a_2, \dots, a_m$. A group of algorithms for generating test data from a relation involving operators on composite objects are summarized in **Algorithm-comp**.

Algorithm-comp: Let $q = 1$ and the relation $Q(x_1, x_2, \dots, x_q)$ becomes $Q(x_1)$, where x_1 denotes a composite object of type A , $Q(x_1)$ is a relation involving x_1 and an operator defined on composite types, such as *make function* (e.g., $mk_A(v_1, v_2, \dots, v_m)$), *field select* (e.g., $x_1.a_1$), *modify* (e.g., $modify(mk_A(v_1, v_2, \dots, v_m), a_1 \rightarrow newValue)$), *equality* ($=$), and *inequality* ($<>$). Then, a collection of algorithms for generating a test data from each kind of relation is shown in Table 8.5.

Algorithm (1) in Table 8.5 suggests that a composite object x_1 of type A be created to satisfy the equation $x_1 = mk_A(v_1, v_2, \dots, v_m)$ by assigning the designated values v_j ($j = 1, \dots, m$) to the corresponding field variables $x_1.a_j$. Algorithm (3) states that a composite object x_1 can be generated to meet the condition $x_1 = modify(y, a_1 \rightarrow v_1, a_2 \rightarrow v_2)$ by executing the three assignment statements sequentially: $x_1 := y$ (assigning composite object y to x_1), $x_1.a_1 := v_1$ (assigning value v_1 to the field variable a_1 of x_1), and $x_1.a_2 := v_2$. The other algorithms can be easily interpreted similarly, and their explanations are therefore omitted.

Since a test condition of a functional scenario can be a disjunction of several conjunctions of atomic predicates, the algorithms on atomic predicates discussed so far are not sufficient. We must discuss the issues in relation to test data generation from a conjunction and disjunction, respectively. Below in Sect. 8.3.3.2 we focus on conjunctions and in Sect. 8.3.3.3 we discuss for disjunctions.

8.3.3.2 Generation from Conjunctions

As indicated in **Criterion 2** of Sect. 8.3.1, the test condition T of a functional scenario $T \ \&\& \ D$ can be transformed into an equivalent disjunctive normal form, say $Q_1 \vee Q_2 \vee \dots \vee Q_m$ ($m \geq 1$), where each Q_j ($1 \leq j \leq m$) is a conjunction of atomic predicates, say $R_j^1 \wedge R_j^2 \wedge \dots \wedge R_j^w$ ($w \geq 1$). The issue of our interest here is how to efficiently generate a test data from such a conjunction $R_j^1 \wedge R_j^2 \wedge \dots \wedge R_j^w$. This issue is equivalent to finding a test data satisfying the conjunction.

The essential idea of the algorithm presented below for generating test data from the conjunction is first to form an *ordered partition* of the atomic predicate set $\{R_j^1, R_j^2, \dots, R_j^w\}$ according to *predicate dependency*, and then properly apply the algorithms mentioned previously to generate a test data satisfying all the atomic predicates in the conjunction if it is satisfiable. To describe the idea clearly, we first need to clarify the concepts of predicate dependency and ordered partition.

Notation:

- $Var(E)$ denotes the set of free variables occurring in predicate E .
- $[1..n]$ denotes the set of integers $\{1, 2, \dots, n\}$.
- $\{R_j^1, R_j^2, \dots, R_j^w\}$ denotes all the atomic predicates in the conjunction $R_j^1 \wedge R_j^2 \wedge \dots \wedge R_j^w$.

Definition 8.1 (*predicate dependency*) Let E_1 and E_2 be two predicate expressions. If $Var(E_1) \subset Var(E_2)$, E_2 is said to be dependent on E_1 , which is represented as $E_1 \sqsubset E_2$.

For example, predicate $x * y > 20$ is dependent on $x > 0$ since the set of free variables of $x > 0$ is $\{x\}$, which is a subset of the set of free variables $\{x, y\}$ of the predicate $x * y > 20$. That is, $x > 0 \sqsubset x * y > 20$.

Definition 8.2 (*ordered partition*) Let $\{H_1, H_2, \dots, H_u\}$ be a set of predicate sets. If it satisfies the following two conditions

$$(1) \forall_{i \in [1..u-1]} \forall_{E_1 \in H_i} \exists_{E_2 \in H_{i+1}} \cdot E_1 \sqsubset E_2$$

$$(2) \forall_{i \in [1..u]} \forall_{E_1, E_2 \in H_i} \cdot \neg(E_1 \sqsubset E_2),$$

we say $\{H_1, H_2, \dots, H_u\}$ is an ordered partition on $\sqsubset$.

For instance, $\{x > 0, y > 1\}, \{x * y > 20\}, \{x + y * z > 1, x + y * z < 100\}$ is an ordered partition on $\sqsubset$, whereas $\{x > 0, x + y > 1\}, \{x * y > 10\}, \{x + y * z > 1, x + y * z < 100\}$ is not because there exists a predicate $x + y > 1$ in the first predicate set on which the predicate $x * y > 10$ in the second predicate set is not dependent (violating condition (1)). It is also because that the two predicates in the first predicate set satisfies the dependency relation, i.e., $x > 0 \sqsubset x + y > 1$, which violates condition (2) in the definition.

Definition 8.3 (*predicate set satisfaction*) Let H be a predicate set and t be a test data. If t satisfies every predicate in H , we say t satisfies H .

Let $H = \{x > 0, x * y < 10\}$ and a test data $t = \{(x, 2), (y, 3)\}$. Then, apparently t satisfies H by definition because t satisfies both $x > 0$ and $x * y < 10$.

The rationale for forming the ordered partition based on the predicate dependency in the algorithm is that producing values for the variables in a predicate (e.g., $x + y * z > 1$) depends on the values already generated from the depended predicates (e.g., $x > 0$ and $y > 1$). For instance, suppose we generate 5 for x and 8 for y , which satisfy $x > 0$ and $y > 1$, respectively. Then we can substitute 5 for x and 8 for y in the predicate $x + y * z > 1$ to obtain another predicate $5 + 8 * z > 1$. Simplifying it leads to $z > -4/8$, which can be further reduced to $z > -1/2$.

The algorithm for generating a test data satisfying the conjunction of $R_j^1 \wedge R_j^2 \wedge \dots \wedge R_j^w$ is given in **Algorithm-conj** below and its main idea is summarized as follows. Firstly, construct $\{H_1, H_2, \dots, H_u\}$ as an ordered partition on $\square$ for the set $\{R_j^1, R_j^2, \dots, R_j^w\}$ ($1 \leq u \leq w$). Thus, $\{H_1, H_2, \dots, H_u\}$ becomes an alternative expression of $\{R_j^1, R_j^2, \dots, R_j^w\}$ but more suitable for test data generation. Secondly, generate a test data that satisfies all the predicates in H_i ($1 \leq i < u$) and then utilize the resultant test data to generate a more complete test data for H_{i+1} (i.e., produce values for more variables involved). Repeat this process until H_u is reached and a qualified test data is produced. However, if the generation fails for H_i , it will go one step back to retry generating another test data for H_{i-1} (provided that $i - 1 \geq 1$) and then repeat the same process. But if the number of failures to generate a qualified test data satisfying all $H_1, H_2, \dots, H_u$ reaches a pre-defined one (e.g., 100), a failure message will be issued as the result of the algorithm.

Algorithm-conj:

```

No. 1 Construct an ordered partition  $\{H_1, H_2, \dots, H_u\}$  for the set  $\{R_j^1, R_j^2, \dots, R_j^w\}$  ( $1 \leq u \leq w$ );
No. 2  $t_0 := \{\}$ ;  $i := 1$ ;  $flag := 0$ ; /*initializing variable  $t_0$  representing the initial test data*/
No. 3 while ( $i \leq u$  &&  $flag \leq NoOfFailure$ ) {
     $A := ObtainInstantiatedPredicates(H_i, t_{i-1})$ ; /* $A$  is an array of predicates*/
     $t_i := GenerateTestData(A)$ ; /* $t_i$  is a new test data generated based on the predicates in  $A$ */
    if ( $t_i = \{\}$ )
        {if ( $i \leq 1$ )  $\{i := 1\}$ 
          else  $\{i := i - 1\}$ ;
           $flag := flag + 1$ ; }
    else  $\{i := i + 1\}$ 
    }
No.4 if ( $flag > NoOfFailure$ )
    {Display a test data generation failure message}
    else {Display a test data generation success message and
           $t_i$  is the generated test data}
No. 5 End.
```

In this algorithm, the *function* (or *method* in Java) *ObtainInstantiatedPredicates*(H_i, t_{i-1}) obtains an array A whose elements are the atomic predicates resulting from substituting the value of every variable in test data t_{i-1} for the same variable in the atomic predicates in H_i . For instance, suppose $H_i = \{x * y + z > 1, x + y * z < 100\}$, containing two predicates and $H_{i-1} = \{x + y > 10\}$, and the test data for H_{i-1} is $t_{i-1} = \{(x, 8), (y, 9)\}$. Then, we get A from *ObtainInstantiatedPredicates*(H_i, t_{i-1}) as follows:

$A = \{8 * 9 + z > 1, 8 + 9 * z < 100\}$.

To generate a test data for H_i based on A , we need to apply the function *GenerateTestData*(A). The test data generated from this function is actually a more complete one than t_{i-1} that satisfies all the atomic predicates in A . Assuming that array A contains n atomic predicates, we give an algorithm used to implement the function *GenerateTestData*(A) in Algorithm-gta below.

Algorithm-gta:

```

satisfyingConjunction := false;
for (int i := 0; i < A.length(); i++){
   $t_a$  :=
  GenerateTestDatafromFirstPredicate(A[0]);
  k := 0;
  while (k < A.length() && Satisfy(A(k),  $t_a$ )){
    k := k + 1;
  }
  if (k < A.length())
    {A := Rotate(A); }
  else {satisfyingConjunction := true;
        break; }
}
if (satisfyingConjunction == false) {
   $t_a$  = {};
}
}
return  $t_a$ ;

```

This algorithm first generates a test data satisfying the first atomic predicate of A (i.e., $A[0]$) and then tests whether it satisfies all the other atomic predicates in A . If yes, a successful test data is generated; otherwise, repeat the same process for the other atomic predicates in A until all the atomic predicates of A are exhausted. In this algorithm, the function *GenerateTestDatafromFirstPredicate*($A[0]$) produces a test data as the result that satisfies the first atomic predicate in A . *Satisfy*($A(k), t_a$) yields true if test data t_a satisfies $A(k)$; otherwise it yields false. *Rotate*(A) yields a permutation of A by moving the first element of A to the last position and all the other elements are shifted to one position left in A .

8.3.3.3 Generation from Disjunction

Compared to test data generation from a conjunction, algorithm for test data generation from the disjunction $Q_1 \vee Q_2 \vee \dots \vee Q_m$ ($m \geq 1$) is rather simple. There are at least two kinds of solutions. One is to take a non-empty test set generated from any of the constituent predicate Q_j ($1 \leq j \leq m$) as the resultant test set for the disjunction, but this is unlikely to exercise all the constituent predicates. A better choice is to take the union of all the test sets generated from all the constituent predicates as the test set for the disjunction. Since the algorithm is trivial, we do not discuss its details here.

8.3.4 “Vibration” Testing Method—V-Method

The *scenario-branch-sequence coverage* described in **Criterion 1** indicates the adequacy for test set generation in the specification-based testing approach. It requires that not only all the functional scenarios in the specification be covered, but also all the feasible branch sequences of the program be traversed. Our discussions on algorithms for test data generation so far have focused on achieving a full coverage of functional scenarios in the specification, but those algorithms do not deal with the problem of how to gain a full coverage of the feasible branch sequences in the corresponding code. As is well-known in testing, bugs on a branch sequence is unlikely to be revealed if the sequence is never executed.

To address this concern, more test data need to be produced if the test set already generated does not ensure the expected coverage of branch sequences. The challenge is what and how test data should be generated to allow us to deal with this problem. Since this challenge is a common problem to all existing black-box testing approaches, it is important to address this issue.

A new method called “Vibration” testing method, or V-Method for short, can help address this problem. The V-Method was first proposed by the author in his conference paper [4] and developed to a more mature level in a more recent publication [1]. In this section, we present the essential idea of the V-Method and point out the directions of future research along this line.

8.3.4.1 Principle of V-Method

The V-Method is a heuristic method of local search for new test data, depending on the notion of “distance” based on a relation (atomic predicate). Let $E_1(x_1, x_2, \dots, x_n) \ominus E_2(x_1, x_2, \dots, x_n)$ denote that expressions E_1 and E_2 have relation $\ominus$ (e.g., $<$, $>$). Applying the V-Method to this relation, we first assign some values to $x_1, x_2, \dots, x_n$ such that the relation $E_1(x_1, x_2, \dots, x_n) \ominus E_2(x_1, x_2, \dots, x_n)$ holds with an initial distance between E_1 and E_2 , and then repeatedly create further values for the variables such that the relation still holds but the distance between E_1 and E_2 “vibrates”

(changes repeatedly according to the distance) between the initial distance and the “maximum” or “minimum” distance, until a decision for terminating the testing is made. The decision is made based on one of the two conditions. One is that all the related branch sequences have been covered, and the other is that a required number (usually pre-defined) of test data are generated. The former indicates that the test data have exercised all the related branch sequences, while the latter implies that covering all the related branch sequences is difficult (possibly due to the existence of bugs or infeasibility of some branch sequences).

The rationale for the V-Method comes from the theory that a functional scenario is usually refined into a program fragment that may implement the defining condition of the scenario conditionally. This point can be understood easily with an example. Assume $x < y + 800$ && $z = x + y$ is a single functional scenario in which $x < y + 800$ is the test condition involving only input variables x and y and $z = x + y$ is the defining condition involving output variable z . This scenario can, for example, be refined into the following conditional statement in the program:

```

1 if (x < y + 100) {
2   i = i + 1;
3   z = x + y;
4 } else if (x >= y + 100 && x < y + 500) {
5   i = i + 2;
6   z = x + y;
7 } else if (x >= y + 500 && x < y + 800){
8   i = i + 3;
9   z = x + y;
10  } .

```

In any branch of the nested conditional statement, the defining condition is correctly implemented, but meanwhile for some reason the code also needs to update another variable i , which can be either a local or “global” variable, differently in different branches.

To make each decision true and false at least once, respectively, different test data changing within the range $x < y + 800$ need to be used. For example, let $x = 100$ and $y = -100$ as the first test data. The distance between the two expressions x and $y + 800$ is 600. This test data makes the first decision $x < y + 100$ false and the second decision $x >= y + 100$ && $x < y + 500$ true, thus covering the path [1, 4, 5, 6]. Next, we enlarge the distance between x and $y + 800$ to 750, and generate the new test data $x = 50$ and $y = 0$. It makes the first decision $x < y + 100$ true, thus covering the path [1, 2, 3]. Another change is to decrease the distance 750 to 250, and generate the test data $x = 350$ and $y = -200$. This makes the first decision $x < y + 100$ false, the second decision $x >= y + 100$ && $x < y + 500$ false, and the third decision $x >= y + 500$ && $x < y + 800$ true, thus covering the path [1, 4, 7, 8, 9, 10]. Using the above test, all the feasible branch sequences in the program fragment implementing the functional scenario are traversed. Table 8.6 shows a summary of the most relevant information of the test for readability.

Table 8.6 Summary of the example for the V-Step

Test data	Distance	Distance up	Distance down	Decision evaluation
$x = 100,$ $y = -100$	600			$x < y + 100$ (F) $x \geq y + 100$ && $x < y + 500$ (T)
$x = 50,$ $y = 0$	750	150		$x < y + 100$ (T)
$x = 350,$ $y = -200$	250		500	$x < y + 100$ (F) $x \geq y + 100$ && $x < y + 500$ (F) $x \geq y + 500$ && $x < y + 800$ (T)

8.3.4.2 Critical Issues on V-Method

There are several issues critical to the application and the effectiveness of the V-Method. The first is how the distance between the two expressions E_1 and E_2 can be defined. This might be straightforward if both E_1 and E_2 are numeric expressions, but it would become more difficult if E_1 and E_2 involve operators defined on compound types, such as set types, sequence types, and map types. For example, consider the expression $union(a_1, a_2) = elems(s)$ as an example, where a_1 and a_2 are two sets and s a sequence. This relation states that the union of a_1 and a_2 is equal to the element set of sequence s . When applying the V-Method, we need to determine the distance between the two sets $union(a_1, a_2)$ and $elems(s)$, no sensible definition of the distance between two sets is available, although some suggestions in the paper [1] have been made. This is also true for other compound objects, such as sequences and maps.

The second critical issue is how to determine the way to “vibrate” the distance in the V-Method. In other words, what is the principle for systematically changing the distance between two expressions E_1 and E_2 for test data generation? This is still an open problem in the V-Method.

The third issue is concerned with the way to applying the principle of the V-Method to test data generation from a conjunction, such as $x > 10 \wedge x < 90$ and $x * y > z \wedge x + y < z$. The characteristic of the conjunction $x > 10 \wedge x < 90$ is that only one free variable x is used in each relation of the two involved. In this case, we first obtain a value interval in which x can freely take a value, which can be $[11, 89]$, assuming that x is an integer. Then, we can define a distance between the value chosen (test data) and the minimum (or maximum) value 11 (or 89). Let this distance “vibrate” within the range between 11 and 89, more test data can be generated. However, this technique does not seem to be directly applicable to the conjunction $x * y > z \wedge x + y < z$ since no value interval can be easily determined. In this case, we can first convert the conjunction $x * y > z \wedge x + y < z$ to a conjunction with the same style

as $x > 10 \wedge x < 90$ in which only one variable (i.e., x) is involved. The way to do it is to assign values chosen randomly from the corresponding types to variables y and z . For instance, by assigning 10 to y and 500 to z , the conjunction is changed to $x * 10 > 500 \wedge x + 10 < 500$. From this conjunction we can derive a value interval [51, 489] and then apply the same technique discussed previously to generate test data based on this interval.

The fourth issue is when the “vibration” should stop for the test data generation from a functional scenario. A reasonable answer is that the vibration testing should stop when all the branch sequences implementing the functional scenario are covered. However, determining whether all the branch sequences are all covered for the functional scenario is quite difficult in practice because how many branch sequences are used to implement the functional scenario is unknown in advance.

Addressing all the above issues using effective techniques is important for developing the V-Method and more studies need to be carried out in this area.

8.4 Test Oracle Derivation

Test oracle is a mechanism that can help judge whether a test has uncovered bugs or not. It can be a person, organization, an expected value, or a condition. In the FSBT testing approach, we can derive a test oracle from the functional scenario of interest. Let T & D be a functional scenario for test data generation, T_s be the test set generated from T . Then, if the condition

$$\forall_{t_a \in T_s} \cdot T(t_a) \wedge \neg D(t_a, C(t_a))$$

holds, it will indicate that test data t_a has confirmed the existence of bugs in the code C implementing the functionality defined in the specification involving the functional scenario T & D . In other words, this condition can be used as a test oracle for test result analysis. Apparently, the test oracle shows an instance violating the proof obligation for defining the correctness of the code. It states that if there exists a test data t_a satisfying the test condition T but the final state $C(t_a)$ as a result of executing the code C with t_a does not satisfy the defining condition D , it implies the existence of bugs in the code.

8.5 Practical Issues About FSBT

In this section, we discuss several important issues to be possibly encountered when the FSBT is applied in practice. These issues include specification language, feasibility of automation, usage of test data and test oracle, and software tools to support automatic FSBT. We also point out the ultimate goal of the FSBT in the end for future development of the FSBT approach.

8.5.1 *Specification Language*

The discussions about the FSBT above are mainly based on formal specifications written in a formal notation or language, but in practice specifications may not be written formally; they can be informal or semi-formal as well. The question is whether the FSBT techniques can be applied to informal or semi-formal specifications.

The answer is positive. The only difference is that when the specification is formal, the FSBT techniques can be automatically applied with the support of software tools, but when the specification is informal or semi-formal, the FSBT would have to be applied manually since the derivation of test conditions and test oracle would have to involve dealing with ambiguous descriptions in natural language. Although software tools can also be constructed to support the application of the FSBT in this case, the tool would be limited only to managerial support, it can hardly be used for automatic generation of test data and test oracle.

8.5.2 *Feasibility of Automation*

The algorithms for automatic test data generation introduced before are suitable for liner expressions in predicates, but if an atomic predicate involves non-linear expressions, such as $x^5 + y^{10} \times z^2 > w^2 + z^3$, automatic generation of test data from it would be rather difficult. The hardest thing is how to form a general algorithm that can deal with all kinds of such expressions with a practically acceptable efficiency. A similar difficulty also exists in dealing with some set expressions, such as $x \text{ not in } E$ (i.e., $x \notin E$) where E is a very large or infinite set.

8.5.3 *Usage of Test Data and Test Oracle*

Since the specification language and the programming language adopt different syntax, the test data generated from the specification can hardly be directly used by the code. To ensure that the test data and test oracle generated can be used properly and effectively to the code under testing, the code must preserve the signature of its specification (i.e., all input, output, external variables of both the specification and the code must be the same, although their types can be in different format). However, this seems difficult to guarantee in practice, although theoretically the consistency can be achieved through configuration management. One possible way to deal with this problem is to manually or automatically assign properly the generated test data to the corresponding input variables in the code. That is, the tester can write a *test driver*, a piece of code, in the same programming language in which all the necessary test data are assigned to the actual parameters and then use them to call the code (e.g., a method in Java) under testing.

When a single test is completed using a single test data, the test result must be analyzed to determine whether the test has uncovered any errors. To this end, the test oracle derived from the corresponding functional scenario in the specification must be evaluated. This will require that the output variables involved in the test oracle be first assigned the corresponding output values of the code, which are treated as the test result. Since the output values of the code are given in a syntax that can be different from the corresponding output variables in the test oracle, an appropriate assignment from the code output values to the specification output variables must be ensured before the test oracle can be evaluated for a decision.

8.5.4 *Software Tools for FSBT*

Building a software tool to support the automatic application of the FSBT needs to handle several important tasks. Figure 8.2 shows a general layered architecture of the tool, indicating all major functions to be supplied in the tool.

The essential functions include *specification analysis* and *code analysis*. The specification analysis is responsible for extracting all functional scenarios and all necessary elements from each functional scenario, such as test conditions and defining conditions. The code analysis is intended to insert probes for recording traversed code paths by tests. Each probe is usually a short code that records the executed statement of the code in a file and it is usually inserted just after the statement in the code under testing. The code analysis is also responsible for identifying all the input variables and output variables of the code that will be used for generating a testing code and for collecting the output values as a test result.

The next outer layer functions, which depend on the essential functions, include *test data generation*, *test oracle generation*, *test driver generation*, and *execution of code*. All the functions are expected to be fully automated. While the functions of test data generation and test oracle generation can be rather easily realized by taking advantage of the test data and test oracle generation algorithms discussed in previous sections, the function of test driver generation would be relatively more difficult to support due to the potential problems of signature inconsistency and syntactical gap between the specification and the code. As far as the function of execution of code is concerned, the tool would need to be able to automatically convert the given code for testing into a code with appropriate stubs if the code includes calling of other subroutines (e.g., methods in Java, functions in C). A stub acts as a small piece of code that replaces a calling statement of a subroutine and it is intended to simulate the behavior of the subroutine for testing purpose. This function is also expected to automatically supply the traversed program paths that will be utilized to make decisions of whether further test data need to be generated to meet the branch sequence coverage.

The second outer layer functions are *test results collection*, *evaluable test oracle generation*, *test oracle evaluation for test result analysis*, and *error report*. The function of test results collection is aimed at collecting all the output values of the

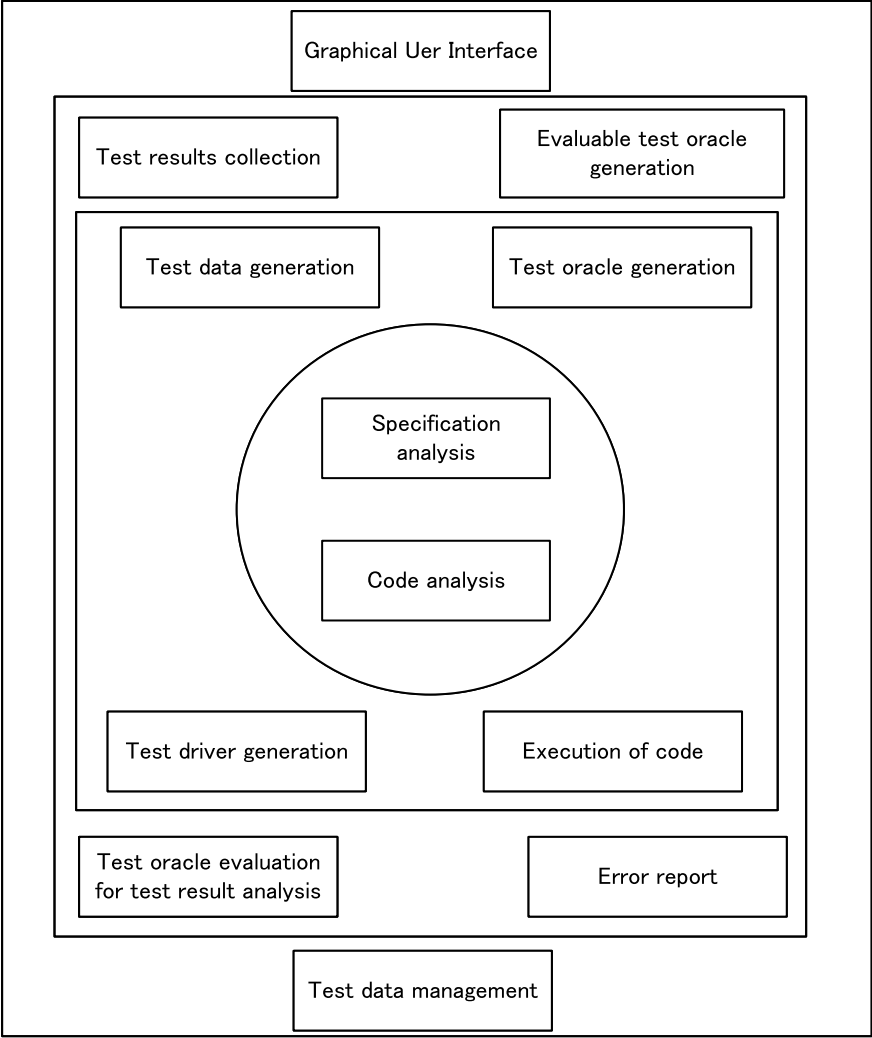


Fig. 8.2 General architecture of software tools for FSBT

test. The function of evaluable test oracle generation is designed to substitute the code output values for the corresponding output variables involved in the test oracle to obtain a condition that can be evaluated to true or false provided that all the input variables are given the corresponding test values (i.e., test data). The function of test oracle evaluation is intended to assign test data to the corresponding input variables in the evaluable test oracle to finally carry out the evaluation of the test oracle. The result of the evaluation will be used to determine whether the test has found an error

or not. The function of error report is required to report all errors found in the testing in a comprehensible and systematic manner.

The outmost layer functions include *graphical user interface* (GUI) and *test data management*. The function of GUI is expected to provide an efficient means for interactions between the user and the tool, therefore, its usability is extremely important. The function of test data management is required to manage all data generated for testing, including specifications, codes, test data, test oracle, test drivers, traversed paths, test results, and errors reports.

8.5.5 Ultimate Goal of FSBT

While the previous discussions in this chapter are concentrated on automatic testing using the FSBT, the ultimate goal of the FSBT is much more ambitious. In addition to the automatic testing capability, the FSBT is also expected to deliver automatic debugging and elimination of bugs in the future. While these two functions have been recognized to be extremely difficult to achieve, making useful progresses in these areas would be feasible. One potential solution would possibly come from an appropriate integration of dynamic testing and static analysis of code based on formal specifications, but how to fulfil this goal is still an open problem.

Finally, when using the test data generated based on the specification syntax to test programs, the test data must be translated into the program language suitable for executing the program. While such a translation can be rather easily done by humans who are familiar with both the specification notation and the program language, it may require a considerable support in a software tool. The reason is that an abstract type of an input variable in the specification (e.g., set type) can be implemented by one of the several different concrete types (e.g., array, vector, list) in the program and such an implementation is unknown in advance. If the test cases need to be automatically translated into the program syntax suitable for running the program, the tool must prepare all of the possible solutions to deal with every possible choice of the concrete data structure.

References

1. S. Liu and S. Nakajima. Automatic test case and test oracle generation based on functional scenarios in formal specifications for conformance testing. *IEEE Transactions on Software Engineering*, 48(2):691–712, 2022.
2. T. J. Ostrand and M. J. Balcer. The Category-Partition Method for Specifying and Generating Functional Tests. *Communications of the ACM*, 31(6):676–686, 1988.
3. P. Ammann and J. A. Offutt. *Introduction to Software Testing*. Cambridge University Press, 2008.

4. S. Liu and S. Nakajima. A Vibration Method for Automatically Generating Test Cases Based on Formal Specifications. In *18th Asia-Pacific Software Engineering Conference (APSEC 2011)*, pages 73–80, HCM City, Vietnam, Dec. 5-8 2011. IEEE CS Press.

Chapter 9

Testing-Based Formal Verification



In general, testing can tell the presence of bugs but cannot tell their absence due to the fact that testing usually only uses selected data from the input domain to check the behaviors of software systems. This is an intrinsic limitation of testing compared with formal verification of code, under the assumption that the specification accurately reflects the user's perception of the requirements.

The FSBT technique introduced in the Chap. 8 can be used to help validation of both the specification and the corresponding code. This validation would help the developer correct the specification and the code, but due to the intrinsic limitation of testing, using the FSBT still cannot ensure that the code contains no bugs. If we can formally verify that the code is correct with respect to the specification, it will help strengthen our confidence in the correctness of the code. If such a verification can be done fully automatically, it will provide a significant help for software industry to save time and cost in verification.

As mentioned before, Hoare logic can be used for the formal verification of code but can hardly be applied easily by practitioners and automatically. The greatest difficulty lies in the process of deriving appropriate loop invariants. Is there any possibility to avoid this difficulty in utilizing Hoare logic for verification of code? The answer is positive since the author of this book has proposed a new technique called *testing-based formal verification* (TBFV) [1, 2] that integrates FSBT and Hoare logic for automatic formal verification of code and has developed the TBFV into a rather mature level with his student and colleagues [3, 4]. The TBFV technique can fulfil the goal of automatic verification, albeit with a compromised effect.

In this chapter, a unified TBFV technique derived from our research results before is introduced. Its principle and application procedure are first explained and its usage is then illustrated with examples.

9.1 The Principle of TBFV

Suppose $T \ \&\& \ D$ is one of the functional scenarios of process P and code C is the corresponding implementation. The essential principle of the TBFV suggests that a single test data t_d generated from test condition T be first used to run code C to obtain a traversed code path and then both the functional scenario and the path-related information are used to formally prove the correctness of the path. Compared with Hoare logic, the goal of the TBFV, which is to prove the correctness of traversed paths, is not as ambitious as formal verification due to the difficulty in achieving a full coverage of paths by testing, but it will significantly improve the efficiency, the quality, and the effectiveness of testing since it can automatically ensure the correctness of every tested path.

Specifically, this principle can be realized by the following steps:

Step 1: Generate a single test data t_d from T .

Step 2: Execute the code C with test data t_d .

Step 3: Collect the traversed path p .

Step 4: Form the functional scenario for path p , denoted by $T_p \ \&\& \ D_p$.

Step 5: Prove the theorem $T \wedge T_p \wedge D_p \Rightarrow D$ to determine the correctness of path p with respect to the functional scenario $T \ \&\& \ D$.

Theoretically, the functional scenario $T \ \&\& \ D$ in the specification defines a set of code paths. Using a single test data t_d generated from T to run the corresponding code C will obtain one path p of the path set defined by $T \ \&\& \ D$. Actually, path p is an instance of $T \ \&\& \ D$ and it has its own *path condition* (like a test condition) T_p and *state expression* (like a defining condition) D_p . If the behavior of p satisfies $T \ \&\& \ D$, its defining condition D_p , which defines the final state of the execution of C with t_d , must guarantee D under the assumption of T and its own path condition T_p , which is a conjunction of all the decision conditions occurring in path p . That is, the theorem $T \wedge T_p \wedge D_p \Rightarrow D$ must be valid in order to ensure the correctness of p with respect to $T \ \&\& \ D$.

The key point here is how to derive T_p and D_p for path p in order to form its functional scenario $T_p \ \&\& \ D_p$. There are two approaches for this based on the traditional symbolic execution techniques. Let's call them *backward symbolic execution* (BSE) and *forward symbolic execution* (FSE), respectively.

9.1.1 Backward Symbolic Execution (BSE)

Let x be the input and y the output of code C . Let p denote a path of C , which is a sequence of conditions and/or assignment statements. For instance, the following sequence of statements and conditions reflects the characteristics of a code path:

$$\begin{aligned}
v_1 &:= E_1(x), \\
b_1(v_1, x), \\
v_2 &:= E_2(v_1, x), \\
b_2(v_2, v_1, x), \\
&\dots \\
v_n &:= E_n(v_{n-1}, v_{n-2}, \dots, v_1, x), \\
b_n(v_n, v_{n-1}, \dots, v_1, x), \\
y &:= E(v_n, v_{n-1}, \dots, v_1, x),
\end{aligned}$$

The BSE starts by forming the initial pair of path condition and state expression:

$$\{ ; y' = E(v_n, v_{n-1}, \dots, v_1, x) \}$$

based on the last assignment statement (i.e., $y := E(v_n, v_{n-1}, \dots, v_1, x)$), where the braces $\{$ and $\}$ are used to enclose the pair of the path condition and the state expression separated by a semicolon. In this pair, the current path condition is empty and the current state expression is $y' = E(v_n, v_{n-1}, \dots, v_1, x)$. We use y' to represent the very lastly used output variable y in order to distinguish from the same variable y used in the middle of the path (e.g., any of the local variables v_1 to v_n can be y). If there are more than one output variable, we will do the same thing for all of them. Then, we take a backward order to derive previous pairs of path condition and state expression, respectively, by doing two things: (1) repeatedly make substitutions of v_n by $E_n(v_{n-1}, v_{n-2}, \dots, v_1, x)$, v_{n-1} by $E_{n-1}(v_{n-2}, \dots, v_1, x)$, ..., and v_1 by $E_1(x)$ in $E(v_n, v_{n-1}, \dots, v_1, x)$ until E contains only the input variable x , (2) incrementally form the conjunction of the conditions $b_n(v_n, v_{n-1}, \dots, v_1, x)$, $b_n(v_{n-1}, \dots, v_1, x)$, ..., and $b_1(v_1, x)$ as the path condition and meanwhile make the same substitutions in every derived current conjunction. Suppose we eventually derive the pair of path condition and state expression as follows:

$$\{B(x); y' = E(x)\},$$

then we will form the final pair of path condition T_p and defining condition D_p of path p as follows:

$$\{B(x); y = E(x)\},$$

where T_p is $B(x)$ and D_p is $y = E(x)$. If there are more than one input variable and output variable, the similar strategy can be taken to derive the defining condition of the path.

Note that all the basic statements that do not change the state of the code, such as *break*, *continue*, *println*, and *return*, will all be ignored during the symbolic execution since they do not contribute to the substitution process as mentioned above.

9.1.2 Forward Symbolic Execution (FSE)

The FSE is opposite to the BSE, starting from the first statement or condition of path p to repeatedly carry out the variable substitutions as mentioned above. This technique

is the same as the traditional symbolic execution. Since the target to which the symbolic execution is applied is a single path, which is a sequence of basic statements and/or conditions, the symbolic execution should be much easier to perform. Again, we assume that code C has the same structure as given in the BSE section.

We start by forming the initial pair of path condition and state expression:

$$\{ ; v_1 = E_1(x) \}$$

based on the first assignment statement (i.e., $v_1 := E_1(x)$), where the current path condition is empty and the current state expression is $v_1 = E_1(x)$. Then, we take a forward order to derive pairs of path condition and state expression by doing one thing: incrementally form the conjunction of the conditions $b_1(v_1, x)$, $b_2(v_2, v_1, x)$, ..., $b_n(v_n, v_{n-1}, \dots, v_1, x)$ together with the conjunction of all the equations resulting from the corresponding assignment statement and meanwhile repeatedly make substitutions of v_1 by $E_1(x)$, v_2 by $E_2(E_1(x), x)$, ..., v_n by $E_n(E_{n-1}(x), E_{n-2}(x), \dots, E_1(x), x)$ in every derived current path condition and the right hand side of every equation involved in the state expression until the final state expression $y = E(v_n, v_{n-1}, \dots, v_1, x)$ is properly handled. Note that the rules for symbolic execution to derive state expressions in the traditional symbolic execution techniques should be applied in above derivation process. Ultimately, we can derive the final pair of path condition and state expression as follows:

$$\{ B(x); y = E(x) \},$$

where T_p is $B(x)$ and D_p is $y = E(x)$. If there are more than one input variable and output variable, the similar strategy can be taken to derive the pair of path condition and state expression.

Similar to the BSE, all the basic statements that do not change the state of the code, such as *break*, *continue*, *println*, and *return*, will all be ignored during the symbolic execution.

9.1.3 Theorem Proving

Another important issue is how to prove the theorem $T \wedge T_p \wedge D_p \Rightarrow D$ to determine the correctness of path p . There are two ways to do so. One is *formal proof* and the other is *predicate-based testing*.

Formal proof of the theorem is a process of deducing the conclusion D from the hypothesis $T \wedge T_p \wedge D_p$ based on well-established axioms and rules in predicate logic. This process can only be fully automated using a mature theorem prover if the theorem is rather simple. Even if the theorem cannot be automatically proved, the proof is feasible to conduct manually and can be much easier than using Hoare logic because the difficulty in deriving loop invariants is avoided. Note that since D_p may involve two states (e.g., z and $\sim z$, where z denotes the final value of the external variable z and $\sim z$ the initial value of z), we must keep consistency in using the initial value and the final value of every external variable involved when forming the

theorem $T \wedge T_p \wedge D_p \Rightarrow D$. To this end, all the external variables (e.g., z) occurring in T , T_p , and D_p must be changed to the decorated variables (e.g., $\sim z$).

To ensure a full automation of the proving process in general, we will have to use predicate-based testing approach, although it is weaker in effect than formal proof theoretically.

The essential idea of the predicate-based testing for proving the theorem $T \wedge T_p \wedge D_p \Rightarrow D$ is to generate test data, say t , that satisfies the hypothesis $T \wedge T_p \wedge D_p$ and then use t to evaluate the conclusion D . If $T \wedge T_p \wedge D_p(t)$ is true and $D_p(t)$ is false, it will prove that the theorem is not valid. The challenge is that it cannot tell whether the theorem is invalid or valid when $T \wedge T_p \wedge D_p(t)$ and $D_p(t)$ are both true for all the possible test data used. To find the truth, more test data must be generated and used, but if the evaluation situation remains the same all the time, no conclusion can be made theoretically. However, in practice, if the number of the generated test data is sufficiently great for specific applications, the conclusion of the validity of the theorem could be made, although certain risk will remain. Determining the level of the risk will require engineering judgement of the practitioners.

9.2 Examples

In this section, we use two examples to explain how the BSE and the FSE work in practice, respectively. Both strategies can be applied to large scale program paths, but to help the reader understand the principles of the strategies easily, simple examples are deliberately used.

9.2.1 Buy Railway Ticket

A process called *BuyTicket* is intended to define the function of buying a railway ticket. The price for passengers with different age is different. The passenger under twelve years old can enjoy 50% discount while the one over twelve years old will have to pay the full price. The formal specification of the process is given below.

```

process BuyTicket(age: int, normal_price: real)
    final_price: real
FSF:
    age > 12 and normal_price >= 1 &&
    final_price = normal_price
||
    age >= 0 and age <= 12 and normal_price >= 1 &&
    final_price = normal_price * 0.5
||
    others: undefined
end_process

```

This process describes three functional scenarios:

- (1) $age > 12$ and $normal_price \geq 1$ &&
 $final_price = normal_price$,
- (2) $age \geq 0$ and $age \leq 12$ and $normal_price \geq 1$ &&
 $final_price = normal_price * 0.5$,
- (3) *others: undefined*,

where *undefined* is a keyword used in Agile-SOFL to mean that the defining condition corresponding to the test condition *others* is not specified. It will play two possible roles. One is to remind the developer to replace it with a specific definition anytime appropriate later in the development project. Another possible role is to tell the programmer that anything sensible can be implemented when the test condition *others* is true, if *undefined* in the functional scenario is not replaced with any specific defining condition. Note that the keyword *undefined* can only be used as a defining condition in a functional scenario; *it cannot be used as a test condition*.

Suppose the process is implemented by the method *BuyTicket* of a class in Java given below.

```
double BuyTicket(int age, int normal_price) {
(1)   If (age >= 0 && normal_price >= 1) {
(2)     if (age > 12)
(3)       {final_price := normal_price;}
(4)     else {final_price := normal_price;
(5)       final_price := normal_price ** 2 - final_price * 0.5;}
(6)     return final_price; }
(7)   else System.out.println("The age or normal price
      is not appropriate."); }
```

To avoid confusions, in this code we use the syntax $:=$ rather than $=$ for the assignment operator. We also give a line number (e.g., (2)) to each statement and deliberately include a faulty statement

$$final_price = normal_price ** 2 - final_price * 0.5$$

to facilitate our discussions about this code, where $normal_price ** 2$ means $normal_price$ squared. The correct assignment should be

$$final_price = final_price * 0.5$$

in accordance with the specification.

Having the specification and the implemented code of the process *BuyTicket*, we can now discuss how the BSE and the FSE can be applied, respectively.

9.2.1.1 Application of BSE

Let us focus on functional scenario (2) in the specification. Suppose we generate a test data (i.e., two values for the two input variables, respectively)

age = 5 and *normal_price* = 10

based on the test condition (i.e., satisfying the test condition)

age >= 0 and *age* <= 12 and *normal_price* >= 1

of the functional scenario. Using this test data to execute the code, we obtain the code path denoted abstractly by [(1), ~(2), (4), (5), (6)] where ~(2) represents the negation of the condition given in line (2). The concrete content of this path is as follows:

```
age >= 0 && normal_price >= 1;
age <= 12;
final_price := normal_price;
final_price := normal_price ** 2 - final_price * 0.5;
return final_price.
```

Since the statement

```
return final_price
```

does not change the state of the code, we remove it from the path. As a result, we obtain the path

```
age >= 0 && normal_price >= 1;
age <= 12;
final_price := normal_price;
final_price := normal_price ** 2 - final_price * 0.5
```

to which the BSE technique can be applied. The derivation process and the result is shown below.

```
{age >= 0    and    normal_price >= 1    and    age <= 12;
final_price' = normal_price ** 2 - normal_price * 0.5}
age >= 0 && normal_price >= 1;
{age <= 12; final_price' = normal_price ** 2 - normal_price * 0.5}
age <= 12;
{ ; final_price' = normal_price ** 2 - normal_price * 0.5}
final_price = normal_price;
{ ; final_price' = normal_price ** 2 - final_price * 0.5}
final_price = normal_price ** 2 - final_price * 0.5
```

Having obtained the final path condition and the state expression, which is given at the top of the derivation sequence above, we can form the theorem for proving the correctness of the traversed path. But to ensure the consistent use of the output variable, *final_price*, in the theorem, we must first change the decorated variable *final_price'* in the derived state expression to *final_price* since *final_price'*

actually means the same value represented by the variable *final_price* in the defining condition, $final_price = normal_price * 0.5$, of the corresponding functional scenario (i.e., scenario (2)). Specifically, we will get the following path condition and state expression ready for forming the theorem:

$$\{age \geq 0 \quad \text{and} \quad normal_price \geq 1 \quad \text{and} \quad age \leq 12; \\ final_price = normal_price * 2 - normal_price * 0.5\}.$$

Now we can form the theorem for proving the correctness. Since the test condition $age \geq 0$ and $age \leq 12$ and $normal_price \geq 1$ of the functional scenario is equivalent to the derived path condition $age \geq 0$ and $normal_price \geq 1$ and $age \leq 12$, the theorem is given below in a simplified form by removing the derived path condition from the hypothesis of the theorem. That is,

$$age \geq 0 \text{ and } age \leq 12 \text{ and} \\ normal_price \geq 1 \text{ and} \\ final_price = normal_price * 2 - normal_price * 0.5 \Rightarrow \\ final_price = normal_price * 0.5.$$

Apparently, since the condition $final_price = normal_price * 2 - normal_price * 0.5$ in the hypothesis does not always guarantee the conclusion $final_price = normal_price * 0.5$, this theorem is proved to be invalid. This implies that a bug exists in the path.

9.2.1.2 Application of FSE

The only difference between the BSE and the FSE lies in their way to derive the final pair of path condition and state expression. As discussed previously, the BSE carries out the derivation process from the last assignment statement to the first statement or condition of the path under verification while the FSE performs the derivation task from the first assignment statement or condition of the path to the last assignment statement.

Applying the FSE to the same functional scenario, test data, and traversed path, we can have the following derivation process and result:

$$\begin{aligned} &age \geq 0 \ \&\& \ normal_price \geq 1; \\ &\{age \geq 0 \text{ and } normal_price \geq 1; \} \\ &age \leq 12; \\ &\{age \geq 0 \text{ and } normal_price \geq 1 \text{ and } age \leq 12; \} \\ &final_price := normal_price; \\ &\{age \geq 0 \text{ and } normal_price \geq 1 \text{ and } age \leq 12; \quad \quad \quad final_price = \\ &normal_price\} \\ &final_price := normal_price * 2 - final_price * 0.5 \\ &\{age \geq 0 \text{ and } normal_price \geq 1 \text{ and } age \leq 12; \quad \quad \quad final_price = \\ &normal_price * 2 - normal_price * 0.5\}. \end{aligned}$$

This derivation process has two specific differences from the derivation process and result of applying the BSE. One is that the derived path condition and state expression are given after each decision condition or statement on the path to indicate that the derived condition and state expression are the result of executing the decision condition or statement of the path. The other difference is that there is no need to create a decorated output variable like *final_price'* to denote the final state of the path since no confusion occurs.

Apparently, the derived condition and state expression are the same as that of applying the BSE. Therefore, the same theorem can be formed and its validity can be verified using the same technique as discussed previously.

9.2.2 Search an Element

The reader might have noticed that the example of *BuyTicket* is rather simple because its implementation does not involve any iteration statement and compound data structures like array. In this subsection, we use a little more complicated example involving iteration statement and an array to explain the ways of applying the BSE and the FSE.

Searching a given element in a given sequence is a common operation in computing. Suppose the specification of a process called *Search* is given below.

```
process Search(x: int, list: seq of int) pos: int
FSF:
  exists[i : inds(list)] | list(i) = x &&
  pos = hd([i | i: inds(list) & list(i) = x])
  ||
  x notin elems(list) && pos = -1
end_process.
```

This process defines two functional scenarios:

- (1) $exists[i : inds(list)] \mid list(i) = x \ \&\&$
 $pos = hd([i \mid i : inds(list) \ \& \ list(i) = x]),$
- (2) $x \ notin \ elems(list) \ \&\& \ pos = -1.$

Functional scenario (1) states that if x is an element of the sequence $list$, the output pos will represent the position of x in its first appearance on $list$. Scenario (2) tells that if x is not an element of the $list$, pos will be equal to -1 , indicating the fact that x does not appear on the $list$.

Assume that the process is implemented by the code as follows:

```
int Search(int x, int list[]) {
  int pos := -1;
  int k := 0;
  while (k < list.length) {
```



```

if (list[k] == x) {
  pos := k;
  break;
} else {
  k := k + 1;
}
return pos;
}.

```

Obviously, this code involves a while loop and an array list. Let us focus on functional scenario (1) mentioned above. Suppose we generate a test data that comprises two values for the two input variables x and $list$:

$x = 9, list = [3, 9, 20]$.

Using this test data to run the code, we will obtain the following path:

```

pos := -1;
k := 0;
k < list.length
list[k] <> x;
k := k + 1;
k < list.length;
list[k] == x;
pos := k;
break;
return pos;

```

where we use the symbol $< >$ to denote inequality in order to keep the consistency with the syntax in Agile-SOFL for the convenience in theorem formation and proving. Since both break and return pos are two statements that do not change the state of the code, we remove them and obtain the following path suitable for applying the BSE and the FSE:

```

pos := -1;
k := 0;
k < list.length
list[k] <> x;
k := k + 1;
k < list.length;
list[k] == x;
pos := k.

```

9.2.2.1 Applying BSE

Applying the BSE, we obtain the derivation process and result:

```

{0 < list.length and list[0] <> x and 0 + 1 < list.length and list[0 + 1] =
x;   pos' = 0 + 1}
pos := -1;
{0 < list.length and list[0] <> x and 0 + 1 < list.length and list[0 + 1] =
x;   pos' = 0 + 1}
k := 0;
{k < list.length and list[k] <> x and k + 1 < list.length and list[k + 1] =
x;   pos' = k + 1}
k < list.length
{list[k] <> x and k + 1 < list.length and list[k + 1] = x; pos' = k + 1}
list[k] <> x;
{k + 1 < list.length and list[k + 1] = x; pos' = k + 1}
k := k + 1;
{k < list.length and list[k] = x; pos' = k}
k < list.length;
{list[k] = x; pos' = k}
list[k] == x;
{ ; pos' = k}
pos := k.

```

Note that the assignment statement, $\text{pos} := -1$, does not change the derived state expression at the top of the derivation process because pos (not pos') does not appear in the current state expression $\text{pos}' = 0 + 1$.

Having the derived path condition $0 < \text{list.length}$ and $\text{list}[0] \neq x$ and $0 + 1 < \text{list.length}$ and $\text{list}[0 + 1] = x$ and the state expression $\text{pos} = 0 + 1$ (the predicate resulting from substituting pos for pos' in $\text{pos}' = 0 + 1$), we can now form the theorem for correctness proving:

```

exists[i: inds(list)] | list(i) = x and
0 < list.length and list[0] <> x and
0 + 1 < list.length and list[0 + 1] = x and
pos = 0 + 1 ⇒
pos = hd([i | i: inds(list) & list(i) = x]).

```

The proof of this theorem is quite straightforward. From the hypothesis, we know that $1 < \text{list.length}$, $\text{list}[0] \neq x$, and $\text{list}[1] = x$. This implies that the sequence $[i \mid i: \text{inds}(\text{list}) \ \& \ \text{list}(i) = x]$ will have the index of x as its first element (i.e., the head of the sequence). Therefore, $\text{hd}([i \mid i: \text{inds}(\text{list}) \ \& \ \text{list}(i) = x]) = 1$ holds. From the hypothesis we also know $\text{pos} = 1$, therefore, the conclusion $\text{pos} = \text{hd}([i \mid i: \text{inds}(\text{list}) \ \& \ \text{list}(i) = x])$ holds. This completes the proof of the theorem, indicating that the path is correct with respect to the corresponding functional scenario.

9.2.2.2 Applying FSE

Applying the FSE to the same functional scenario, test data, and traversed path, we obtain the following derivation process and result:

```

pos := -1;
{ ; pos = -1 }
k := 0;
{ ; pos = -1 and k = 0 }
k < list.length
{ 0 < list.length; pos = -1 and k = 0 }
list[k] <> x;
{ 0 < list.length and list[0] <> x; pos = -1 and k = 0 }
k := k + 1;
{ 0 < list.length and list[0] <> x; pos = -1 and k = 0 + 1 }
k < list.length;
{ 0 < list.length and list[0] <> x and 0 + 1 < list.length;
pos = -1 and k = 0 + 1 }
list[k] == x;
{ 0 < list.length and list[0] <> x and 0 + 1 < list.length and list[0 + 1] =
x; pos = -1 and k = 0 + 1 }
pos := k
{ 0 < list.length and list[0] <> x and 0 + 1 < list.length and list[0 + 1] =
x; pos = 0 + 1 and k = 0 + 1 }.

```

To improve the readability, the derived path condition $0 < \text{list.length}$ and $\text{list}[0] \neq x$ and $0 + 1 < \text{list.length}$ and $\text{list}[0 + 1] = x$ and the state expression $\text{pos} = 0 + 1$ and $k = 0 + 1$ are simplified into

```

{ 0 < list.length and list[0] <> x and 1 < list.length and list[1] = x;
pos = 1 and k = 1 }.

```

We can now form the following theorem for the correctness proof:

```

exists[i : inds(list)] | list(i) = x and
0 < list.length and list[0] <> x and
1 < list.length and list[1] = x and
pos = 1 and k = 1 ⇒
pos = hd([i | i : inds(list) & list(i) = x]).

```

Apparently, this theorem is equivalent to the one given in the previous subsection since variable k involved in the hypothesis does not appear in the conclusion and all the other parts of the theorem are the same as the one given previously.

9.3 Practical Issues

When the TBFV is applied in practice, several issues need to be handled properly. The first issue is how to keep symbols consistent in the theorem for proving the correctness of the path. Since the path condition and the state expression are derived based on the conditions or assignment statements on the path, they use the syntax of the programming language in which the code is written. However, in the theorem both the elements of the functional scenario (i.e., test condition and defining condition) and the elements of the path (i.e., path condition and state expression) are involved. Since the syntax of the scenario elements is usually different from that of the programming language, the theorem may not be expressed consistently in syntax. To resolve this problem, the path condition and the state expression should first be transformed to equivalent ones in the syntax of the specification language before the theorem is formed. Of course, a software tool may be built to help the transformation task.

The second issue is how to keep the variable names consistent in expressing the path traversed by a test data. In general, a code is written in a modular manner, therefore, it is possible to encounter subroutine calls when a path is traversed. A subroutine is an operation in general but may be called differently in different programming languages. For example, a subroutine refers to a function in C language but a method of a class in Java or similar object-oriented programming languages. When a subroutine call is encountered during the execution of a path, the execution will go into the subroutine algorithm in its definition in order to obtain the corresponding path content (i.e., the traversed conditions or statements). However, since the subroutine may have some local variables (and possibly subroutine parameters as well) whose names are the same as the ones declared in the calling subroutine, it is possible for the local variables to appear on the traversed path ultimately obtained that includes other variables with the same name. This would cause incorrect result of symbolic executions of the traversed path. To prevent such confusions, certain measures must be taken to distinguish the local variables in the called subroutine from the variables with the same name in the calling subroutine (or the current code).

Another issue is concerned with the limitation of TBFV in ensuring the correctness of the whole code. TBFV can only be used to ensure the correctness of traversed paths, that is, if a test data is used to test the code, the traversed path, as a result of the execution, can be ensured by TBFV to be correct with the corresponding functional scenario. For the paths that have never been tested, their correctness cannot be guaranteed by TBFV. The more paths are tested, the higher the possibility of ensuring the correctness of the whole code will become. The V-Method for testing introduced in the previous chapter can be utilized to explore more paths.

The final issue is about finding bugs in the path under verification. If the theorem for its correctness is proved to be invalid, that will imply the existence of bugs in the path. The challenge is how to efficiently determine the location and the nature of the bugs. This is still an open problem, but perhaps keeping verifying the theorem in every step of the derivation of the path condition and the state expression using the BSE or the FSE can provide a partial solution. More research needs to be done in this area.

References

1. S. Liu. Utilizing Hoare Logic to Strengthen Testing for Error Detection in Programs. In *Proceedings of The Turing Centenary Conference*, pages 229–238, Manchester, UK, June 2012. EPiC Series.
2. S. Liu. Testing-Based Formal Verification for Theorems and Its Application in Software Specification Verification. In *Proceedings of 10th International Conference on Tests and Proofs (TAP 2016)*, pages 112–129. LNCS 9762, Springer, July 2016.
3. R. Wang, S. Liu, and Y. Sato. SIT-SE: A Specification-based Incremental Testing with Symbolic Execution. *IEEE Transactions on Reliability*, 70(3):1053–1070, September 2021.
4. A. Liu and S. Liu. Enhancing the Capability of Testing-Based Formal Verification by Handling Operations in Software Packages. *IEEE Transactions on Software Engineering*, 49(1):304–324, January 2023.

Chapter 10

Software Project Management



Software development methods and techniques alone can hardly guarantee the success of a software project, simply because software projects are human activities. How humans apply the methods and techniques may produce quite different results. Therefore, software management becomes a critical factor contributing to the success of software projects.

Software management is an important activity to decide how people in software projects can be properly organized to work together and what strategies and techniques should be taken to fulfil the required development tasks. In this chapter, we discuss several strategies and specific techniques for project management when Agile-SOFL is applied. Some of the notions in our proposals may differ significantly from traditional ones.

10.1 “Total Football” Method

Total football is a tactical system for playing football games whose foundation was laid by English football player and coach Jimmy Hogan. This system became famous when it was adopted by Netherlands national team in 1970s. In total football every player can dynamically take over the role of any other player in a team to act as a defender, midfielder, or attacker. This essential idea can be effectively applied to software project management, especially when Agile-SOFL is employed.

Agile-SOFL advocates the strategy of writing a hybrid specification first and then carrying out incremental implementation during which inspection and testing are performed for quality assurance in a software project. If traditional management approaches are adopted to manage the project, many potential problems will occur, which will pose a threat to the quality of the software developed eventually. The traditional management approaches adopted in software companies is characterized by the separation of the roles of developers in a project team. The developer called *analyst* is responsible for requirements analysis and specification.

The specification is then delivered to another developer called *designer* for designing a software system as a solution. After the design is completed, it is given to another developer called *programmer* to construct the code for implementation. Of course, many companies may just separate analyst and programmer where the programmer also acts as a designer.

This kind of approach faces two challenges, at least. First, the understanding of the user's requirements gained by the analyst through the writing of the specification will have to be regained by the designer or the programmer. This would inevitably prolong the time and increase the cost for the project. Second, due to whatever reasons (e.g., ambiguous language or inappropriate expressions are used), it is likely that the specification is misinterpreted by the designer or the programmer, and the original user's requirements may also be understood incorrectly if mistakes are made in the specification, which often happens. To deal with this problem, the "Total Football" method, *TF-Method* for short, can be adopted to effectively and efficiently apply Agile-SOFL.

In the TF-Method, we only have developers who will carry out requirements analysis and specification, design, and coding. In other words, the same developer will perform all the jobs necessary for constructing a software system. The developer can efficiently utilize the knowledge learned from the requirements analysis and specification in designing and implementing the system. This would significantly reduce the efforts, cost, and mistakes.

Some companies may be concerned about the feasibility of the TF-Method because conducting different activities by the same developer would require him or her to have a strong capability. While this concern is understandable, it is actually not a problem since specifications in Agile-SOFL share many common characteristics, albeit at an abstract level, with programming languages, which allows developers with experiences of programming languages to easily understand and work on specifications and code.

10.2 Novel View on the Role of Documents

Traditionally a document produced in an early phase of software development is generally regarded as the foundation for the subsequent documents to be produced. For instance, a requirements specification should be the foundation for design and a design document should be the base for coding. This view has been shared by many practitioners who are using waterfall model, model-driven development model, or formal methods. However, in the reality of practical software development the functions and data items defined in an early-produced document (e.g., specification) may often need to be changed in the late-produced document (e.g., code). Since the early-produced document is believed to serve as the foundation for the late-produced document, developers often believe that the early-produced document must be evolved to keep consistency with the late-produced document. This idea is especially strongly reflected in the refinement technique of formal methods. Since this kind of configu-

ration management task usually consumes a lot of time, many developers in practice may not be positive to work on it.

In the Agile-SOFL method, we employ a novel view on documents (i.e., various specifications and code) produced at different stages. The early-produced document only reflects the current understanding of the developer about the system under construction. It can be ambiguous or precise, incomplete or complete, inaccurate or accurate, inappropriate or appropriate in expressions. The late-produced document should be a clarification of the early-produced document in all those aspects and it is the developer's responsibility to decide how the clarification can be done. For example, the informal specification provides ambiguous descriptions of desired functions, data items, and constraints, but they will be clarified into a semi-formal specification in which the original informal descriptions are more precisely interpreted and defined. The formal specification of some processes will further clarify the meaning of the semi-formal specification. Finally, the code will provide an ultimate clarification of the hybrid specification.

The important thing in such a clarification process is to treat all the early-produced specifications before coding as temporary, auxiliary documents, which do not have authoritative power but with a reference value, for producing subsequent documents. This means that a specification or code produced at a later time needs to take the relevant early-produced specification into account but does not need to strictly conform to it logically; necessary changes in functions, data items, and constraints can be made properly if they are desirable with respect to the user's perception of requirements. If such changes occur in the late-produced document, there is *no* need to modify the early-produced specification for configuration management purpose, but a traceability link between the elements of the early-produced document and the corresponding ones in the late-produced document is advocated to establish to facilitate the understanding of the "historical details" of developing the ultimate code from scratch or beginning. The finally produced code with comprehensible comments will be used as the *only* document to define the behaviors of the system and to be delivered to the end-user. All the other auxiliary specifications can be kept only as a reference for software validation, verification, and maintenance.

10.3 Management for Outsourced Development

Outsourcing has become one of the main ways to develop software systems in industry to help reduce the cost, but it may face a threat in software quality assurance. It seems that the threat mainly comes from outsourcing companies misunderstanding the requirements of the outsourced company. To effectively overcome this threat in software projects using Agile-SOFL, we can require the outsourcing company to first clarify the requirements of the outsourced company. Such a clarification activity can not only help the outsourcing company accurately learn the requirements, but also provide a solid foundation for constructing the code efficiently and correctly.

Specifically, the outsourced company should first write a rather detailed informal specification for the outsourcing company. To understand the functions, data items, and constraints described in the informal specification, the outsourcing company must write a hybrid specification first. During the writing process, the developers in the outsourcing company will need to communicate with the outsourced company to learn the requirements and clarify ambiguities in the descriptions of the informal specification. The resulting hybrid specification will be able to serve as a solid reference for implementation and validation of the code developed ultimately as well. This approach would be especially effective for international outsourcing where the developers in the outsourcing company and the ones in the outsourced company may speak different natural languages.

10.4 Flexibility in Using Techniques

The Agile-SOFL method offers a systematic way to develop software systems, but it may not completely adoptable by some companies for many reasons. Actually, companies do not necessarily adopt the whole Agile-SOFL techniques, instead, they can pick up some of the techniques involved in the Agile-SOFL to integrate them into their own development approaches. There are several possibilities.

The first possibility is to only use the informal specification since the informal specification of Agile-SOFL provides a rather effective mechanism and structure to help developers capture and define the user's requirements. If a company is using the existing agile development paradigm in software projects, it might be a good idea to integrate the Agile-SOFL informal specification into the agile development process. Specifically, a rather detailed informal specification can be written first and then the traditional agile development can be carried out accordingly.

For the companies working on safety-critical systems or other kinds of critical systems (e.g., security-critical systems), software quality is of paramount importance. In this case, the whole Agile-SOFL can be adopted. If CDFDs do not fit into the development context in the company, they can be replaced with explicit specifications of Agile-SOFL or even with pseudocode. If effective, only the three-step hybrid specification approach can be utilized together with traditional inspection and testing activities the company has been practicing.

Finally, training developers to have a rigorous mind is considerably helpful for the enhancement of software productivity and quality. To this end, teaching formal specification techniques and the TBFV method, in a comprehensible fashion, would be effective because learning and practicing formal specification and test-based formal verification techniques can help developers build up the habit of thinking rigorously. This habit will likely to play a positive role in software quality assurance when the developer performs development tasks.

The Agile-SOFL method offers an approach to bridging agile with formal engineering to enhance both software productivity and reliability, but its potential benefits can be well enjoyed by practitioners only when it is integrated properly with effective management strategies. If you want to experience the “taste” of the Agile-SOFL, you will have to “bite” into it.